

Universidad de Oviedo

ESCUELA POLITÉCNICA DE INGENIERÍA DE GIJÓN

GRADO EN CIENCIA E INGENIERÍA DE DATOS

ÁREA DE ESTADÍSTICA E INVESTIGACIÓN OPERATIVA

Automatización de flujos predictivos mediante Agentes IA

Mario Vallina García

TUTORES:

Agustina Bouchet Gutiérrez

Emilio Torres Manzanera

FECHA: Julio de 2026

Índice

1. Introducción	5
1.1. Motivación	5
1.2. Objetivos	6
1.3. Alcance del trabajo	9
1.4. Estructura del trabajo	11
2. Conceptos básicos	14
2.1. MLOps	14
2.2. Sistemas basados en LLMs y agentes	15
2.3. JSON como contrato de entrada del pipeline	16
2.4. API de OpenAI como capa de acceso al modelo	17
2.5. LangGraph: orquestación mediante grafos de estado	18
2.6. Patrón ReAct y agentes con herramientas	19
2.7. Human-in-the-Loop en sistemas basados en IA generativa	20
2.8. Principio de restraint agéntico: cuándo no usar agentes	21

2.9. Herramientas auxiliares del ciclo MLOps	21
3. Estudio de alternativas	25
3.1. Alternativas a LangGraph como framework de orquestación	28
3.2. Alternativas al proveedor del modelo de lenguaje	30
3.3. Alternativas al almacén de experiencias	34
3.4. Alternativas al stack de interfaz de usuario	37
4. Arquitectura desarrollada	39
4.1. Visión general del sistema	39
4.2. Contrato de entrada del pipeline	42
4.3. Tipos de nodos y contratos de ejecución	45
4.4. Descubrimiento de joins en el agente de validación	53
4.5. Patrón controlador determinista y agentes especializados	56
4.6. Seguimiento experimental y registro de modelos con MLflow	60
4.7. Experience Pool como memoria empírica del sistema	61
4.8. Flujo de ejecución end-to-end del pipeline	66

5. Resultados y análisis	69
5.1. Contexto y dependencias de los resultados	69
5.2. Diseño de la evaluación experimental	70
5.3. Composición y validación inicial del Experience Pool	71
5.3.1. Experiencias de predicción temporal	72
5.3.2. Experiencias de clasificación y regresión	75
5.4. Comportamiento agéntico en ejecuciones de forecasting	78
5.4.1. Recuperación de experiencias y razonamiento del planificador	83
5.4.2. Validación de datos y descubrimiento de joins	84
5.4.3. Escalabilidad del coste agéntico frente al tamaño del dataset	86
5.4.4. Variabilidad del coste agéntico y control del contexto	88
5.5. Robustez ante errores controlados	90
5.6. Desglose de coste por nodo	92
5.7. Supervisión humana en las puertas HITL	96
6. Conclusiones y líneas futuras	98
Bibliografía	101

Anexo A. Reproducción y despliegue de la aplicación	107
A.1. Requisitos previos y variables de entorno	107
A.2. Ejecución gratuita con GitHub Models	108
A.3. Despliegue con Docker	109
A.4. Sembrado del pool de experiencias	110
A.5. Verificación	111
 Anexo B. Procedencia de los conjuntos de datos	 112

1. Introducción

1.1. Motivación

Este Trabajo Fin de Grado surge de la experiencia adquirida durante unas prácticas realizadas en una empresa del sector químico, en las que se participó en el diseño de sistemas de inteligencia artificial aplicados a problemas reales de empresas industriales. Uno de los proyectos abordados consistía en construir un sistema de predicción de precios de materias primas, en el que cada decisión de modelado –selección de algoritmos, tratamiento de variables exógenas, estrategia de validación, criterios de promoción a producción– requería horas de trabajo manual repetitivo por parte de los científicos de datos. Esa observación sobre el terreno, sostenida durante meses, fue la chispa concreta que motivó este trabajo: ¿hasta qué punto es posible automatizar la parte “rutinaria” del desempeño del experto sin sacrificar trazabilidad ni rigor metodológico?

Desplegar modelos predictivos en producción exige articular un flujo completo que incluye ingestión de datos, limpieza, análisis, entrenamiento, evaluación, selección, versionado, despliegue, monitorización y mantenimiento. En este contexto surgen las operaciones de aprendizaje automático (MLOps, del inglés *Machine Learning Operations*), una disciplina orientada a gobernar el ciclo de vida del aprendizaje automático desde el desarrollo del modelo hasta su operación continua en entornos productivos [1].

Sin embargo, los escenarios reales mantienen todavía una brecha relevante: numerosas decisiones de integración, validación y operación dependen de intervención manual, conocimiento experto y resolución ad hoc de incidencias. La experiencia adquirida durante las prácticas permitió observar que los equipos industriales suelen enfrentarse a tres dificultades recurrentes: datasets que llegan fragmentados en varios ficheros con esquemas inconsistentes, conocimiento empírico sobre qué modelos funcionan en qué contextos que permanece disperso, y ciclos de iteración demasiado largos cuando hay que probar nuevas series tempo-

rales con horizontes y frecuencias distintas. Estas tres fricciones limitan la reproducibilidad del proceso, incrementan el esfuerzo técnico necesario y dificultan la estandarización de los flujos predictivos en contextos industriales.

Este trabajo plantea la automatización de dicho flujo mediante un enfoque basado en agentes de IA, con la ambición de aportar una respuesta concreta a esa problemática. Frente a una automatización puramente determinista, la propuesta aprovecha la capacidad de los modelos de lenguaje de gran tamaño (LLM, del inglés *Large Language Model*) para interpretar el estado del proceso, planificar pasos y ejecutar acciones mediante herramientas, con el fin de construir una automatización flexible, trazable y adaptable. La empresa cliente planteó dos objetivos principales: una demostración técnica de que el patrón multi-agente podía aplicarse a un flujo MLOps real sin convertirse en una caja negra total, y un prototipo reproducible que le permitiese evaluar internamente si esta línea de trabajo merecía industrializarse. Ese es el trabajo que aquí se presenta.

El antecedente directo de esta línea de trabajo aparece en el aprendizaje automático automatizado (AutoML, del inglés *Automated Machine Learning*), que automatiza partes del pipeline como el preprocesado, la selección de modelos y el ajuste de hiperparámetros [2]. En una dirección complementaria, trabajos recientes describen una evolución de las prácticas operativas desde MLOps, centrado en modelos tradicionales, hacia las operaciones sobre modelos de lenguaje (LLMOps, del inglés *Large Language Model Operations*) y las operaciones sobre agentes (AgentOps, del inglés *Agent Operations*), donde el incremento de autonomía demanda nuevas capacidades de operación, observabilidad y control [3].

1.2. Objetivos

El objetivo principal de este trabajo es diseñar, implementar y validar un prototipo funcional de sistema MLOps multi-agente capaz de automatizar flujos de predicción de series temporales con variables exógenas de la clase que la empresa cliente utiliza para asistir su toma de decisiones operativas en el sector industrial. El encargo en sí, como se detalla en



el alcance del trabajo (sección 1.3), fue deliberadamente abierto. La empresa solicitaba explorar la aplicación de inteligencia artificial (IA) generativa a sus procesos analíticos, sin imponer ni arquitectura ni capacidades concretas. La traducción de ese encargo en un objetivo técnico —un sistema capaz de absorber la heterogeneidad de los datasets que llegan al equipo de ciencia de datos, proponer planes de entrenamiento razonados, ejecutarlos de forma reproducible y dejar trazabilidad suficiente para que un analista senior pueda auditar cada decisión a posteriori— es una decisión propia, fruto de la observación directa de los flujos del equipo durante las prácticas y de identificar dónde se concentraban las fricciones reales.

La finalidad no consiste en sustituir por completo el trabajo experto, sino en estudiar cómo los componentes agénticos pueden apoyar distintas fases del ciclo de vida de un modelo predictivo dentro de un contexto industrial concreto: desde la recepción y preparación inicial de los datos que envía el cliente, hasta la selección de un modelo candidato para su posterior promoción. La participación humana sigue siendo necesaria desde el inicio, ya que aporta la definición del problema, el esquema esperado de los datos, las restricciones técnicas y de negocio, y los criterios que permiten considerar válida una solución. El enfoque propuesto no elimina el criterio técnico y de dominio del analista que dialoga con el cliente, sino que busca reducir tareas repetitivas y apoyar la toma de decisiones dentro de un flujo controlado y trazable.

A partir de este objetivo principal, el trabajo persigue cuatro objetivos específicos. Primero, identificar qué fases del flujo MLOps de la empresa pueden beneficiarse de capacidades agénticas y cuáles deben permanecer bajo lógica determinista. Dicha distinción no es retórica. En un entorno donde los modelos van a alimentar decisiones operativas, delegar la ejecución del entrenamiento o la evaluación a un LLM es inaceptable. A partir de esta separación se plantea una arquitectura híbrida en la que los agentes intervienen en los puntos de mayor incertidumbre semántica —interpretar el contrato de entrada, recuperar experiencias previas, proponer joins entre tablas, redactar informes— mientras que las operaciones críticas de entrenamiento, validación y evaluación se ejecutan mediante código reproducible.

Segundo, construir una arquitectura multi-agente capaz de coordinar componentes es-



pecializados a lo largo del flujo, manteniendo un estado compartido entre fases, contratos explícitos de entrada y salida, y puntos de intervención humana en las decisiones de mayor impacto. El sistema debe organizar tareas como la validación inicial de datos, la planificación del entrenamiento, la ejecución de modelos candidatos, la evaluación de resultados y la promoción del modelo seleccionado, sin que esa coordinación dependa de un supervisor LLM cuya conducta sea difícil de auditar.

Tercero, implementar un flujo de entrenamiento y evaluación aplicable principalmente a predicción temporal con variables exógenas —el caso de uso central de la empresa cliente, asociado a precios de materias primas, indicadores macroeconómicos y variables de mercado relacionadas—, conservando además la posibilidad de ejecutar problemas tabulares de clasificación y regresión para demostrar la generalidad del sistema. En todos los casos, la selección de modelos no debe depender únicamente de una decisión manual, sino de un plan estructurado que combine información del dataset, conocimiento experto codificado y experiencias acumuladas de ejecuciones anteriores.

Cuarto, incorporar mecanismos de trazabilidad que permitan al equipo auditar cada ejecución: modelos candidatos evaluados, métricas obtenidas, artefactos generados, criterios de selección e información asociada a cada ejecución. Esta trazabilidad responde a una preocupación muy común en el ámbito profesional; la relevancia de intentar entender por qué se toman las decisiones. Ningún modelo promovido a producción debe ser una caja negra total, ni siquiera aunque el camino para construirlo haya pasado por un componente agéntico.

Conviene dejar claro desde el inicio que el objetivo de este Trabajo Fin de Grado no es construir el mejor modelo predictivo posible para el problema concreto que motivó el encargo, sino demostrar que el patrón multi-agente propuesto aporta valor verificable al flujo MLOps que lo rodea. El contexto industrial descrito en la motivación no opera como benchmark de validación experimental, sino como fuente de requisitos arquitectónicos: define el tipo de problema que el sistema debe poder procesar —forecasting univariante con múltiples exógenas heterogéneas, frecuencia regular, posible presencia de variables con disponibilidad futura conocida—, las puertas de control que el flujo necesita y los criterios de auditabilidad que las decisiones deben respetar.

1.3. Alcance del trabajo

El encargo inicial no llegó en forma de especificación cerrada con entregables prefijados, sino como una pregunta abierta por parte de la empresa. En sus propios términos, se trataba de «abrir nuevas vías para la aplicación de IA generativa en la industria química, facilitando el desarrollo de modelos predictivos sin necesidad de conocimientos avanzados en programación o estadística», con el fin de «evaluar la viabilidad de estas tecnologías en entornos reales y su potencial para mejorar la eficiencia y accesibilidad del análisis de datos». Este encuadre refleja una situación habitual en el tejido industrial actual: organizaciones que perciben que la IA generativa puede aportar valor a sus procesos, pero que no han identificado todavía dónde ni cómo aplicarla con criterio técnico. La empresa no solicitó un sistema concreto, sino la exploración del espacio del problema y la propuesta de una solución defendible.

A partir de ese encargo abierto, una parte sustantiva del trabajo consistió en acotar el problema. Tras analizar los flujos de análisis de datos del equipo y los puntos donde los analistas dedicaban más tiempo a tareas repetitivas, se identificó el cuello de botella en puntos como: interpretar datasets heterogéneos, decidir qué modelo probar primero, recordar qué funcionó en proyectos previos, justificar las decisiones de modelado y producir resultados auditables. Sobre esta lectura, se propuso a la empresa un enfoque concreto, es decir, un sistema multi-agente que automatizase el flujo de predicción end-to-end, manteniendo los componentes deterministas allí donde se exige reproducibilidad y delegando en agentes asistidos por LLM únicamente las fases con incertidumbre semántica. La elección del enfoque agéntico —en lugar de una interfaz visual tipo no-code, una herramienta AutoML clásica o un chatbot sobre datos— responde a esa lectura del problema: la accesibilidad solicitada no se resuelve eliminando complejidad visualmente, sino encapsulándola tras una capa que sepa razonar sobre el contexto.

La evaluación experimental del sistema, descrita en el Capítulo 5, no se realiza sobre los datos reales del cliente, sino sobre conjuntos públicos y datos sintéticos generados ad hoc que reproducen las propiedades estructurales del tipo de problema descrito: forecasting univariante con múltiples exógenas, frecuencia regular y posible presencia de variables

con disponibilidad futura conocida. Dicha decisión es metodológica y deliberada. El sistema, por diseño, puede procesar conjuntos de cualquier escala y complejidad: la arquitectura no impone restricciones sobre el tamaño del dataset, el número de exógenas o la longitud del histórico, y de hecho parte de esos experimentos estudian explícitamente cómo se comporta el coste agéntico cuando se varía la escala del problema. La razón para no utilizar la información del cliente en la evaluación es distinta y responde a una decisión de foco: lo que se evalúa en este trabajo es el comportamiento del sistema agéntico —calidad de la recuperación de ejecuciones anteriores, coherencia del planificador, robustez del controlador, coste y variabilidad de los nodos LLM, utilidad de las puertas humanas— y para medir ese comportamiento con limpieza se requieren conjuntos cuyas propiedades estructurales sean conocidas, controlables y reproducibles por cualquier lector. Un conjunto industrial real, con decenas de exógenas heterogéneas, requisitos de tratamiento específicos por columna y horizonte de despliegue propio, habría desplazado inevitablemente el foco hacia la ingeniería de características y la búsqueda de técnicas predictivas del estado del arte para ese caso concreto. Ese desplazamiento habría producido un trabajo distinto, en el que el sistema agéntico habría quedado como soporte de un proyecto de modelado en lugar de como objeto de estudio.

En cuanto a las capacidades concretas que el sistema pone sobre la mesa, el prototipo aborda tres familias de problemas predictivos: clasificación tabular, regresión tabular y predicción temporal. El caso de uso principal corresponde a predicción de series temporales (forecasting) con una única serie objetivo y múltiples variables exógenas, escenario representativo de los problemas que se encontrarían en el contexto industrial de origen —precios de materias primas, variables de mercado o consumos energéticos relacionados—. El prototipo incluye la construcción de un perfil descriptivo del problema, la validación inicial de datasets mediante un agente de validación con herramientas, la generación de planes de entrenamiento mediante un agente planificador asistido por memoria empírica, la ejecución determinista de distintos modelos candidatos, el ajuste de hiperparámetros, el registro de métricas y artefactos en MLflow, y la selección automática de un modelo campeón sometida a revisión humana antes de su promoción lógica. Asimismo, incorpora un Experience Pool sobre SQLite donde se conservan resultados de ejecuciones anteriores, configuraciones de entrenamiento y métricas obtenidas, lo que permite reutilizar conocimiento empírico co-

mo apoyo a la toma de decisiones en futuras ejecuciones. La interfaz web, desarrollada con Next.js y FastAPI, muestra el avance del pipeline en tiempo real e incorpora dos puertas de supervisión humana en los puntos críticos del flujo: la aceptación del dataset preparado y la promoción del modelo seleccionado.

En el caso específico de predicción temporal, el trabajo contempla el uso de variables exógenas y distingue entre aquellas conocidas a futuro —por ejemplo, calendarios de festividades o variables planificadas— y aquellas que deben estimarse antes de realizar la predicción. Tal distinción, frecuentemente descuidada en pipelines, permite evitar fugas durante la validación del modelo y mejora la fidelidad de la evaluación respecto a un escenario real de despliegue.

El sistema se ha validado como prototipo experimental, no como plataforma industrial completa. Quedan fuera de esta versión la monitorización productiva en tiempo real, el despliegue continuo sobre infraestructura cloud corporativa, la gestión avanzada de usuarios y permisos, y el gobierno de modelos a nivel organizativo. Tampoco se pretende construir una plataforma AutoML generalista capaz de cubrir cualquier tipo de problema predictivo, sino un prototipo acotado que permita evaluar la viabilidad del patrón multi-agente dentro de un flujo MLOps real. Por el mismo motivo, el sistema no pretende reemplazar la intervención experta: la arquitectura asigna a los agentes tareas de interpretación, planificación y apoyo a la decisión, mientras que las operaciones críticas de entrenamiento, validación y evaluación permanecen implementadas mediante componentes deterministas y reproducibles, en aplicación del principio de restraint agéntico [4].

1.4. Estructura del trabajo

El resto de esta memoria se organiza en seis capítulos que avanzan desde los fundamentos teóricos hasta la validación experimental del prototipo, complementados por una sección final de anexos.



El capítulo 2 presenta los conceptos básicos necesarios para situar el trabajo. Se introducen las nociones de MLOps y de sistemas basados en modelos de lenguaje y agentes, distinguiendo entre componentes que se limitan a generar salidas estructuradas y aquellos que incorporan razonamiento sobre herramientas externas. A continuación, se discuten los elementos técnicos sobre los que se construye el sistema: el uso de JSON como contrato de entrada del pipeline, la API de OpenAI como capa de acceso al modelo, LangGraph como infraestructura de orquestación mediante grafos de estado, el patrón ReAct para agentes con herramientas y los mecanismos de Human-in-the-Loop en sistemas asistidos por IA generativa. El capítulo cierra con el principio de restraint agéntico, que vertebra las decisiones arquitectónicas del trabajo, y con una revisión de las herramientas auxiliares del ciclo MLOps utilizadas.

El capítulo 3 recoge el estudio de alternativas tecnológicas considerado para cada componente principal del sistema. Se analizan opciones para el framework de orquestación, frente al cual se selecciona LangGraph; para el proveedor del modelo de lenguaje, donde se contrastan la API directa de OpenAI con GitHub Models, Azure OpenAI, Anthropic Claude, soluciones locales como Ollama o LM Studio y Hugging Face Inference Providers; para el almacén de experiencias, comparando SQLite, PostgreSQL, bases vectoriales y ficheros JSON; y para el stack de interfaz de usuario, contraponiendo Streamlit y Gradio frente a la combinación finalmente adoptada de Next.js y FastAPI. Cada decisión se justifica en función de los requisitos del prototipo.

El capítulo 4 expone la arquitectura desarrollada del sistema. Comienza con una visión general que sitúa el grafo LangGraph dentro de una aplicación contenedorizada con interfaz web, API REST y almacenes de seguimiento experimental. A continuación se describe el contrato de entrada del pipeline, materializado en un esquema JSON que define el problema predictivo y las restricciones sobre los datos. La parte central del capítulo desarrolla los tipos de nodos del grafo y los contratos de ejecución que los gobiernan, dedicando una subsección específica al descubrimiento de joins entre tablas por parte del agente de validación. Posteriormente se presenta el patrón de controlador determinista combinado con agentes especializados, junto con su extracto de código, y se detallan los mecanismos de seguimiento experimental con MLflow y la memoria empírica del sistema —el Experience Pool— como

soporte de la planificación. El capítulo concluye recorriendo el flujo de ejecución end-to-end del pipeline desde la perspectiva del usuario.

El capítulo 5 contiene la evaluación experimental del prototipo. Tras delimitar el contexto y las dependencias de los resultados y describir el diseño de las pruebas, se analiza la composición y validación inicial del Experience Pool a partir de los benchmarks. A continuación, se estudia el comportamiento agéntico en ejecuciones completas de forecasting, prestando atención a la recuperación de experiencias y al razonamiento del planificador, al descubrimiento de joins por parte del agente de validación y al comportamiento del coste agéntico frente al tamaño del dataset y entre repeticiones de un mismo caso. El capítulo continúa con un análisis de robustez ante errores controlados, un desglose del coste por nodo —separando latencia agéntica de cómputo determinista— y una valoración del papel de las puertas de supervisión humana durante las ejecuciones.

El capítulo 6 recoge las conclusiones del trabajo y plantea las líneas futuras. Se sintetizan los hallazgos principales de los experimentos, se cuantifican las cifras más representativas del comportamiento del prototipo y se identifican las limitaciones del alcance abordado, abriendo direcciones para una posible evolución del sistema hacia entornos más amplios.

Por último, los anexos reúnen el material complementario del trabajo. El *Anexo A* (Reproducción y despliegue) documenta los pasos necesarios para reproducir y desplegar la aplicación, incluyendo la configuración mediante variables de entorno, la ejecución gratuita a través de GitHub Models, el despliegue con Docker, el sembrado inicial del Experience Pool, la verificación funcional y el enlace al repositorio público de código. El *Anexo B* (Procedencia de los conjuntos de datos) detalla las fuentes de los datasets utilizados en la evaluación experimental, distinguiendo entre los conjuntos reales obtenidos de fuentes públicas y los conjuntos sintéticos generados específicamente para este trabajo.

2. Conceptos básicos

2.1. MLOps

MLOps (*Machine Learning Operations*) es la disciplina que aplica los principios de DevOps (*Development & Operations*) al ciclo de vida completo del aprendizaje automático. Su objetivo es establecer prácticas reproducibles, automatizadas y monitorizadas para el desarrollo, despliegue y mantenimiento de modelos en producción [5].

Un flujo MLOps típico comienza con la ingestión de datos, donde se cargan los conjuntos desde sus fuentes originales —archivos CSV, bases de datos, APIs o almacenamiento en la nube— y se hacen accesibles para el resto del pipeline. Una vez disponibles, la siguiente fase es el análisis exploratorio de datos (EDA, *Exploratory Data Analysis*), cuyo propósito es comprender la estructura del conjunto: examinar el esquema (tipos de cada columna, cardinalidad de las variables categóricas, distribuciones de las numéricas), identificar la presencia y proporción de valores ausentes, detectar outliers y, en definitiva, diagnosticar qué transformaciones serán necesarias para que la información sea utilizable por un modelo. Este diagnóstico alimenta directamente la fase de preparación de datos (*data preparation*), donde se ejecutan las operaciones de limpieza, transformación e imputación que el EDA reveló como necesarias: eliminación o imputación de valores ausentes, codificación de variables categóricas, escalado de variables numéricas, eliminación de duplicados y, cuando procede, ingeniería de características derivadas. Con la información ya limpia y transformada, la preparación para el entrenamiento se ocupa de una tarea distinta a la calidad del dato: la partición del conjunto en subconjuntos de entrenamiento, validación y test, asegurando que no se produzcan fugas de información entre particiones.

Solo entonces comienza el entrenamiento propiamente dicho, que abarca la selección de algoritmos, la búsqueda de hiperparámetros y el ajuste del modelo sobre el conjunto de entrenamiento, registrando métricas y artefactos en cada iteración. Una vez entrenado, el modelo

candidato pasa a la fase de evaluación, donde se compara contra unos umbrales de calidad previamente definidos en métricas objetivas como “precisión”, “F1” y “AUC-ROC” sobre el conjunto de test, y se toma la decisión de promoción. Si el modelo supera los umbrales establecidos, la etapa de despliegue lo pone en producción y el modelo comienza a realizar predicciones en el entorno real. Por último, la monitorización cierra el ciclo detectando degradación del rendimiento y ‘drift’ de datos en producción, generando alertas que pueden desencadenar un nuevo ciclo de reentrenamiento.

Google Cloud define tres niveles de madurez MLOps [5]: el nivel 0 (manual), donde cada paso requiere intervención humana; el nivel 1 (automatización del pipeline), donde el entrenamiento se ejecuta de forma continua pero el despliegue sigue siendo manual; y el nivel 2 (CI/CD de ML), donde tanto el entrenamiento como el despliegue están integrados en un pipeline automatizado con puertas de aprobación humana.

2.2. Sistemas basados en LLMs y agentes

Un modelo de lenguaje grande puede utilizarse de distintas formas dentro de una aplicación. En el caso más simple, el LLM actúa como un componente generativo o analítico: recibe una entrada, sigue unas instrucciones y produce una salida textual o estructurada. Este tipo de uso resulta útil, por ejemplo, para generar informes, resumir resultados o producir objetos JSON que después puedan validarse mediante esquemas formales. Sin embargo, no todo componente que utiliza un LLM puede considerarse un agente.

Un agente de IA añade una capa de decisión sobre el modelo de lenguaje. Además de generar texto, puede seleccionar acciones, invocar herramientas externas y adaptar su comportamiento en función de los resultados obtenidos. Estas herramientas pueden incluir consultas a bases de datos, ejecución de código, recuperación de información o llamadas a otros servicios. Por tanto, la capacidad de un agente no depende únicamente del LLM subyacente, sino también del conjunto de herramientas disponibles, de las instrucciones que delimitan su comportamiento y del mecanismo de orquestación que controla su ejecución [4].

Un sistema multiagente coordina varios componentes especializados para resolver una tarea compleja. En este contexto, algunos componentes pueden ser agentes propiamente dichos, al combinar razonamiento del LLM con herramientas y capacidad de decisión, mientras que otros pueden ser nodos LLM más acotados, encargados de producir una salida estructurada sin interactuar directamente con herramientas externas. Dicha distinción resulta importante para evitar una visión excesivamente amplia del término agente y para diseñar arquitecturas más controladas, donde cada parte del sistema asume únicamente la autonomía necesaria para cumplir su función.

La literatura describe distintas arquitecturas de coordinación para sistemas multiagente [6]. Una posibilidad consiste en ejecutar los componentes de forma secuencial, siguiendo un orden fijo. Otra opción es emplear mecanismos de enrutamiento, donde el sistema decide qué componente debe actuar en función del tipo de entrada o del estado actual del proceso. El patrón supervisor introduce un nodo central encargado de observar el estado del flujo y decidir cuál debe ser el siguiente paso. También existen arquitecturas más descentralizadas, en las que los agentes se transfieren el control entre sí, o estructuras jerárquicas, donde varios supervisores organizan subgrupos de agentes.

Este proyecto adopta una arquitectura secuencial basada en un grafo de estados. El flujo no se organiza alrededor de un supervisor que decide dinámicamente el siguiente agente, sino como un pipeline explícito en el que cada nodo cumple una función concreta y actualiza el estado compartido antes de ceder el control al siguiente componente. Tal decisión reduce la complejidad de la orquestación y facilita la trazabilidad del sistema, ya que el recorrido principal del pipeline queda definido de forma clara.

2.3. JSON como contrato de entrada del pipeline

JSON, acrónimo de JavaScript Object Notation, es un formato ligero de intercambio de datos basado en pares clave-valor, listas y tipos simples como cadenas de texto, números, booleanos y valores nulos. Su uso resulta habitual en aplicaciones web, APIs y sistemas de

configuración porque permite representar información estructurada de forma legible tanto para personas como para programas.

En este trabajo, JSON se utiliza como formato para definir un contrato de entrada entre el usuario y el pipeline. Este contrato describe el problema predictivo que se desea resolver, las columnas esperadas, la variable objetivo y determinadas restricciones que deben cumplirse durante la validación de datos. En problemas de predicción temporal, también permite indicar la columna de fecha, el horizonte de predicción, la frecuencia de la serie y la disponibilidad futura de variables exógenas.

El uso de un contrato estructurado evita que el sistema tenga que inferir desde cero el objetivo del problema. La persona experta define el marco del problema, el objetivo predictivo, las columnas relevantes y las restricciones principales; el sistema utiliza esa información para automatizar las fases posteriores con mayor seguridad, trazabilidad y coherencia.

2.4. API de OpenAI como capa de acceso al modelo

Los componentes basados en modelos de lenguaje requieren una capa de acceso al modelo que permita enviar instrucciones, recibir respuestas y, cuando procede, obtener salidas estructuradas o invocar herramientas. En este trabajo, dicha capa se implementa mediante la API de OpenAI, que proporciona una interfaz programática para interactuar con modelos de lenguaje desde el código de la aplicación.

La documentación oficial de OpenAI describe su API como una interfaz accesible mediante peticiones REST, streaming y comunicación en tiempo real, con autenticación basada en claves API [7]. Desde el punto de vista de implementación, estas claves deben gestionarse como secretos y cargarse desde variables de entorno o mecanismos equivalentes, evitando incorporarlas directamente al código fuente.

En el prototipo desarrollado, la API de OpenAI actúa como proveedor del modelo uti-

lizado por los nodos que requieren capacidades lingüísticas. El sistema encapsula esta dependencia en una factoría de modelos, de forma que el resto de la arquitectura no interactúa directamente con la API, sino con una abstracción interna encargada de construir el cliente LLM correspondiente. Dicha decisión desacopla la lógica del sistema —grafo de ejecución, contratos, herramientas y nodos deterministas— del proveedor concreto del modelo.

La API del modelo no ejecuta por sí misma el flujo MLOps. Su responsabilidad consiste en proporcionar capacidad de interpretación, razonamiento lingüístico y generación de salidas estructuradas a los componentes que las requieren. Las fases críticas del pipeline, como la validación de datos, el entrenamiento, la optimización de hiperparámetros, la evaluación y el registro de artefactos, permanecen implementadas mediante código determinista.

Dicha separación refuerza la filosofía arquitectónica del proyecto: utilizar modelos de lenguaje para tareas de planificación, coordinación y explicación, pero mantener las operaciones reproducibles bajo control programático. De este modo, la API de OpenAI funciona como infraestructura de acceso al modelo, tanto para agentes con herramientas como para nodos LLM más acotados.

2.5. LangGraph: orquestación mediante grafos de estado

LangGraph es un framework de código abierto desarrollado por LangChain para construir aplicaciones basadas en modelos de lenguaje y agentes mediante grafos de estado que pueden incorporar ciclos [8]. Forma parte del ecosistema LangChain, por lo que se integra de forma natural con sus abstracciones para modelos de lenguaje, herramientas, mensajes y salidas estructuradas. No obstante, su aportación principal no consiste simplemente en facilitar llamadas a un LLM, sino en proporcionar una capa de orquestación capaz de organizar un workflow completo mediante estado compartido, nodos de ejecución y transiciones explícitas.

Su modelo conceptual se articula en torno a varios elementos fundamentales. El esta-

do representa la memoria estructurada del grafo y se comparte entre los distintos nodos; en Python suele definirse mediante un esquema tipado, por ejemplo un TypedDict, cuyas claves pueden incorporar *reducers* para acumular valores en lugar de sobrescribirlos [9]. Los nodos son funciones que reciben el estado actual y devuelven una actualización parcial de este. Esta abstracción permite encapsular componentes de naturaleza distinta dentro de una misma arquitectura: llamadas a modelos de lenguaje, agentes con herramientas externas, validaciones programáticas o lógica Python determinista. Las aristas definen el flujo de control entre nodos y, en su variante condicional, permiten seleccionar dinámicamente el siguiente paso en función del contenido del estado. Además, la API Command permite combinar en una misma operación la actualización del estado y la redirección del flujo hacia otra etapa. Por último, el checkpointer proporciona persistencia mediante puntos de control del estado, lo que permite inspeccionar y reanudar ejecuciones, así como implementar mecanismos de supervisión humana o *human-in-the-loop* en puntos concretos del flujo. En el contexto de este trabajo, LangGraph resulta adecuado porque permite representar el pipeline MLOps como un flujo explícito y trazable, combinando componentes agénticos, nodos LLM acotados y etapas deterministas dentro de una misma estructura de ejecución.

2.6. Patrón ReAct y agentes con herramientas

El patrón ReAct (*Reasoning + Acting*) propone combinar el razonamiento del modelo de lenguaje con la ejecución de acciones sobre herramientas externas [10]. En lugar de limitarse a generar una respuesta final en una única llamada, el agente puede descomponer la tarea, decidir qué herramienta necesita invocar, observar el resultado obtenido y continuar el proceso hasta alcanzar una solución satisfactoria. Este esquema permite que el modelo no dependa únicamente de la información contenida en sus parámetros, sino que pueda apoyarse en fuentes externas, funciones programáticas, bases de datos o servicios auxiliares.

En una aplicación práctica, este patrón suele materializarse como un bucle de interacción entre el modelo y el entorno. El LLM interpreta el objetivo recibido y decide una acción; el sistema ejecuta la herramienta correspondiente y devuelve una observación; posteriormen-

te, el modelo utiliza esa observación para decidir el siguiente paso o generar la respuesta final. De este modo, el agente puede abordar tareas que requieren consultar información, validar condiciones, recuperar conocimiento o ejecutar operaciones que no deberían delegarse directamente al modelo.

En el contexto de este trabajo, ReAct resulta relevante para aquellos nodos que combinen un LLM con herramientas externas. No obstante, no todos los componentes basados en modelos de lenguaje siguen este patrón. Algunos nodos utilizan el LLM de forma más acotada, por ejemplo para generar salidas estructuradas o informes, sin invocar herramientas ni controlar iterativamente el entorno. Tal distinción permite separar los agentes con capacidad de acción de los componentes LLM que actúan únicamente como módulos de generación o análisis.

2.7. Human-in-the-Loop en sistemas basados en IA generativa

El concepto de supervisión humana en el flujo (Human-in-the-Loop, HITL) se refiere a la inclusión de puntos de control donde se requiere supervisión o aprobación humana antes de que un sistema automático ejecute una acción irreversible o de alto impacto [11]. Esta supervisión resulta especialmente importante cuando el flujo automatizado puede producir efectos difíciles de revertir, como aceptar un conjunto de datos procesado, promover un modelo como candidato a despliegue o ejecutar una operación con impacto sobre un entorno productivo.

En sistemas que incorporan modelos de lenguaje y agentes, el HITL permite reducir el riesgo asociado a salidas incorrectas, incompletas o inesperadas. Aunque los LLMs pueden aportar capacidades útiles de interpretación, planificación y generación de explicaciones, sus resultados no deben asumirse como válidos sin mecanismos de control cuando afectan a decisiones críticas. La intervención humana introduce una barrera adicional de verificación y permite comprobar si la decisión propuesta responde al objetivo del sistema.

Además de su papel en seguridad y control, el HITL aporta trazabilidad. Cada aprobación o rechazo puede registrarse junto con el estado del flujo, las métricas disponibles y la justificación asociada, generando una evidencia auditable de cómo se tomó una decisión. Esta idea conecta con el enfoque regulatorio del Reglamento Europeo de Inteligencia Artificial, cuyo Artículo 14 establece requisitos de supervisión humana para sistemas de IA clasificados como de alto riesgo [11]. Aunque el prototipo desarrollado en este trabajo no se plantea como un sistema regulado de alto riesgo, este principio sirve como referencia de diseño para incorporar mecanismos de control humano en puntos críticos del pipeline MLOps.

2.8. Principio de restraint agéntico: cuándo no usar agentes

Uno de los principios de diseño más importantes —y menos intuitivos— en sistemas agénticos es saber cuándo *no* usar un agente. Anthropic lo formula explícitamente [4]:

«Recomendamos encontrar la solución más simple posible, y solo aumentar la complejidad cuando sea necesario. Esto puede significar no construir sistemas agénticos en absoluto.»

OpenAI identifica solo tres escenarios donde los agentes genuinamente aportan valor sobre código determinista [12]: (1) decisiones complejas que requieren juicio, (2) sistemas de reglas que se han vuelto inmanejables, y (3) tareas que dependen fuertemente de datos no estructurados.

Este principio guía toda la arquitectura de este proyecto: cada etapa del pipeline comienza como código determinista y solo escala a agente cuando existe incertidumbre genuina que justifique el coste adicional en latencia, tokens y complejidad de depuración.

2.9. Herramientas auxiliares del ciclo MLOps

El prototipo desarrollado se apoya en distintas herramientas técnicas para cubrir las fases principales del flujo predictivo. Tales herramientas no constituyen por sí solas la aportación



principal del trabajo, pero proporcionan la infraestructura necesaria para registrar experimentos, validar contratos, optimizar modelos, conservar experiencias y ejecutar algoritmos predictivos de forma reproducible.

MLflow se utiliza como capa de seguimiento de experimentos y registro de modelos. Durante la ejecución del entrenamiento, el sistema registra parámetros, métricas, artefactos y modelos candidatos, manteniendo una relación entre la ejecución principal y los ensayos asociados a cada candidato. Además, la fase final de promoción utiliza el registro de modelos de MLflow para asignar el alias `champion` al modelo seleccionado. En este trabajo, dicha promoción debe entenderse como un despliegue lógico dentro del registro de modelos, no como la exposición de un servicio de inferencia en producción.

Optuna proporciona el mecanismo de optimización de hiperparámetros. El ejecutor determinista recibe un plan de entrenamiento, consulta el registro de modelos y construye los espacios de búsqueda definidos para cada candidato. A partir de estos espacios, Optuna explora distintas configuraciones bajo una métrica objetivo. La estrategia de validación no se optimiza como si fuera un hiperparámetro, ya que representa el protocolo de evaluación y debe permanecer fijada para garantizar una comparación coherente entre modelos.

Pydantic se emplea para definir contratos tipados y validables entre componentes. El sistema lo utiliza en estructuras como perfiles de dataset, planes de entrenamiento, salidas del planificador, registros de experiencia y reportes estructurados de evaluación. Esta validación resulta especialmente importante en los puntos donde interviene un modelo de lenguaje, ya que impide que una salida generada introduzca modelos no registrados, configuraciones incompatibles, referencias de evidencia inexistentes o decisiones fuera del contrato esperado.

SQLite actúa como mecanismo local de persistencia para el Experience Pool. Cada ejecución completada puede almacenar información sobre el perfil del dataset, el plan utilizado, los modelos evaluados, el modelo seleccionado, las métricas obtenidas y las estrategias aplicadas, manteniendo además una copia JSON auditable. Esta memoria permite recuperar experiencias similares en ejecuciones posteriores y proporcionar al planificador evidencia empírica procedente de resultados anteriores.



El sistema incorpora también un registro declarativo de modelos definido en YAML. Este registro funciona como fuente de verdad sobre los modelos soportados, sus claves internas, familias, librerías, factorías de construcción, espacios de búsqueda, restricciones y recomendaciones de uso. Gracias a esta capa, el planificador no puede proponer modelos arbitrarios: únicamente puede seleccionar candidatos existentes en el registro, mientras que el ejecutor determinista se encarga de construirlos y entrenarlos.

Junto a esta memoria empírica, el prototipo incluye una base de conocimiento estática en YAML con reglas de aprendizaje automático. Estas reglas recogen criterios generales sobre selección de modelos, estrategias de validación y tratamiento de variables exógenas. El planificador puede recuperar tanto experiencias previas como reglas aplicables al perfil del dataset, combinando así evidencia observada en ejecuciones anteriores con conocimiento experto codificado.

En la fase de validación de datos se emplean herramientas programáticas basadas principalmente en pandas. Dichas herramientas permiten cargar conjuntos, aplicar mapeos de columnas, validar esquemas, detectar valores ausentes, imputar datos cuando procede, convertir columnas temporales y revisar aspectos propios de series temporales, como la existencia de huecos en la secuencia. Los resultados de estas comprobaciones se devuelven como observaciones estructuradas que el agente de validación puede interpretar antes de continuar el flujo.

Finalmente, las bibliotecas de modelado proporcionan los algoritmos predictivos disponibles para el entrenamiento. scikit-learn se emplea para modelos base de clasificación, regresión y algunos estimadores utilizados en forecasting supervisado; LightGBM, XGBoost y CatBoost aportan modelos de gradient boosting; statsforecast proporciona modelos estadísticos como naive, seasonal naive, ETS y AutoARIMA; y skforecast permite adaptar modelos supervisados a predicción temporal mediante retardos. La ejecución de estos modelos permanece bajo control programático, de forma que los componentes LLM intervienen en la planificación y explicación, pero no ejecutan libremente código ni seleccionan algoritmos fuera del registro definido.



En conjunto, estas herramientas forman la base técnica sobre la que se construye el sistema. La contribución del trabajo reside en integrarlas dentro de una arquitectura híbrida, donde los modelos de lenguaje aportan capacidades de interpretación, planificación y generación estructurada, mientras que las fases críticas del ciclo MLOps se mantienen trazables, reproducibles y controladas mediante código determinista.

Las herramientas descritas en esta sección constituyen elecciones habituales en proyectos de aprendizaje automático y sistemas agénticos actuales: Pydantic en su papel de capa de validación tipada, MLflow como solución de referencia para seguimiento experimental, Optuna a modo de motor de optimización de hiperparámetros, scikit-learn como base de modelado supervisado, y statsforecast y skforecast como librerías especializadas en series temporales. Su selección no se desarrolla en el capítulo de alternativas porque, frente a las cuatro decisiones que sí se discuten allí —framework de orquestación, proveedor de LLM, almacén de experiencias y stack de interfaz—, estas herramientas se han adoptado siguiendo prácticas ampliamente extendidas en el ecosistema sin que se identificara una tensión clara entre opciones durante el diseño del prototipo.

3. Estudio de alternativas

Este capítulo analiza las principales alternativas tecnológicas consideradas para cada componente del sistema y justifica las elecciones adoptadas. Antes de entrar en la comparativa, conviene explicitar los criterios que han guiado cada decisión, de modo que el análisis no resulte arbitrario. Las elecciones se han tomado bajo cinco vectores concretos del proyecto, que merece la pena describir antes de aplicarlos.

El primer vector es el contexto del encargo: una empresa del sector químico que solicitó explorar vías de aplicación de IA generativa a sus procesos analíticos, sin especificar arquitectura ni entregables. La acotación del problema y la elección del enfoque multi-agente forman parte del propio trabajo, lo que sitúa estas decisiones al mismo nivel que las elecciones tecnológicas posteriores.

A este contexto se añade un segundo vector, los objetivos funcionales que el prototipo debe cubrir. Solo una parte de estos objetivos procede del encargo original: la empresa esperaba un sistema capaz de procesar datasets reales del estilo de los que maneja a diario, automatizar el flujo de predicción de principio a fin y mantener una traza auditable de las decisiones tomadas. El resto de las capacidades del prototipo —la organización del flujo en agentes especializados con responsabilidades acotadas, la incorporación de una memoria empírica reutilizable entre ejecuciones, la ubicación concreta de las dos puertas de supervisión humana en los puntos de mayor riesgo del pipeline, y la separación deliberada entre planificación asistida por LLM y ejecución determinista del entrenamiento— son decisiones de diseño propias, derivadas de tres fuentes complementarias: la revisión de literatura sobre sistemas agénticos, MLOps y patrones de human-in-the-loop; el contacto previo con sistemas multi-agente durante un proyecto académico de carácter teórico y una estancia de prácticas en otra empresa, que aportó cierta familiaridad con los problemas prácticos de orquestación que aparecen al pasar del diseño sobre el papel al código que se ejecuta; y la observación directa del trabajo del equipo de ciencia de datos durante las prácticas en las que se enmarca este TFG. Cualquier tecnología que no permitiera articular tanto las capacidades pedidas por

la empresa como las añadidas durante el diseño quedaba automáticamente descartada.

A esos dos vectores se suman tres restricciones operativas. Las limitaciones técnicas exigen que el sistema pueda ejecutarse en local, sin depender de infraestructura cloud corporativa durante la fase de prototipo; esta restricción descarta soluciones gestionadas que requieran integración profunda con proveedores específicos. Sobre esa restricción de ejecución local, la decisión de empaquetar el sistema en contenedores Docker es una elección propia: containerizar los tres servicios —el servidor de seguimiento MLflow, el backend FastAPI con los agentes LangGraph y la interfaz web Next.js— aporta limpieza arquitectónica, aísla las dependencias y permite trasladar el prototipo entre equipos sin reproducir manualmente el entorno de ejecución.

La segunda restricción operativa, el presupuesto, conviene desdoblarla en dos planos distintos. En el plano de la empresa, no se autorizó la contratación de licencias de pago para un prototipo de investigación; la empresa cliente, en una fase aún temprana de adopción de IA, descartó tanto OpenAI Enterprise como Azure OpenAI por su coste para un proyecto exploratorio, y rechazó también GitHub Models a pesar de su gratuidad porque el procesamiento se realiza en infraestructura externa, lo que entraba en conflicto con sus criterios internos de tratamiento de datos. La propuesta inicial recibida fue, por tanto, ejecutar todo el prototipo contra modelos abiertos servidos en local mediante Ollama o LM Studio.

Sin embargo, como se detalla en la sección 3.2, los modelos abiertos disponibles en la actualidad presentan limitaciones reales para los nodos LLM más exigentes del sistema —el validador de datos y el planificador—, y un equipo doméstico no dispone de la memoria necesaria para ejecutar alternativas lo suficientemente capaces. Esta restricción de partida obligó a redefinir el plano económico del proyecto. Dado que el TFG no tiene por qué ceñirse de forma estricta a la información y la infraestructura reales del cliente, y dado que la sustitución de los conjuntos reales por datos sintéticos ya rompía deliberadamente esa atadura, la decisión adoptada fue costear de forma particular el acceso a la API de OpenAI durante el desarrollo y la evaluación experimental. Tal elección permite trabajar con modelos suficientemente capaces para que las conclusiones sobre el comportamiento del sistema sean significativas, manteniendo a la vez el coste por ejecución por debajo de un dólar en los

escenarios típicos analizados.

Conviene dejar explícita una implicación de esta elección: el prototipo, tal como se evalúa en esta memoria, no replica las condiciones económicas que la empresa exigiría en un despliegue real. En un entorno productivo y con presupuesto disponible, la opción más alineada con los criterios corporativos sería OpenAI Enterprise o Azure OpenAI —ambas, como el resto de proveedores compatibles con la API de OpenAI utilizada, sin impacto en la arquitectura del sistema—. La vía gratuita basada en GitHub Models se ha mantenido documentada en el Anexo A y operativa en el código como alternativa para experimentación, en línea con la práctica habitual de ofrecer un canal sin coste para reproducción académica.

Por último, las condiciones humanas del proyecto —un único estudiante en el marco de un Trabajo Fin de Grado con un horizonte temporal limitado— penalizan tecnologías inmaduras o con curvas de aprendizaje muy elevadas frente a alternativas igualmente válidas pero más asentadas. Este criterio actúa de forma transversal y opera sobre el resto: ante dos tecnologías que satisfacen el contexto, los objetivos, las limitaciones técnicas y el presupuesto, se prefiere aquella cuya documentación, comunidad y madurez reducen el riesgo de bloqueos durante el desarrollo.

Estos cinco vectores —contexto, objetivos, limitaciones técnicas, presupuesto y condiciones humanas— aparecen recurrentemente en las subsecciones que siguen y son los que permiten distinguir, ante cada alternativa, si su adopción habría aportado valor real al proyecto o habría introducido complejidad innecesaria. La selección no es trivial: los ecosistemas de orquestación de agentes, proveedores de modelos de lenguaje, mecanismos de persistencia y herramientas MLOps han evolucionado rápidamente en los últimos años, y no todas las opciones ofrecen el mismo grado de control, reproducibilidad, madurez o adecuación para un prototipo de investigación académica enmarcado en un contexto industrial. En cada subsección se identifican los criterios relevantes para la decisión, se comparan las alternativas y se razona la elección final.

3.1. Alternativas a LangGraph como framework de orquestación

La elección del framework de orquestación constituye una de las decisiones arquitectónicas más relevantes del proyecto, ya que condiciona la forma en que se modelan el estado compartido, los nodos de ejecución, el flujo de control, los ciclos de revisión y los mecanismos de supervisión humana. En este trabajo se buscaba una infraestructura capaz de representar un flujo MLOps compuesto por agentes con herramientas, nodos LLM más acotados y componentes deterministas, manteniendo una ejecución explícita, auditable y recuperable. Bajo estos criterios, se analizaron varias alternativas.

Microsoft Agent Framework (MAF) representa la alternativa más sólida a LangGraph dentro del ecosistema actual de frameworks agénticos [13]. Este framework surge como evolución y unificación de las líneas de trabajo de AutoGen y Semantic Kernel, incorporando abstracciones para agentes, workflows, gestión de estado, telemetría y soporte empresarial. Su propuesta resulta especialmente atractiva en entornos corporativos integrados con tecnologías de Microsoft, como Azure, .NET, servicios de observabilidad o conectores empresariales. Además, MAF proporciona soporte para flujos multiagente, patrones de workflow y mecanismos de *human-in-the-loop*, por lo que conceptualmente cubre una parte importante de los requisitos de este proyecto.

Sin embargo, en el momento en que se tomó la decisión arquitectónica inicial, Microsoft Agent Framework todavía se encontraba en una fase temprana de madurez. No fue hasta abril de 2026 cuando Microsoft anunció la versión 1.0 de Agent Framework para .NET y Python, presentándola como una versión estable y preparada para producción [14]. Adoptar una tecnología todavía en fase beta habría incrementado el riesgo tecnológico del proyecto, especialmente al tratarse de un Trabajo Fin de Grado con un horizonte temporal limitado. Además, la aplicación desarrollada no dependía de servicios específicos del ecosistema Microsoft ni requería una integración profunda con Azure, .NET o herramientas corporativas. Por tanto, aunque MAF constituye una alternativa especialmente potente para escenarios empresariales, LangGraph ofrecía una solución más ligera, madura y directamente alineada con el ecosistema utilizado en el proyecto.

AutoGen y Semantic Kernel se consideraron inicialmente como alternativas independientes [15, 16]. AutoGen organiza los agentes en calidad de participantes en conversaciones multi-turno, lo que resulta útil para prototipos conversacionales y colaboración entre agentes. Semantic Kernel, por su parte, proporciona un SDK orientado a la integración de modelos de lenguaje mediante funciones, conectores y plugins, con especial orientación a escenarios empresariales. No obstante, la evolución reciente del ecosistema de Microsoft ha desplazado ambas alternativas hacia Microsoft Agent Framework [13, 15], que actúa como sucesor natural de estos enfoques. Por este motivo, en la decisión final no se trataron como opciones principales, sino como antecedentes tecnológicos de MAF.

CrewAI ofrece una abstracción de alto nivel basada en agentes, tareas, crews y procesos [17]. Tal aproximación permite construir sistemas multiagente con poca configuración y resulta adecuada para prototipos en los que se desea modelar de forma sencilla la colaboración entre agentes especializados. Su modelo de ejecución contempla procesos secuenciales y jerárquicos, y sus Flows permiten definir automatizaciones más estructuradas con gestión de estado y control del flujo. Sin embargo, su abstracción principal no es un grafo de estados tipado comparable al de LangGraph, sino una organización de agentes y tareas de mayor nivel. Por ello, se adapta peor a una arquitectura en la que deben convivir agentes con herramientas, nodos LLM sin herramientas, validaciones deterministas, ejecución reproducible de entrenamiento, persistencia explícita del estado e interrupciones para supervisión humana.

Código Python sin framework también se valoró como alternativa. Esta opción ofrece el máximo control sobre la implementación, pero obligaría a desarrollar manualmente la gestión del estado compartido, el enrutamiento condicional, los ciclos del grafo, la persistencia mediante checkpoints y el mecanismo de interrupción para supervisión humana. En el contexto de este TFG, dedicar un esfuerzo significativo a construir esta infraestructura desde cero habría desviado el foco principal del trabajo, centrado en la automatización de un flujo MLOps mediante agentes IA. En la tabla 1 se recoge una comparativa de las alternativas consideradas para la orquestación del sistema.

LangGraph fue seleccionado porque satisfacía de forma directa los requisitos estructurales del proyecto: modelado explícito del flujo como grafo de estados, estado compartido



Criterio	LangGraph	MAF	CrewAI	Python
Grafo de estados con ciclos	Sí	Sí	No nativo	Manual
Estado compartido tipado	Sí	Sí	Limitado	Manual
Nodos LLM y deterministas intergrados	Sí	Sí	Limitado	Manual
Control granular del flujo condicional	Sí	Sí	Limitado	Manual
Checkpointing e interrupción HITL	Sí	Sí	Limitado	Manual
Madurez al inicio del TFG	Alta	Limitada	Media	No aplica
Integración empresarial Microsoft/Azure	Media	Alta	Baja	Baja

Tabla 1: Comparativa de alternativas para la orquestación del sistema.

tipado, combinación de agentes, nodos LLM acotados y componentes deterministas, persistencia mediante checkpoints e interrupciones para human-in-the-loop. Microsoft Agent Framework representa la alternativa más cercana y potente, especialmente en escenarios empresariales integrados con Microsoft, pero su grado de madurez en el momento de inicio del proyecto y la ausencia de dependencias específicas con Azure o .NET hacían menos conveniente su adopción. En consecuencia, LangGraph ofrecía una relación más favorable entre expresividad, simplicidad, madurez y adecuación al alcance del TFG.

3.2. Alternativas al proveedor del modelo de lenguaje

La opción finalmente utilizada en el proyecto fue el acceso directo a la API de OpenAI [18, 19]. Esta alternativa proporciona acceso inmediato a modelos de alta calidad, buen soporte para salidas estructuradas y una integración sencilla dentro del stack Python del sistema. En la implementación desarrollada, la creación del cliente LLM queda encapsulada en una factoría propia, de modo que el resto de componentes no dependen directamente de la forma concreta en que se invoca el proveedor. Tal decisión simplifica el desarrollo y permite sustituir el backend del modelo con cambios localizados en la configuración.

La principal desventaja de esta opción es el coste por uso, ya que cada ejecución del pipeline consume tokens de entrada y salida. Sin embargo, para el alcance de este TFG, el coste resultó asumible y se consideró compensado por la calidad de los modelos disponibles, su buen comportamiento en tareas de seguimiento de instrucciones y su mayor fiabilidad en la generación de salidas estructuradas. Además, el uso de la API directa permitió evitar las limitaciones de cuota propias de plataformas gratuitas o de prototipado, facilitando un desarrollo iterativo más estable. En cuanto a privacidad, los datos enviados a la API no se utilizan por defecto para entrenar o mejorar modelos, salvo activación explícita de mecanismos de compartición de datos. No obstante, las peticiones sí abandonan el entorno local y son procesadas por un proveedor externo. En este prototipo se trabaja con datasets experimentales y no sensibles, pero en una implantación industrial sería necesario valorar medidas adicionales como minimización de datos, anonimización, acuerdos contractuales específicos o uso de soluciones empresariales.

Durante las primeras fases del desarrollo se consideró GitHub Models como alternativa atractiva para prototipado académico [20]. Esta plataforma permite acceder a distintos modelos mediante una API de inferencia autenticada con un token de GitHub, ofreciendo uso gratuito con límites de cuota. Su principal ventaja reside en facilitar pruebas iniciales sin coste económico directo. Además, al exponer una interfaz cercana a la API de OpenAI, la migración hacia OpenAI directo resulta sencilla cuando la creación del cliente se encuentra encapsulada.

Sin embargo, GitHub Models presenta dos limitaciones relevantes para este proyecto. La primera es operativa: el uso gratuito está orientado a experimentación y puede verse limitado por cuotas, lo que dificulta el desarrollo iterativo cuando el pipeline ejecuta varios nodos LLM por prueba. La segunda está relacionada con el tratamiento de datos. Al utilizar GitHub Models, las entradas y salidas también se envían a una plataforma externa y quedan sujetas a las condiciones de las funciones de IA de GitHub. En cuentas individuales, los términos de GitHub contemplan el uso de inputs y outputs para desarrollar, entrenar y mejorar modelos, salvo que el usuario desactive esta opción; en entornos empresariales, el tratamiento queda condicionado por los acuerdos contractuales correspondientes [21, 22]. Por estos motivos, GitHub Models se mantuvo como alternativa configurable para prototipado, pero no como

proveedor principal de la versión final.

Azure OpenAI, actualmente integrado dentro del ecosistema Microsoft Foundry, constituye una opción especialmente relevante en contextos empresariales [23]. Su interés no reside únicamente en el acceso a modelos avanzados, sino también en las capacidades de gobierno, seguridad, control de acceso, residencia de datos e integración corporativa propias de Azure. Esta alternativa facilita escenarios donde los datos deben procesarse bajo mayores garantías organizativas, especialmente cuando se trabaja con información empresarial, requisitos de cumplimiento o restricciones de residencia de datos. En despliegues dentro de regiones o zonas de datos europeas, Azure permite aproximar el tratamiento de prompts, respuestas y datos almacenados a los requisitos habituales de organizaciones sujetas al Reglamento General de Protección de Datos. No obstante, esta opción introduce mayor complejidad administrativa y normalmente requiere una suscripción Azure habilitada, permisos corporativos y configuración específica de despliegues. En el contexto de un TFG desarrollado desde una cuenta personal de estudiante, esta alternativa no resultaba la más práctica, aunque sería una opción natural para una implantación industrial del sistema.

También se valoraron proveedores alternativos como Anthropic Claude [24, 25]. Sus modelos ofrecen capacidades competitivas en razonamiento, seguimiento de instrucciones y generación de texto estructurado. Desde el punto de vista del tratamiento de datos, la API comercial de Anthropic tampoco utiliza por defecto inputs y outputs para entrenar modelos, salvo autorización explícita. No obstante, su adopción implicaría adaptar la factoría de modelos y las integraciones utilizadas por el sistema. Además, algunos modelos de Claude presentan costes por millón de tokens superiores a las variantes ligeras utilizadas habitualmente para prototipado con OpenAI, por lo que no aportaban una ventaja diferencial suficiente para justificar el cambio de proveedor en el alcance de este trabajo.

Otra familia de alternativas corresponde a la ejecución local de modelos abiertos mediante herramientas como Ollama o LM Studio [26, 27]. Esta opción evita el coste por petición y reduce la dependencia de proveedores externos, además de ofrecer ventajas claras cuando la privacidad de los datos es prioritaria. En este caso, las peticiones pueden procesarse en la propia máquina del usuario o dentro de la infraestructura de la organización, evitando que

los datos abandonen el entorno controlado. LM Studio, además, puede exponer endpoints compatibles con la API de OpenAI, lo que facilitaría reutilizar parte del código cliente modificando la URL base.

Sin embargo, la ejecución local introduce limitaciones importantes. Los modelos disponibles dependen del hardware de la máquina, pueden requerir grandes cantidades de memoria y suelen ofrecer menor rendimiento que los modelos comerciales más avanzados cuando se ejecutan en equipos convencionales. Además, aunque muchos modelos abiertos han mejorado notablemente, su comportamiento puede ser menos estable en tareas que requieren seguimiento estricto de esquemas JSON, uso consistente de instrucciones largas o generación de salidas compatibles con contratos Pydantic. Para nodos críticos como el planificador, donde una salida inválida puede romper la ejecución del pipeline, esta variabilidad introduce un riesgo adicional.

Finalmente, Hugging Face Inference Providers proporciona acceso gestionado a numerosos modelos abiertos y comerciales mediante distintos proveedores de inferencia [28]. Si bien existe una capa de acceso gratuito, los modelos disponibles en ella rara vez ofrecen soporte fiable para tool calling o salidas estructuradas, y los límites de tasa son más restrictivos que los de GitHub Models; para acceder a modelos competitivos con garantías de calidad es necesario recurrir a planes de pago o a Inference Endpoints dedicados. Más allá del coste, su principal limitación para este proyecto es la heterogeneidad del catálogo: la calidad, latencia, disponibilidad y compatibilidad con salidas estructuradas dependen en gran medida del modelo y proveedor concretos. Por ello, Hugging Face se consideró una opción útil para experimentación, pero menos adecuada como proveedor principal de un sistema donde se prioriza la estabilidad del comportamiento del LLM, la simplicidad de integración y la fiabilidad de las salidas estructuradas.

En consecuencia, la API directa de OpenAI fue seleccionada como proveedor principal por ofrecer una combinación favorable de calidad del modelo, estabilidad en salidas estructuradas, facilidad de integración y ausencia de restricciones severas de cuota durante el desarrollo. GitHub Models se mantuvo en su papel de alternativa de bajo coste para prototipado, Azure OpenAI se identificó como opción natural para una futura implantación empresarial,

y los modelos locales se reservaron como alternativa cuando la privacidad o la ejecución dentro de infraestructura propia sean requisitos prioritarios. En la tabla 2 se comparan los proveedores de modelos de lenguaje evaluados para los nodos LLM del sistema.

Criterio	OpenAI	GitHub Models	Azure OpenAI	Claude	Local	HF
Calidad de seguimiento de instrucciones	Alta	Variable	Alta	Alta	Variable	Variable
Salidas estructuradas fiables	Alta	Variable	Alta	Alta	Media	Variable
Soporte nativo para <i>tool calling</i>	Alta	Variable	Alta	Alta	Variable	Variable
Coste cero para prototipado	No	Sí	No	No	Sí	Parcial
Sin dependencia de internet	No	No	No	No	Sí	No
Adecuado para datos sensibles	Medio	Medio	Alto	Medio	Alto	Medio
Facilidad de integración en Python	Alta	Alta	Alta	Alta	Media	Media
Límites de cuota en uso gratuito	No aplica	Sí	No aplica	No aplica	No aplica	Sí/Parcial
Necesidad de infraestructura propia	No	No	No	No	Sí	No
Adecuado para implantación empresarial	Medio/Alto (Acuerdos empresariales)	Medio	Alto	Medio	Medio	Medio

Tabla 2: Comparativa de proveedores de modelos de lenguaje para los nodos LLM del sistema.

3.3. Alternativas al almacén de experiencias

El sistema incorpora un *Experience Pool* encargado de conservar información procedente de ejecuciones anteriores. Cada experiencia almacena metadatos del dataset, características del problema, modelos candidatos evaluados, métricas obtenidas, modelo seleccionado, referencias a artefactos de MLflow y, en el caso de predicción temporal, información sobre la estrategia de validación y el tratamiento de variables exógenas. Este componente cumple una doble función: por un lado, actúa como almacén persistente de experiencias; por otro, permite recuperar ejecuciones similares para proporcionar evidencia empírica al planificador.

Conviene distinguir entre el mecanismo de almacenamiento y el mecanismo de recuperación. El primero determina dónde se guardan las experiencias, por ejemplo en ficheros JSON, una base de datos SQLite o un servidor PostgreSQL. El segundo determina cómo se

calcula la similitud entre una nueva ejecución y las experiencias anteriores. En este trabajo, la similitud se calcula de forma determinista sobre atributos estructurados del perfil del dataset, como el tipo de problema, el tamaño del conjunto de datos, el número de variables, la presencia de valores ausentes o la disponibilidad futura de variables exógenas.

Una alternativa posible para el almacenamiento habría sido utilizar únicamente ficheros JSON. Esta opción resulta sencilla, auditable y fácil de inspeccionar, y de hecho el sistema mantiene una copia JSON de cada experiencia generada. Sin embargo, los ficheros JSON no ofrecen por sí solos un mecanismo cómodo de consulta. Recuperar las experiencias más similares exigiría recorrer todos los ficheros, cargarlos en memoria y aplicar manualmente la función de similitud en cada ejecución. Esta estrategia puede ser suficiente para depuración o trazabilidad, pero resulta menos adecuada como base persistente para recuperación sistemática.

También se valoró el uso de una base de datos relacional completa, como PostgreSQL o MySQL. Estas soluciones ofrecen capacidades avanzadas de consulta, índices, transacciones, concurrencia y escalabilidad, por lo que resultarían adecuadas en una implantación industrial con múltiples usuarios, gran volumen de experiencias y acceso concurrente. No obstante, para el alcance de este TFG, el número esperado de registros es reducido y el sistema se ejecuta como un prototipo local o contenerizado. Incorporar un servidor de base de datos completo habría aumentado la complejidad de instalación, configuración y despliegue sin una necesidad real para la escala del proyecto.

Otra posibilidad sería reutilizar MLflow como almacén principal de experiencias, dado que ya registra métricas, parámetros y artefactos de cada ejecución. Esta opción tendría la ventaja de evitar una pieza adicional de persistencia, pero no se ajusta bien al propósito del Experience Pool. MLflow está orientado al seguimiento de experimentos y al registro de modelos, mientras que el sistema necesita recuperar experiencias a partir de atributos del perfil del dataset y compararlas por similitud estructurada. Sobrecargar MLflow con esta lógica implicaría mezclar dos responsabilidades distintas: el tracking experimental y la memoria empírica utilizada por el planificador.

Por último, se consideró la representación vectorial de experiencias como posible evolución del mecanismo de recuperación. En ese enfoque, los atributos del perfil podrían codificarse mediante técnicas como one-hot encoding o normalización de variables continuas, y los vectores resultantes almacenarse en un índice especializado o una base de datos vectorial. Dicha aproximación resultaría interesante si el sistema creciese hasta manejar un volumen elevado de experiencias o si incorporase información textual menos estructurada. Sin embargo, en la versión actual del prototipo el perfil contiene un conjunto reducido de atributos de significado explícito, por lo que resulta más transparente utilizar una función determinista de similitud ponderada donde se controla directamente qué factores influyen en la recuperación y con qué importancia. Esta evolución queda abierta como trabajo futuro si el volumen de experiencias lo justificase.

Por estos motivos, se seleccionó SQLite como solución. SQLite permite almacenar las experiencias en una base de datos relacional local, sin necesidad de desplegar un servidor independiente y manteniendo una integración sencilla con Python y Docker. Dicha elección resulta suficiente para la escala del prototipo, permite consultar y filtrar registros de forma estructurada, y conserva una arquitectura reproducible con una configuración mínima. Además, el sistema mantiene copias JSON auditables de las experiencias, combinando así una base SQLite para consulta y recuperación con ficheros legibles para inspección y trazabilidad.

La elección de SQLite no pretende cerrar la puerta a soluciones más avanzadas. Si el sistema creciese hasta manejar muchas más experiencias, múltiples usuarios concurrentes o perfiles más complejos, tendría sentido replantear tanto el almacén como el mecanismo de recuperación. En ese escenario podría estudiarse una migración a PostgreSQL, una representación vectorial de los perfiles o una estrategia híbrida que combinase filtrado estructurado por SQL con búsqueda vectorial para información semántica no estructurada. En la versión actual, sin embargo, esta evolución queda fuera del alcance del TFG. En la tabla 3 se resumen las alternativas valoradas para el almacén de experiencias.

Criterio	SQLite	PostgreSQL	BD vectorial	JSON
Sin servidor externo	Sí	No	Parcial	Sí
Consultas relacionales estructuradas	Sí	Sí	No	No
Recuperación por perfil categórico	Sí	Sí	Parcial	No
Cero configuración de despliegue	Sí	No	No	Sí
Esquema jerárquico (3 tablas)	Sí	Sí	No	Parcial
Escalabilidad a gran volumen de experiencias	Limitada	Alta	Alta	No

Tabla 3: Comparativa de alternativas para el almacén de experiencias.

3.4. Alternativas al stack de interfaz de usuario

La interfaz de usuario del sistema no se limita a mostrar un resultado final una vez completada la ejecución. El flujo MLOps genera eventos intermedios que deben presentarse al usuario mientras el pipeline avanza, como mensajes del planificador, progreso del entrenamiento, resultados parciales o informes de evaluación. Además, la aplicación incorpora puntos de supervisión humana en los que el usuario debe revisar información contextual y aprobar o rechazar la continuación del proceso. Por este motivo, la elección del stack de interfaz debía considerar tanto la facilidad de desarrollo como la capacidad para gestionar actualizaciones en tiempo real, formularios interactivos y separación entre frontend y backend.

Una primera alternativa considerada fue Streamlit. Esta herramienta permite construir aplicaciones de datos directamente en Python, con muy poca configuración y sin necesidad de desarrollar una interfaz web desde cero. Esta característica la hace especialmente atractiva para prototipos exploratorios, dashboards sencillos o demostraciones internas de modelos. Sin embargo, el modelo de Streamlit resulta menos adecuado cuando la aplicación requiere una separación clara entre backend y frontend, control fino del estado de una ejecución larga y consumo estable de eventos en tiempo real. En este proyecto, los flujos de aprobación humana y la necesidad de mostrar eventos progresivos del pipeline hacían conveniente una arquitectura más explícita y desacoplada.

También se valoró Gradio, una herramienta orientada a crear interfaces rápidas para modelos de aprendizaje automático. Su principal ventaja es la simplicidad: permite exponer modelos o funciones mediante formularios web con muy poco código, lo que resulta útil para demos, pruebas de inferencia o validación rápida de prototipos. No obstante, el sistema desarrollado no consiste únicamente en probar un modelo con una entrada y una salida, sino en supervisar un pipeline MLOps compuesto por varias fases, eventos intermedios, métricas, artefactos y decisiones humanas. Por ello, Gradio se consideró menos adecuado para representar una experiencia de usuario completa alrededor del flujo de ejecución.

La opción finalmente adoptada fue una arquitectura basada en Next.js y FastAPI. Next.js permite construir una interfaz React con componentes reutilizables, mayor control sobre la experiencia de usuario y una gestión más natural de pantallas, formularios y estados de la aplicación. FastAPI, por su parte, permite exponer el backend Python mediante endpoints tipados y mantener la lógica del pipeline separada de la capa visual. Tal separación encaja mejor con la arquitectura del proyecto, ya que el backend ejecuta el flujo MLOps y emite eventos, mientras que el frontend los consume y los presenta al usuario. Además, el canal WebSocket permite transmitir eventos del pipeline en tiempo real desde el backend hacia el frontend sin bloquear la interfaz.

La elección de Next.js y FastAPI introduce más complejidad inicial que Streamlit o Gradio, ya que obliga a mantener dos servicios diferenciados y a desarrollar una interfaz web de forma explícita. Sin embargo, esta complejidad resulta justificada por los requisitos del prototipo: visualización progresiva de eventos, formularios de aprobación humana, separación entre lógica de ejecución e interfaz, y posibilidad de desplegar frontend y backend como servicios independientes. En consecuencia, el stack seleccionado ofrece una base más adecuada para una aplicación MLOps interactiva que debe mostrar el estado del sistema durante la ejecución y no solo resultados finales. En la tabla 4 se comparan las alternativas consideradas para la interfaz de usuario.



Criterio	Next.js+FastAPI	Streamlit	Gradio
Soporte nativo HTTP + WebSocket	Sí	Parcial	Parcial
Componentes reactivos interactivos	Sí	Parcial	Parcial
Flujos HITL con estado persistente	Sí	Parcial	No
Separación limpia frontend/backend	Sí	No	No
Despliegue en contenedores independientes	Sí	Parcial	Parcial
Tiempo de desarrollo del prototipo	Alto	Bajo	Bajo
Curva de aprendizaje	Alta	Baja	Baja
Código exclusivamente Python	No	Sí	Sí

Tabla 4: Comparativa de alternativas para la interfaz de usuario.

4. Arquitectura desarrollada

4.1. Visión general del sistema

La arquitectura del sistema se organiza como un pipeline MLOps híbrido, en el que una columna vertebral determinista implementada en Python coordina componentes basados en modelos de lenguaje únicamente en aquellas fases donde aportan valor frente a una solución programática convencional. Dicha decisión aplica el principio de restricción agéntica descrito en la sección 2.8: las operaciones críticas del ciclo de vida del modelo deben mantenerse reproducibles, trazables y verificables, mientras que los modelos de lenguaje se emplean para tareas que requieren interpretación, planificación, explicación o interacción con herramientas externas.

El flujo principal se implementa mediante LangGraph como grafo de estados. En lugar de utilizar un supervisor LLM que decida dinámicamente qué agente debe actuar en cada momento, el sistema adopta una estructura secuencial y explícita, donde cada nodo cumple una función concreta y actualiza un estado compartido antes de ceder el control al siguiente paso. Tal elección reduce la complejidad de la orquestación y facilita la auditoría del pipeline,

ya que el recorrido principal queda definido de forma clara. Las transiciones del grafo se resuelven mediante lógica determinista en Python, especialmente en los puntos donde el sistema debe esperar una aprobación humana o decidir si puede continuar hacia la siguiente fase.

Dentro de este grafo conviven tres tipos de componentes. En primer lugar, aparecen nodos agénticos, como el validador de datos y el planificador de modelos, que utilizan un LLM junto con herramientas externas para interpretar información, recuperar conocimiento y generar decisiones estructuradas. En segundo lugar, existen nodos que emplean un LLM de forma más acotada, sin acceso directo a herramientas, como el componente encargado de redactar el informe de evaluación a partir de métricas y resultados ya calculados. Por último, el núcleo operativo del sistema se mantiene determinista e incluye la ejecución del entrenamiento, la optimización de hiperparámetros, la evaluación de los candidatos, la promoción del algoritmo seleccionado y la actualización del almacén de ejecuciones previas.

La separación entre estos componentes permite evitar que el LLM controle directamente las fases más sensibles del pipeline. El planificador puede proponer un plan de entrenamiento, pero dicho plan debe ajustarse a contratos Pydantic y a un registro explícito de modelos soportados. Del mismo modo, el entrenamiento no se ejecuta mediante código generado por un agente, sino mediante un ejecutor determinista que consulta el registro de modelos, aplica los espacios de búsqueda definidos y registra los resultados en MLflow. Esta estructura permite combinar flexibilidad en la toma de decisiones con control programático en las operaciones que requieren reproducibilidad.

La arquitectura incorpora además dos puntos de supervisión humana. El primero aparece tras la validación inicial del dataset, donde el usuario puede revisar el resultado antes de permitir que el sistema continúe hacia la planificación y el entrenamiento. El segundo se sitúa antes de la promoción final del modelo, de forma que la decisión de aceptar el candidato seleccionado no queda completamente automatizada. Estos mecanismos human-in-the-loop se integran en el grafo mediante el estado compartido y permiten suspender, revisar y reanudar la ejecución con contexto suficiente.

En torno al grafo se sitúa la capa de infraestructura de la aplicación. FastAPI actúa como backend del sistema, mientras que la interfaz desarrollada en Next.js permite al usuario cargar datos, iniciar ejecuciones, seguir el progreso del pipeline, visualizar eventos intermedios y responder a los puntos de aprobación humana. La comunicación entre ambas capas combina peticiones HTTP y un canal WebSocket. Las peticiones HTTP se utilizan para operaciones puntuales de tipo petición-respuesta, como iniciar una ejecución, subir el dataset y el esquema de entrada, consultar información auxiliar o enviar las decisiones de aprobación humana. Estas acciones tienen una semántica clara de comando: el frontend envía una solicitud concreta y el backend responde con el resultado correspondiente.

El canal WebSocket se utiliza con un propósito distinto. Una vez iniciada una ejecución y obtenido su identificador, el frontend abre una conexión persistente asociada al `run_id`. A través de esa conexión, el backend envía objetos `PipelineEvent` en formato JSON a medida que el grafo avanza por sus nodos. Aunque WebSocket es un protocolo bidireccional, en esta implementación se emplea principalmente como canal de notificación desde el backend hacia el frontend. Dicha separación responde a un patrón de diseño simple: comandos mediante HTTP y eventos en tiempo real mediante WebSocket. De este modo, las acciones discretas del usuario mantienen una lógica de petición-respuesta fácil de validar y depurar, mientras que los eventos generados de forma asíncrona por LangGraph se transmiten al navegador sin necesidad de realizar consultas periódicas al backend.

MLflow proporciona seguimiento de experimentos, almacenamiento de artefactos y registro del modelo campeón. SQLite actúa como almacén local del Experience Pool, conservando información estructurada de ejecuciones anteriores que puede recuperarse en futuras planificaciones. Finalmente, Docker permite empaquetar los distintos servicios y facilitar la reproducción del entorno de ejecución en otros equipos. En la figura 4.1 se muestra una visión general de la arquitectura del sistema con sus principales componentes.

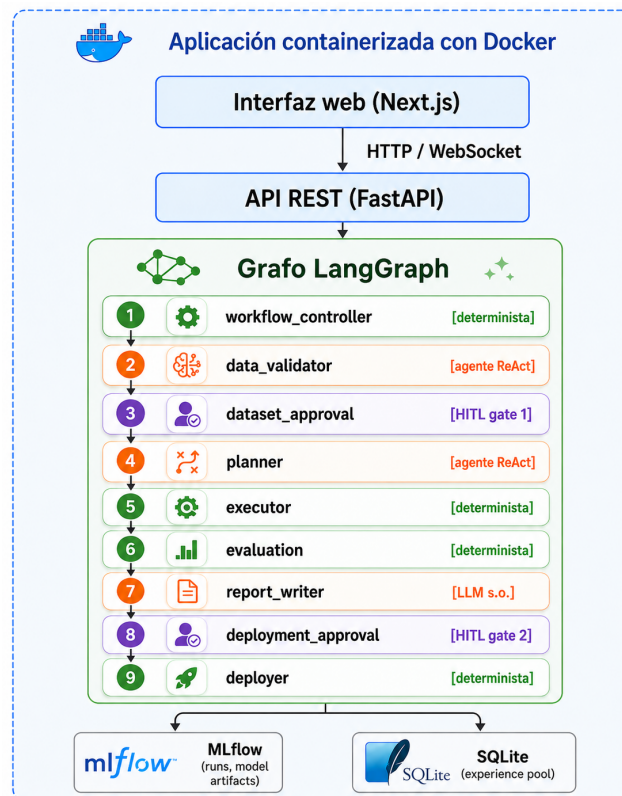


Figura 4.1: Visión general de la arquitectura del sistema. LLM s.o. = nodo LLM con salida estructurada sin bucle de herramientas.

4.2. Contrato de entrada del pipeline

El pipeline no recibe únicamente un conjunto de datos, sino también un contrato de entrada que define el problema predictivo que debe resolverse. Este contrato se proporciona en formato JSON y actúa como punto de partida del flujo: describe el tipo de tarea, la variable objetivo, las columnas relevantes, las restricciones de validación y, cuando procede, la estructura temporal del problema. De este modo, el sistema no intenta deducir completamente desde cero el significado del dataset, sino que parte de una especificación inicial proporcionada por la persona experta.

Esta es una decisión central en la arquitectura. Un sistema MLOps asistido por agentes necesita contexto para interpretar los datos de forma segura. Sin una definición explícita del

problema, un agente podría identificar columnas relevantes de forma ambigua, asumir una variable objetivo incorrecta, confundir una columna temporal con una variable ordinaria o tratar como disponible a futuro una variable que en realidad no se conocería en un escenario real de predicción. El contrato JSON evita esta ambigüedad al establecer una frontera clara entre la especificación humana del problema y las fases automatizadas del pipeline.

El contrato incluye, en primer lugar, metadatos generales de la tarea. Entre ellos, el campo denominado `problem_type` indica si el sistema debe resolver un problema de clasificación, regresión o predicción temporal. Otros campos, como `name` o `description`, proporcionan una identificación legible del caso de uso y ayudan a contextualizar la ejecución. En problemas de forecasting, el contrato incorpora además campos específicos como `target_column`, `datetime_column`, `forecast_horizon` y `frequency`. Estos elementos permiten construir una interpretación temporal del dataset, definir el horizonte de predicción y seleccionar estrategias de validación coherentes con la naturaleza secuencial de los datos.

Un aspecto especialmente relevante aparece en la declaración de variables exógenas. El contrato permite indicar qué variables externas acompañan a la serie objetivo y cuál es su disponibilidad futura. Una variable marcada como `known_future` puede utilizarse en validación y predicción porque sus valores futuros son conocidos de antemano, como ocurre con calendarios de festivos o variables planificadas. En cambio, una variable marcada como `unknown_future` no puede utilizarse directamente con valores reales futuros durante la validación, ya que eso introduciría filtración de información. En estos casos, el sistema debe aplicar una estrategia de extensión o estimación antes de usarla en el modelo. Dicha distinción constituye uno de los mecanismos principales para mantener una evaluación fiel a un escenario real de despliegue.

El contrato también describe el esquema esperado de columnas. Para cada columna puede especificarse su nombre, tipo de dato, descripción, obligatoriedad, posibilidad de valores nulos, restricciones numéricas, valores permitidos o papel como clave primaria. Además, pueden añadirse pistas de mapeo que ayuden al agente de validación a relacionar las columnas esperadas con los nombres presentes en los ficheros de entrada. Estas pistas no sustituyen a la validación programática, pero proporcionan contexto semántico útil cuando los datos

proceden de varias fuentes o cuando existen nombres de columnas próximos entre sí.

Durante la ejecución, el contrato de entrada alimenta varias fases del sistema. El agente de validación lo utiliza para comprobar si los datos recibidos son compatibles con el problema definido. A partir de esta validación se construye un perfil del conjunto de datos, que resume propiedades relevantes como tamaño, tipos de variables, valores ausentes, estructura temporal o presencia de variables exógenas. Posteriormente, el planificador emplea este perfil, junto con el contrato original, para proponer un plan de entrenamiento compatible con el tipo de problema. Finalmente, el ejecutor determinista utiliza dicha información para seleccionar particiones, estrategias de validación, modelos candidatos y tratamiento de variables exógenas.

El contrato JSON representa, por tanto, una pieza de gobierno del sistema. La persona experta define el marco del problema, el objetivo predictivo, las columnas relevantes y las restricciones principales; el pipeline utiliza esa información para automatizar las fases posteriores con mayor seguridad, trazabilidad y coherencia. Tal aproximación evita delegar en el LLM una tarea que requiere conocimiento técnico y de dominio, y refuerza la separación entre especificación humana, razonamiento asistido por LLM y ejecución determinista.

Un ejemplo sencillo de este esquema JSON podría ser el siguiente:

Fragmento de código 1: Ejemplo simplificado de esquema JSON de entrada.

```
1 {  
2   "problem_type": "forecasting",  
3   "name": "energy_forecast",  
4   "target_column": "kwh_consumed",  
5   "datetime_column": "week_date",  
6   "forecast_horizon": 4,  
7   "frequency": "W",  
8   "exogenous_columns": [  
9     {  
10      "name": "avg_temp_c",  
11      "future_availability": "unknown_future"
```

```
12     },
13     {
14         "name": "is_holiday_week",
15         "future_availability": "known_future"
16     }
17 ],
18 "columns": [
19     {
20         "name": "week_date",
21         "dtype": "datetime",
22         "required": true,
23         "nullable": false,
24         "is_key": true
25     },
26     {
27         "name": "kwh_consumed",
28         "dtype": "float",
29         "required": true,
30         "nullable": false,
31         "min": 0.0
32     }
33 ]
34 }
```

4.3. Tipos de nodos y contratos de ejecución

El grafo de ejecución se compone de nodos especializados que comparten un mismo estado, pero no todos tienen la misma naturaleza ni el mismo grado de autonomía. En Lang-Graph, cada nodo puede entenderse como una unidad de ejecución que recibe el estado actual del pipeline, realiza una tarea concreta y devuelve una actualización parcial de dicho estado. Esta actualización no sustituye todo el contenido anterior, sino que modifica únicamente los

campos necesarios para continuar la ejecución. De este modo, el estado actúa como memoria operativa del sistema y como mecanismo de comunicación entre fases.

Dicha estructura permite que los nodos intercambien información sin depender de un historial conversacional acumulado. El flujo no se construye como una conversación larga entre agentes, sino como una sucesión de transformaciones sobre un estado compartido. Cada nodo lee los campos que necesita, ejecuta su lógica y escribe nuevos resultados, como informes de validación, planes de entrenamiento, métricas, decisiones humanas o referencias a artefactos. El controlador determinista inspecciona después ese estado actualizado y decide el siguiente paso.

La arquitectura distingue cuatro tipos principales de nodos: deterministas, agénticos con herramientas, LLM acotados y nodos de supervisión humana. Esta clasificación permite asignar a cada fase el nivel de autonomía estrictamente necesario. Las operaciones críticas, como entrenamiento, evaluación o promoción lógica del modelo, permanecen bajo control programático. Los modelos de lenguaje se reservan para tareas donde aportan valor, como interpretar el contexto del dataset, planificar modelos candidatos o generar explicaciones estructuradas para el usuario. En la tabla 5 se resumen los nodos que componen el grafo MLOps y su función dentro del flujo.

No todos los nodos del grafo utilizan modelos de lenguaje. El controlador, el ejecutor, el evaluador y el desplegador permanecen implementados mediante código determinista. En cambio, los nodos que requieren interpretación, planificación o generación estructurada se apoyan en la API de OpenAI. La selección del modelo no se plantea como una decisión uniforme para todo el sistema, sino como un compromiso entre capacidad, coste y criticidad de cada tarea. En la tabla 6 se detallan los modelos de lenguaje utilizados por cada nodo LLM del sistema.

Esta asignación refleja el criterio general de la arquitectura: emplear modelos más capaces en los componentes donde una salida incorrecta puede condicionar el resto del pipeline, y alternativas más ligeras en tareas de generación estructurada sobre información ya calculada. El validador y el planificador utilizan un modelo de mayor capacidad con razonamiento



Nodo	Tipo	Responsabilidad principal
workflow_controller	Determinista	Centraliza el enrutamiento del grafo a partir del estado compartido.
data_validator	Agente con herramientas	Valida, limpia y prepara el dataset de acuerdo con el contrato de entrada.
dataset_approval	Puerta HITL	Suspende el flujo para que el usuario apruebe o rechace el dataset validado.
planner	Agente con herramientas y salida estructurada	Propone un plan de entrenamiento compatible con el problema, el registro de modelos y las evidencias recuperadas.
executor	Determinista	Entrena modelos candidatos, optimiza hiperparámetros y registra resultados.
evaluation	Determinista	Compara el candidato seleccionado con umbrales y, cuando procede, con el campeón actual.
report_writer	LLM estructurado	Genera un informe de evaluación a partir de métricas y resultados ya calculados.
deployment_approval	Puerta HITL	Solicita aprobación humana antes de promover el modelo seleccionado.
deployer	Determinista	Registra y promueve el modelo en MLflow como modelo campeón.

Tabla 5: Resumen de nodos del grafo MLOps.

Nodo	Modelo	Razonamiento	Coste API	Justificación
data_validator	gpt-5.4-mini	Medio	0,75 USD entrada / 4,50 USD salida por 1M tokens	Requiere interpretar el contrato JSON, hacer joins entre tablas, seleccionar herramientas de validación y reaccionar ante incidencias del dataset. Se emplea un modelo equilibrado entre capacidad de razonamiento, seguimiento de instrucciones y coste.
planner	gpt-5.4-mini	Medio	0,75 USD entrada / 4,50 USD salida por 1M tokens	Es el nodo LLM más crítico: debe consultar herramientas, incorporar evidencias, proponer modelos candidatos y producir un plan estructurado validable. Se prioriza fiabilidad frente a coste mínimo.
report_writer	gpt-5.4-nano	-	0,20 USD entrada / 1,25 USD salida por 1M tokens	No decide el flujo ni entrena modelos; transforma métricas y resultados ya calculados en un informe estructurado. Se utiliza un modelo más ligero para reducir coste y latencia.

Tabla 6: Modelos de lenguaje utilizados por los nodos LLM del sistema.[29]

medio habilitado, ya que ambos deben interpretar contexto, utilizar herramientas y producir resultados que alimentan fases posteriores del flujo. En el caso del validador, esta decisión se justifica especialmente por la incorporación de herramientas de unión entre tablas: el componente debe analizar varios ficheros de entrada, identificar posibles claves de unión, evaluar la compatibilidad entre ellas y ejecutar un plan de integración antes de validar el conjunto de datos resultante. En el caso del planificador, el razonamiento adicional permite sintetizar información procedente del contrato de entrada, el perfil del dataset, el registro de algoritmos, las reglas de conocimiento y las ejecuciones anteriores. En cambio, el redactor del informe trabaja sobre métricas y resultados ya generados, por lo que puede emplear un modelo más ligero. Esta configuración busca mejorar la calidad de las decisiones asistidas por LLM, asumiendo un posible incremento de latencia y coste.

Los nodos deterministas están implementados mediante código Python convencional y no invocan modelos de lenguaje. En esta categoría se encuentran el controlador de flujo, el ejecutor de entrenamiento, el módulo de evaluación y el módulo de despliegue lógico. Su responsabilidad consiste en aplicar reglas explícitas, ejecutar entrenamiento, calcular métricas, registrar artefactos o actualizar sistemas auxiliares. Al no depender de salidas generadas por un LLM, estos nodos proporcionan reproducibilidad y facilitan la depuración.

Los nodos agénticos incorporan un modelo de lenguaje junto con herramientas externas. En estos casos, el LLM no se limita a generar una respuesta textual, sino que puede decidir qué herramienta invocar, interpretar su resultado y continuar el proceso hasta producir una salida útil para el pipeline. El agente de validación pertenece a esta categoría: recibe el contrato de entrada, información sobre los ficheros disponibles y, si existe, retroalimentación procedente de un rechazo humano anterior. A partir de este contexto puede utilizar módulos de código para cargar datos, evaluar columnas candidatas para unión entre ficheros, ejecutar un plan de unión, aplicar mapeos de columnas, validar el esquema, revisar valores ausentes, imputar datos o detectar problemas temporales. Su salida no es una respuesta libre del modelo, sino una actualización estructurada del estado compartido que indica si la validación ha sido satisfactoria, qué incidencias se han detectado y qué información debe revisar el usuario.

El planificador de modelos también pertenece a la categoría de nodos agénticos con herramientas, aunque incorpora una restricción adicional: su salida debe ajustarse a un contrato estructurado. Su función no es entrenar algoritmos ni generar código, sino proponer un plan compatible con el problema definido. Para ello puede consultar el registro de modelos disponibles, recuperar ejecuciones anteriores similares, acceder a reglas de conocimiento estático e inspeccionar detalles concretos de los candidatos. El resultado esperado es un plan de entrenamiento validable mediante Pydantic, que indique los algoritmos candidatos a evaluar, los descartados con su justificación, el presupuesto de búsqueda y, en problemas de forecasting, la estrategia de validación y el tratamiento de variables exógenas. La decisión final de entrenamiento queda delegada al ejecutor determinista, que valida el plan y solo ejecuta modelos registrados.

La tabla 7 resume las herramientas disponibles para los dos nodos agénticos del sistema. Tales herramientas delimitan explícitamente qué acciones puede realizar cada agente, evitando que el LLM opere con total libertad.

Agente	Herramientas disponibles	Finalidad
data_validator	load_dataset, merge_datasets, evaluate_join_candidates, execute_join_plan, apply_column_mapping, validate_against_schema, check_missing_values, check_data_quality, impute_missing_values, parse_datetime_column, detect_temporal_gaps	Permiten cargar los datos, combinar varios ficheros, identificar claves de unión, aplicar mapeos de columnas, validar el contrato de entrada, revisar calidad del dataset, imputar valores ausentes y detectar problemas propios de series temporales.
planner	list_available_models, retrieve_similar_experiences, retrieve_ml_knowledge, inspect_model_details	Permiten consultar los modelos disponibles, recuperar ejecuciones previas similares, acceder a reglas de conocimiento estático e inspeccionar detalles de un modelo concreto antes de generar el plan de entrenamiento.

Tabla 7: Herramientas disponibles para los nodos agénticos del sistema.

Además de los agentes con herramientas, el sistema incorpora un nodo LLM acotado. Este componente utiliza un modelo de lenguaje, pero no sigue un bucle ReAct ni invoca herramientas externas. Su cometido consiste en transformar información ya calculada en una salida estructurada o explicativa. El redactor del informe de evaluación pertenece a esta categoría: recibe métricas, resultados de los candidatos evaluados, información del algoritmo seleccionado, umbrales aplicados y contexto de planificación, y produce un informe que ayuda al usuario a interpretar la decisión de promoción. Al no acceder directamente a herramientas ni controlar el flujo, su autonomía es menor que la de un agente.

Los nodos de supervisión humana actúan como puertas de control dentro del grafo. En lugar de ejecutar una decisión automática, suspenden el avance del pipeline hasta que el usuario revise la información disponible y emita una aprobación o rechazo. En caso de rechazo, el usuario puede proporcionar una explicación sobre el motivo de su decisión, que se incorpora al estado compartido como información de retroalimentación o feedback. La primera puerta aparece tras la validación del dataset y permite comprobar si los datos preparados son adecuados para continuar. La segunda aparece antes de la promoción del modelo seleccionado y permite decidir si el candidato debe aceptarse como modelo promovido. Estas decisiones se almacenan en el estado compartido, de forma que el controlador determinista puede decidir el siguiente paso mediante reglas explícitas.

La tabla 8 resume de forma simplificada qué información consume y qué información produce cada nodo; es decir, cómo interactúa cada nodo con el estado compartido. No pretende enumerar todos los campos internos del estado, sino mostrar cómo se propaga la información principal a lo largo del pipeline. Cada nodo recibe el estado completo y devuelve un diccionario parcial con únicamente los campos que modifica; el framework combina las actualizaciones automáticamente. Tal separación entre lectura y escritura parcial reduce el acoplamiento entre nodos y evita que un componente tenga que reconstruir o sobrescribir información que pertenece a otras fases.



Nodo	Información que utiliza	Información que incorpora al estado
workflow_controller	Estado global del pipeline: errores, validación, aprobaciones humanas, plan de entrenamiento, ejecución, evaluación, decisión de despliegue y contadores de intento.	Actualiza contadores de intento y, cuando procede, mensajes de error o reinicios de control. Principalmente decide el siguiente nodo mediante goto.
data_validator	Ficheros de entrada (resumidos), contrato JSON, comentarios de rechazo si existen y contadores de intento.	Dataset procesado, informe de validación, resumen del dataset, tipo de problema, metadatos de tarea y, si hay varios ficheros, plan de unión, evaluaciones de joins y número de filas base.
dataset_approval	Dataset procesado, informe de validación, resumen del dataset, tipo de problema e información del intento actual.	Decisión humana de aprobación o rechazo del dataset y, si procede, comentario de corrección para reintentar la validación.
planner	Dataset procesado, tipo de problema, metadatos de tarea, contrato de entrada, registro de modelos, reglas de conocimiento, experiencias previas y contexto de validación.	Plan de entrenamiento, análisis del planificador, evidencias utilizadas, advertencias, estado del plan, traza de herramientas, contexto de validación y registro estructurado de salida.
executor	Plan de entrenamiento, dataset procesado, tipo de problema, metadatos de tarea y salida estructurada del planificador.	Identificador de ejecución MLflow, métricas del candidato campeón, modelo seleccionado, rutas de particiones, ruta del modelo entrenado y registro de experiencia.
evaluation	Métricas del candidato, tipo de problema, umbrales y campeón actual en MLflow.	Decisión de evaluación, métricas comparativas, umbrales aplicados e informe determinista.
report_writer	Resultados de entrenamiento, evaluación, planificación, métricas comparativas, umbrales aplicados y evidencias utilizadas.	Informe estructurado de auditoría y estado de generación del informe.
deployment_approval	Informe de evaluación, informe de auditoría, métricas comparativas, plan de entrenamiento, identificador de ejecución y modelo candidato seleccionado.	Decisión humana de promoción o rechazo.
deployer	Identificador de ejecución MLflow, modelo candidato seleccionado y decisión de promoción aprobada.	Estado de despliegue lógico, decisión final de despliegue y URI del modelo registrado como campeón.

Tabla 8: Resumen simplificado de entradas y salidas principales de los nodos del grafo.

Conviene distinguir el papel del `workflow_controller` del resto de nodos del grafo. El controlador determinista no genera un artefacto de dominio, ni valida datos, ni entrena modelos, ni produce informes. Su responsabilidad consiste en emitir decisiones de enrutamiento y, en casos concretos, aplicar reinicios de control sobre el estado, por ejemplo al reintentar una validación tras un rechazo humano. Por este motivo, sus actualizaciones pueden expresarse directamente dentro de la lógica de control.

En cambio, los ocho nodos restantes del flujo escriben resultados asociados a una fase concreta del pipeline: validación, aprobación humana, planificación, entrenamiento, evaluación, generación de informe, aprobación de despliegue y promoción del modelo. En estos casos, las actualizaciones del estado se tratan como contratos tipados de salida, ya que cada nodo debe devolver un conjunto definido de campos que será consumido por fases posteriores. Dicha separación permite mantener el enrutamiento bajo una lógica determinista flexible, mientras que las salidas de dominio permanecen estructuradas, validables y trazables.

La tabla anterior describe el contrato externo de cada nodo con respecto al estado compartido: qué información necesita para operar y qué resultados incorpora al flujo. Sin embargo, en los nodos basados en LLM existe además un segundo nivel de contrato, relacionado con su comportamiento interno. No basta con definir qué campos leen y escriben; también es necesario acotar cómo deben interpretar la tarea, qué restricciones deben respetar, qué herramientas pueden utilizar y en qué formato deben devolver sus resultados.

Por este motivo, todos los nodos basados en LLM se apoyan en plantillas de prompt definidas fuera del código principal. En la implementación desarrollada, estas instrucciones se organizan en ficheros YAML; y se cargan en lo que técnicamente se denomina `system_prompt`, lo que permite separar la lógica programática de las instrucciones lingüísticas que guían el comportamiento del modelo. Cada plantilla define el rol del componente, el objetivo de la tarea, las restricciones que debe respetar, el formato esperado de salida y, cuando procede, el conjunto de herramientas disponibles. Tal separación facilita revisar y ajustar el comportamiento de los agentes sin modificar la lógica del grafo, y contribuye a mantener una arquitectura más trazable y mantenible.

La existencia de contratos explícitos no se limita a los prompts. Las salidas de los nodos críticos también se validan mediante estructuras tipadas, especialmente cuando interviene un LLM. El plan de entrenamiento, el informe de evaluación o los registros de experiencia no se aceptan como texto libre, sino como objetos estructurados que deben cumplir un formato esperado. Esta validación reduce el riesgo de que una salida generada por el modelo introduzca campos incompatibles, modelos no registrados o decisiones que el ejecutor no pueda interpretar.

En conjunto, esta tipología de nodos permite construir un pipeline híbrido y controlado. Los agentes intervienen cuando se requiere interpretación o planificación, los nodos LLM acotados generan explicaciones estructuradas, los módulos deterministas ejecutan las operaciones reproducibles y las puertas humanas introducen supervisión en decisiones críticas. La arquitectura evita delegar todo el flujo en un único agente generalista y distribuye las responsabilidades según el nivel de autonomía que cada fase realmente necesita.

4.4. Descubrimiento de joins en el agente de validación

El descubrimiento de joins constituye una de las funcionalidades más relevantes del sistema, ya que amplía el alcance del pipeline más allá de la validación de un único fichero aislado. En muchos escenarios reales, la información necesaria para resolver un problema predictivo no aparece concentrada en una sola tabla, sino repartida entre varias fuentes relacionadas. Por este motivo, se considera conveniente dedicar una subsección específica a esta capacidad: combina razonamiento semántico del agente, evaluación determinista mediante herramientas y supervisión humana antes de aceptar el dataset canónico resultante.

Cuando el pipeline recibe varios ficheros de entrada, el sistema no puede limitarse a validar cada tabla de forma independiente, ya que el resto del flujo opera sobre un único dataset canónico. Por ello, el agente `data_validator` debe identificar si existen relaciones útiles entre los ficheros, proponer claves de unión y construir un plan de integración compatible con el contrato de entrada. Esta fase resulta especialmente relevante en problemas reales,

donde los datos pueden llegar repartidos en varias tablas y las claves de unión no siempre aparecen con el mismo nombre ni con una cobertura perfecta.

El diseño del subproceso sigue el principio de restricción agéntica aplicado en el resto de la arquitectura. El componente interviene para reducir el espacio de búsqueda y razonar sobre la plausibilidad semántica de las claves, pero no calcula métricas ni ejecuta físicamente las uniones. Para que pueda realizar este razonamiento sin acceder a las tablas completas, el nodo `data_validator` le proporciona un contexto inicial con un resumen estructurado de todos los ficheros recibidos. Este contexto permite al agente conocer qué tablas existen, qué columnas contiene cada una, qué tipos de datos presentan, cuántos valores distintos aparecen en cada campo y qué aspecto tienen algunas filas de muestra. A partir de esa información, puede decidir qué tabla parece más adecuada como base del conjunto canónico y qué columnas podrían funcionar como claves candidatas de unión.

Dicho resumen se construye antes de llamar al modelo mediante código Python ordinario. En concreto, el nodo calcula para cada tabla y columna propiedades como tipo de dato, número de valores no nulos, tasa de nulos, cardinalidad, ratio de unicidad, rangos y algunas filas de muestra. El resultado se inserta directamente en el mensaje de entrada del agente; por tanto, el perfilado no forma parte del bucle ReAct ni constituye una herramienta invocable por el LLM: el agente recibe los perfiles ya calculados, razona sobre ellos y solo después puede invocar las herramientas reales de join, encargadas de evaluar candidatos y ejecutar el plan de integración.

A partir de estos perfiles y del contrato JSON, el agente selecciona primero la tabla base. Esta tabla fija la cardinalidad del conjunto de datos canónico final y se elige principalmente por la cobertura de columnas objetivo del esquema; en caso de empate, se prioriza la tabla con mayor número de filas. Las tablas restantes se tratan como fuentes auxiliares que pueden enriquecer el dataset base mediante left joins, preservando así la población principal del problema.

Una vez seleccionada la tabla base, el agente propone un conjunto reducido de candidatos de join. Esta propuesta no se basa únicamente en coincidencias exactas de nombres

de columnas, sino también en el contenido y significado probable de los campos. Por ejemplo, columnas como `store_id`, `store_code` o `store_ref` pueden representar una misma entidad aunque sus nombres no coincidan literalmente. Del mismo modo, en problemas temporales pueden detectarse claves compatibles basadas en fechas, semanas, meses o periodos. Para acotar el coste y evitar una exploración combinatoria excesiva, el sistema limita el número de candidatos evaluados.

Tras esta propuesta inicial, entran en juego las herramientas deterministas. La herramienta de evaluación mide, para cada candidato, la cobertura de claves, la contención entre conjuntos de valores, el solapamiento, la relación inferida entre columnas, el riesgo de duplicación y el posible multiplicador de filas tras la unión. Estas métricas permiten distinguir entre un join útil, un join parcial pero aceptable y una unión que podría distorsionar el dataset o introducir errores en fases posteriores del pipeline.

Con las evaluaciones disponibles, el agente compone un plan de integración estructurado. Este plan indica qué tabla actúa como base, qué tablas auxiliares deben incorporarse, qué claves se utilizarán, qué candidatos se rechazan y qué advertencias deben revisarse. El agente no puede seleccionar un join que no haya sido respaldado por la evaluación determinista, especialmente si presenta cobertura insuficiente, relación muchos-a-muchos o riesgo elevado de explosión de filas.

La ejecución material del plan queda delegada a una herramienta programática. Esta herramienta aplica únicamente left joins sobre la tabla base, deduplica la clave derecha cuando es necesario, resuelve colisiones de nombres de columnas, bloquea uniones sin solapamiento y verifica al final que las columnas requeridas por el contrato estén presentes. Si alguna de estas garantías falla, el sistema detiene la ejecución de forma controlada en lugar de continuar con un dataset canónico incorrecto.

El resultado de esta fase se incorpora al estado compartido del pipeline en forma de dataset procesado, informe de validación, plan de join y métricas de evaluación de candidatos. Dicha información puede ser revisada posteriormente en la puerta de aprobación del conjunto de datos. De este modo, el descubrimiento de joins combina tres niveles de control: el

agente aporta razonamiento semántico, las herramientas proporcionan medición y ejecución reproducible, y el usuario conserva la posibilidad de revisar las decisiones de integración antes de continuar hacia la planificación y el entrenamiento.

4.5. Patrón controlador determinista y agentes especializados

Los sistemas multi-agente descritos en la sección 2.2 recurren habitualmente a un agente supervisor que utiliza un LLM para decidir qué agente debe actuar a continuación. Este trabajo adopta una variante distinta, motivada por el principio de restraint agéntico: el nodo orquestador es un controlador de flujo determinista, implementado como una máquina de estados Python, que toma todas las decisiones de enrutamiento a partir del estado compartido sin invocar ningún modelo de lenguaje.

La justificación de esta elección es directa. En este pipeline, cada decisión de enrutamiento es deducible a partir de campos booleanos y cadenas de texto del estado: si la validación pasó, si el plan de entrenamiento existe, si la evaluación superó los umbrales. Sustituir esta lógica por una llamada LLM introduciría no determinismo, latencia y tokens adicionales sin añadir ninguna capacidad de razonamiento real. el enrutamiento permanece bajo control programático, mientras que los modelos de lenguaje se reservan para los nodos donde aportan valor.

El controlador no solo decide el siguiente nodo, sino que también gestiona situaciones de recuperación. Por ejemplo, mantiene un contador de intentos para evitar bucles indefinidos cuando un agente no consigue completar correctamente su tarea. Si la validación de datos falla, el flujo puede volver a ejecutar el agente de validación hasta alcanzar un número máximo de intentos. Del mismo modo, si el usuario rechaza el dataset en la primera puerta de supervisión humana, el sistema reinicia la validación y vuelve a dar una oportunidad al flujo. En cambio, si se supera el límite de intentos o si el usuario rechaza la promoción final del modelo, el controlador finaliza la ejecución de forma explícita.

Estos reintentos no se ejecutan de forma ciega. Cuando el flujo vuelve a invocar un nodo tras un fallo de validación, una salida inválida o un rechazo humano, el sistema incorpora al nuevo intento información sobre el motivo que ha provocado la repetición. Este feedback permite que el agente no repita exactamente la misma actuación, sino que pueda corregir su comportamiento en función del error detectado o de la observación recibida. En la implementación actual, el número de intentos se limita mediante un contador almacenado en el estado compartido, de forma que el sistema puede ofrecer oportunidades de corrección sin quedar atrapado en ciclos indefinidos.

Fragmento de código 2: Extracto simplificado del controlador determinista de flujo.

```
1 def workflow_controller(state: dict[str, Any]) -> Command:
2     counts = state.get("agent_attempt_counts") or {}
3     max_attempts = settings.max_attempts_per_agent
4
5     if state.get("error_message"):
6         return Command(goto=END)
7
8     if not state.get("validation_passed"):
9         if counts.get("data_validator", 0) >= max_attempts:
10             return Command(
11                 goto=END,
12                 update={"error_message": "data_validator: max attempts
13                     reached"},
14             )
15
16     counts_next = {
17         **counts,
18         "data_validator": counts.get("data_validator", 0) + 1,
19     }
20     return Command(
21         goto="data_validator",
22         update={"agent_attempt_counts": counts_next},
23     )
```



```
24     if state.get("dataset_approved") is None:
25         return Command(goto="dataset_approval")
26
27     if state.get("dataset_approved") is False:
28         counts_next = {
29             **counts,
30             "data_validator": counts.get("data_validator", 0) + 1,
31         }
32
33         if counts_next["data_validator"] > max_attempts:
34             return Command(
35                 goto=END,
36                 update={"error_message": "Dataset rejected after max
37                     attempts"},
38             )
39
40         return Command(
41             goto="data_validator",
42             update={
43                 "dataset_approved": None,
44                 "validation_passed": False,
45                 "agent_attempt_counts": counts_next,
46             },
47         )
48
49     if not state.get("training_plan"):
50         return Command(goto="planner")
51
52     if not state.get("training_run_id"):
53         return Command(goto="executor")
54
55     if state.get("evaluation_passed") is None:
56         return Command(goto="evaluation")
```

```
57     if state.get("evaluation_report_audit") is None:
58         return Command(goto="report_writer")
59
60     if state.get("evaluation_passed") is False:
61         return Command(goto=END)
62
63     if state.get("deployment_approved") is None:
64         return Command(goto="deployment_approval")
65
66     if state.get("deployment_approved") is False:
67         return Command(goto=END)
68
69     if state.get("deployment_decision") == "pending":
70         return Command(goto="deployer")
71
72     return Command(goto=END)
```

Todos los nodos del grafo, tanto agénticos como deterministas, devuelven un `Command` con destino `workflow_controller`. El controlador centraliza así toda la lógica de transición, lo que facilita la trazabilidad y la depuración: para saber por qué el sistema tomó una decisión de enrutamiento concreta, basta con inspeccionar el estado en el momento en que el controlador fue invocado.

Tal diseño también mejora la robustez del sistema. Los agentes pueden fallar, producir una salida inválida o requerir una nueva iteración tras una revisión humana, pero esos casos no quedan gestionados por el propio LLM, sino por reglas explícitas del controlador. De este modo, la recuperación ante errores, los límites de reintento y las condiciones de parada permanecen definidos de forma programática y auditable.

4.6. Seguimiento experimental y registro de modelos con MLflow

MLflow actúa como la capa de seguimiento experimental del sistema. Durante la ejecución del entrenamiento, el ejecutor determinista registra información asociada a los modelos candidatos evaluados, incluyendo parámetros, hiperparámetros, métricas, artefactos y referencias al modelo entrenado. Dicha información permite reconstruir qué se ha probado en una ejecución concreta y comparar el rendimiento de distintos candidatos bajo un mismo protocolo de validación.

En la arquitectura propuesta, MLflow no se utiliza únicamente como almacén de métricas, sino también a modo de registro del ciclo de vida del modelo. El modelo seleccionado por el ejecutor se registra como artefacto y, si supera la fase de evaluación y la aprobación humana correspondiente, puede promocionarse dentro del Model Registry mediante el alias *champion*. Esta promoción debe entenderse como un despliegue lógico dentro del registro de modelos, no como la exposición automática de un endpoint de inferencia en producción.

La separación entre entrenamiento y promoción resulta importante para mantener control sobre las decisiones críticas. El ejecutor puede entrenar y seleccionar un candidato según métricas objetivas, pero la promoción final queda condicionada por la evaluación determinista y por una puerta de supervisión humana. De este modo, MLflow proporciona trazabilidad sobre los experimentos y artefactos, mientras que el grafo de ejecución mantiene el control sobre cuándo un modelo puede considerarse candidato promovido.

MLflow también cumple una función de auditoría técnica. Cada ejecución conserva identificadores de run, métricas obtenidas y rutas a artefactos generados, lo que permite revisar posteriormente el comportamiento del pipeline. Tal información resulta útil tanto para comparar modelos dentro de una misma ejecución como para analizar la evolución de los resultados a lo largo de distintas pruebas. Sin embargo, MLflow no sustituye al almacén de experiencias del sistema, ya que su objetivo principal es el seguimiento experimental y el registro de modelos, no la recuperación semántica o estructurada de ejecuciones similares.

4.7. Experience Pool como memoria empírica del sistema

El Experience Pool actúa como la memoria empírica del sistema. Su objetivo es conservar una fotografía estructurada de ejecuciones anteriores que pueda reutilizarse como evidencia en nuevas planificaciones. Mientras que MLflow registra métricas, artefactos y modelos desde una perspectiva experimental, el Experience Pool almacena la experiencia en un formato pensado para la recuperación: qué tipo de problema se resolvió, qué características tenía el dataset, qué modelos se evaluaron, cuál fue seleccionado, qué métricas se obtuvieron y qué estrategias de validación o tratamiento de variables se aplicaron.

Cada experiencia resume los elementos principales de una ejecución del pipeline. En problemas de clasificación o regresión, esta información incluye el perfil del conjunto de datos, el tipo de problema, la estrategia de validación utilizada, los modelos candidatos evaluados, sus resultados y el modelo seleccionado. En problemas de predicción temporal, la experiencia puede incorporar además la disponibilidad futura de variables exógenas y las estrategias aplicadas para extenderlas. Al disponer de esta información, el sistema no trata cada nueva ejecución como un caso aislado, sino que puede apoyarse en resultados observados previamente.

Esta idea se relaciona con trabajos recientes como MLCopilot, que propone utilizar modelos de lenguaje para resolver nuevas tareas de aprendizaje automático apoyándose en experiencias previas y razonamiento sobre problemas similares [30]. En este trabajo se adopta una versión más acotada y trazable de esa aproximación: el Experience Pool no genera soluciones completas de forma autónoma, sino que almacena ejecuciones anteriores en un formato estructurado para proporcionar evidencia empírica al planificador. De este modo, las experiencias recuperadas orientan la selección de modelos, pero no sustituyen al registro declarativo, a las reglas estáticas ni a las validaciones deterministas posteriores.

Para complementar esta memoria empírica, el sistema cuenta con una base de conocimiento estática codificada en YAML. Este *static knowledge* incluye reglas y recomendaciones predefinidas sobre el uso de modelos y estrategias en función de características gene-

rales del dataset (por ejemplo, qué modelos convienen para datasets pequeños o grandes, o qué estrategias de validación se aplican según el tipo de serie temporal). A diferencia del Experience Pool, esta base no se actualiza automáticamente a partir de las ejecuciones: permanece fija hasta que los desarrolladores la modifiquen y sirve como referencia para matizar y contextualizar las decisiones.

La persistencia del Experience Pool se implementa mediante SQLite, complementado con copias JSON auditables. SQLite proporciona una base local, ligera y suficiente para la escala del prototipo, permitiendo consultar registros de forma estructurada sin desplegar un servidor externo. Las copias JSON, por su parte, facilitan la inspección manual y la trazabilidad de cada experiencia concreta. Esta combinación separa claramente el uso operativo de la memoria (orientado a consultas) de su uso documental o auditable (orientado a la revisión humana).

Durante la planificación, el sistema puede recuperar ejecuciones similares a partir del perfil del dataset y del tipo de problema. Esta recuperación no se basa en una conversación previa ni en memoria implícita del modelo, sino en registros persistentes y consultables. Para ello, el sistema aplica primero un filtrado estricto por `problem_type`, de modo que un problema de forecasting solo se compara con casos previos de forecasting, uno de clasificación con registros de clasificación y uno de regresión con registros de regresión. Este filtrado evita que el planificador reciba evidencias procedentes de tareas incompatibles.

Una vez aplicado este filtro, la similitud entre el nuevo dataset y las experiencias almacenadas se calcula mediante una puntuación determinista sobre campos del `DatasetProfile`. Cada campo relevante tiene un peso fijo según su importancia para comparar problemas predictivos. Los campos más influyentes reciben peso 3, los de relevancia intermedia peso 2 y los descriptivos generales peso 1. La tabla 9 resume estos pesos.

Peso	Campos del perfil
3	n_rows, history_length, horizon_difficulty, seasonality_detected
2	class_balance, n_classes, target_distribution, exogenous_features_available, frequency, trend_detected, stationarity
1	n_features, missing_rate, n_categorical_features, n_numerical_features

Tabla 9: Pesos utilizados para calcular la similitud entre perfiles de dataset en el *Experience Pool*.

El campo `problem_type` no suma puntuación, ya que se utiliza como condición previa de filtrado. Si dos experiencias no pertenecen al mismo tipo de problema, no se comparan. En cambio, cuando el tipo de problema coincide, el sistema compara los campos aplicables a esa familia de tarea y acumula la puntuación de aquellos que coinciden. Por ejemplo, en forecasting se consideran campos específicos como la longitud histórica, la frecuencia temporal, la dificultad del horizonte, la existencia de estacionalidad o la disponibilidad de variables exógenas; en clasificación se consideran además campos como el número de clases y el balance de clases.

La puntuación bruta se normaliza dividiéndola por la puntuación máxima alcanzable para el tipo de problema correspondiente. Esta normalización resulta necesaria porque no todos los campos existen para todos los tipos de tarea. Por ejemplo, un perfil de forecasting no contiene `n_classes` ni `class_balance`, mientras que un perfil de clasificación no contiene `frequency` ni `horizon_difficulty`. Por ello, el denominador no es la suma de todos los pesos posibles, sino la suma de los pesos aplicables a cada tipo de problema, es decir:

$$\text{similarity_score} = \frac{\text{score}}{\text{max_score}(\text{problem_type})}.$$

En la implementación actual, la puntuación máxima es 11 para clasificación, 9 para regresión y 24 para forecasting. El valor normalizado se expresa en el intervalo $[0, 1]$ y se almacena como `similarity_score`. Además, se asigna una etiqueta cualitativa de relevancia: *high* si la similitud es mayor o igual que 0,7; *medium* si se encuentra entre 0,4 y 0,7;

y *low* si es inferior a 0,4. Las experiencias se ordenan por puntuación bruta y, en caso de empate, se priorizan las más recientes.

Este mecanismo tiene dos ventajas para el alcance del prototipo. En primer lugar, es transparente y auditable: se puede explicar por qué una experiencia ha sido considerada similar observando los campos coincidentes y sus pesos. En segundo lugar, evita introducir una base de datos vectorial o embeddings semánticos cuando el perfil del dataset ya contiene atributos estructurados suficientes para comparar ejecuciones. Si el sistema creciese hasta manejar muchas más experiencias o incorporase descripciones textuales extensas, podría estudiarse una recuperación híbrida o vectorial. En la versión actual, una función de similitud ponderada sobre campos estructurados resulta más simple, controlable y coherente con la filosofía determinista del sistema.

Este esquema captura las dimensiones estructurales principales del perfil del dataset —tamaño, frecuencia, presencia y disponibilidad de variables exógenas, estacionalidad y estacionariedad— y resulta suficiente para el alcance del prototipo. Conviene reconocer, no obstante, que existen matices más finos que la métrica actual no distingue de forma explícita. Por una parte, no diferencia entre tipologías concretas de variables exógenas —por ejemplo, exógenas macroeconómicas correlacionadas, sensoriales independientes, meteorológicas o de calendario—, que en problemas reales condicionan tanto la estrategia de modelado como la interpretación del resultado. Por otra parte, no representa de forma directa la estructura multicíclica de la estacionalidad, que en series de alta frecuencia puede superponer ciclos diarios, semanales y anuales. Estas distinciones quedan parcialmente capturadas a través de otros campos correlacionados —como `frequency` o `exogenous_features_available`— lo que es coherente con el alcance abordado. Refinar la métrica con campos categóricos específicos para estas dimensiones constituye una evolución natural del sistema cuando el catálogo de experiencias crezca, y se recoge como tal en las líneas futuras del capítulo 6.

Conviene aclarar que la similitud no decide por sí misma qué modelos se entrenan. Su función dentro del Experience Pool es ordenar las experiencias previas y seleccionar aquellas que resultan más relevantes para construir el contexto empírico del planificador. Es decir, la métrica de similitud actúa como un mecanismo de recuperación de evidencia, no como una

regla automática de selección de modelos.

Esta distinción resulta importante porque el sistema combina tres fuentes de información diferentes. El registro declarativo de modelos define qué algoritmos existen realmente en el sistema y cuáles pueden ejecutarse para cada tipo de problema. La base de conocimiento estática recoge reglas expertas escritas manualmente, que se activan cuando el perfil del dataset cumple determinadas condiciones. El Experience Pool, por su parte, aporta evidencia empírica procedente de ejecuciones anteriores, ordenada mediante la métrica de similitud ponderada descrita anteriormente.

El planificador recibe estas fuentes como contexto y puede utilizarlas para proponer modelos candidatos, justificar descartes o ajustar decisiones relacionadas con la validación y el tratamiento de variables exógenas. Sin embargo, sus propuestas no se ejecutan directamente. El plan generado debe superar posteriormente una cadena de validaciones deterministas que comprueban, entre otros aspectos, que los modelos propuestos existen en el registro, que corresponden al tipo de problema, que no hay solapamiento entre modelos candidatos y descartados, que las estrategias de forecasting son válidas y que las evidencias citadas pertenecen realmente al contexto recuperado.

Por tanto, el Experience Pool actúa como memoria empírica ponderada, no como mecanismo automático de decisión. Las experiencias similares ayudan a contextualizar la planificación, pero no sustituyen al registro de modelos, ni a las reglas estáticas, ni a las validaciones posteriores. Esta separación mantiene el papel del LLM en una zona controlada: el agente integra evidencias y propone un plan, mientras que el sistema determinista delimita qué información recibe, valida la salida y conserva la responsabilidad última sobre qué puede ejecutarse.

En conjunto, la memoria operativa inmediata del sistema reside en el estado compartido de LangGraph, mientras que la memoria a largo plazo se materializa en el Experience Pool y se complementa con la base de conocimiento estática. Gracias a esta separación, el sistema puede conservar conocimiento empírico entre ejecuciones sin depender de conversaciones anteriores y sin perder trazabilidad sobre el origen de cada recomendación.

4.8. Flujo de ejecución end-to-end del pipeline

Una vez descritos todos los componentes relevantes de la arquitectura del sistema, resulta útil recorrer la ejecución completa del pipeline desde el punto de vista del usuario y del estado compartido. El flujo comienza en la interfaz web, donde el usuario carga uno o varios ficheros de datos junto con el esquema JSON que define el problema predictivo. Dichos datos se envían al backend mediante una petición HTTP, que crea una nueva ejecución y devuelve un identificador `run_id`. A partir de ese momento, el frontend abre un canal WebSocket asociado a dicha ejecución para recibir en tiempo real los eventos generados por el pipeline.

El backend inicializa el estado del grafo con las rutas de los ficheros recibidos, el contrato JSON, los contadores de intentos y los campos necesarios para controlar el avance del flujo. La primera decisión corresponde al controlador determinista, que inspecciona el estado y redirige la ejecución hacia el agente de validación de datos. Este agente recibe el contrato de entrada y los ficheros disponibles, y utiliza sus herramientas para cargar los datos, evaluar posibles uniones entre tablas, ejecutar el plan de integración cuando existen varios ficheros, aplicar mapeos de columnas, comprobar restricciones del esquema y revisar la calidad del dataset resultante.

Si la validación finaliza correctamente, el nodo escribe en el estado la ruta del dataset procesado, el informe de validación, el resumen del dataset, el tipo de problema y los metadatos de la tarea. A continuación, el flujo alcanza la primera puerta de supervisión humana. En este punto, el usuario puede revisar la información generada y decidir si el conjunto de datos preparado resulta adecuado para continuar. Cuando el usuario aprueba, el pipeline avanza hacia la planificación. Si rechaza la validación, puede aportar un comentario explicando el motivo; este comentario se incorpora al estado como retroalimentación y el controlador permite un nuevo intento del agente de validación, siempre dentro del límite máximo de reintentos configurado.

Tras la aprobación del dataset, el sistema construye un perfil descriptivo del problema y activa el agente planificador. Este componente utiliza el contrato de entrada, el perfil del

conjunto de datos, el registro de algoritmos disponibles, las reglas de conocimiento estático y las ejecuciones anteriores recuperadas del Experience Pool. Su objetivo no es entrenar candidatos, sino proponer un plan de entrenamiento estructurado. Dicho plan indica qué algoritmos deben evaluarse, cuáles se descartan y por qué, qué presupuesto de búsqueda se asigna y, en problemas de predicción temporal, qué estrategia de validación y tratamiento de variables exógenas debe aplicarse. Antes de pasar a la siguiente fase, el plan se valida mediante contratos Pydantic y reglas deterministas, de modo que el ejecutor solo recibe configuraciones compatibles con el sistema.

La ejecución del entrenamiento queda a cargo de un módulo determinista. Este componente lee el plan aprobado, carga el dataset procesado, prepara las particiones de entrenamiento y validación, ejecuta los modelos candidatos definidos en el registro y utiliza Optuna para explorar sus hiperparámetros. Durante este proceso, MLflow registra los experimentos, métricas, parámetros y artefactos generados. Al finalizar, el ejecutor selecciona el mejor candidato según la métrica objetivo y actualiza el estado con el identificador de ejecución, las métricas obtenidas, el modelo seleccionado y las rutas de los artefactos principales. Además, se genera un registro de experiencia que resume la ejecución y puede incorporarse al Experience Pool para futuras planificaciones.

Una vez entrenado el modelo candidato, el nodo de evaluación aplica reglas deterministas para decidir si el resultado es aceptable. Esta decisión se basa en las métricas obtenidas, los umbrales definidos para el tipo de problema y, cuando existe, la comparación con el modelo campeón registrado previamente. Posteriormente, el nodo redactor del informe utiliza un LLM de salida estructurada para transformar las métricas y resultados calculados en un informe de auditoría más legible para el usuario. Este informe no modifica la decisión métrica, sino que ayuda a interpretar el resultado y proporciona contexto para la revisión humana.

Si la evaluación no supera los criterios establecidos, el pipeline finaliza sin promocionar el modelo. En cambio, si el candidato cumple los requisitos, el flujo alcanza la segunda puerta de supervisión humana. En este punto, el usuario revisa el informe de evaluación, las métricas comparativas y la acción propuesta antes de aprobar o rechazar la promoción. Si la decisión es negativa, la ejecución termina sin modificar el modelo campeón. Si la decisión

es positiva, el nodo de despliegue lógico registra el modelo en MLflow y le asigna el alias *champion*, dejando constancia de la promoción dentro del registro de modelos.

Durante toda la ejecución, cada nodo escribe únicamente los campos del estado que le corresponden, y el controlador determinista decide el siguiente paso en función de ese estado actualizado. De forma paralela, el backend emite eventos que el frontend recibe mediante WebSocket, permitiendo al usuario seguir el avance del pipeline sin realizar consultas periódicas. El resultado final de una ejecución completa no es solo un modelo promovido, sino también un conjunto de evidencias trazables: contrato de entrada, dataset procesado, plan de entrenamiento, métricas, informes, artefactos MLflow, decisiones humanas y registro de experiencia. Esta trazabilidad permite revisar posteriormente qué ocurrió en cada fase y reutilizar conocimiento empírico en nuevas ejecuciones. En la figura 4.2 se representa una visión general del pipeline end-to-end con las fases descritas.

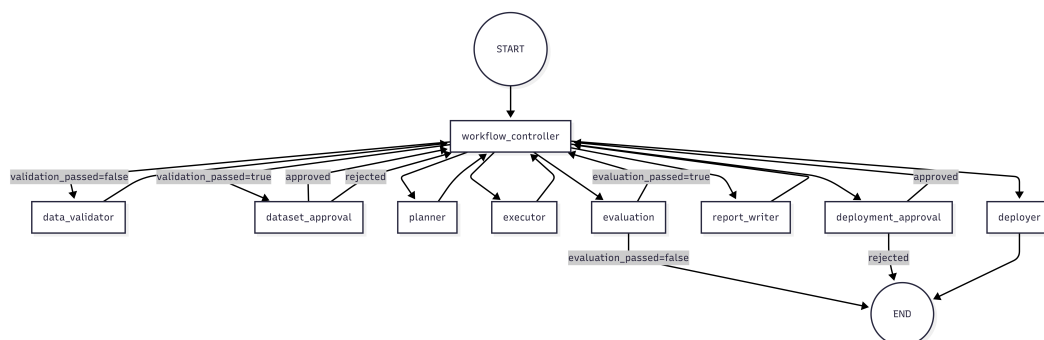


Figura 4.2: Visión general del pipeline end-to-end

5. Resultados y análisis

5.1. Contexto y dependencias de los resultados

La calidad de los resultados obtenidos en este trabajo no puede desligarse de la calidad del contexto en el que se generan. En particular, el comportamiento del pipeline asistido por agentes está condicionado por la riqueza y representatividad del Experience Pool, por la solidez del contrato de entrada que define cada problema y por la calidad de las reglas de conocimiento estático disponibles para el planificador. Un Experience Pool amplio y diverso permite recuperar evidencias relevantes para nuevos casos, mientras que un esquema de datos incompleto o impreciso limita la utilidad de las recomendaciones generadas por el sistema.

Conviene subrayar, por tanto, que la arquitectura propuesta no pretende sustituir al especialista humano, sino apoyarse en su conocimiento. La identificación de las variables relevantes, la definición del objetivo predictivo, la construcción de un esquema coherente y la revisión de las experiencias almacenadas siguen requiriendo juicio experto. El sistema automatiza partes del flujo, pero lo hace a partir de una especificación inicial proporcionada por el usuario y bajo mecanismos explícitos de validación, supervisión y trazabilidad.

Una vez establecido este marco, el análisis de resultados no se plantea como una búsqueda de modelos predictivos óptimos frente al estado del arte, sino como una evaluación del comportamiento del sistema MLOps asistido por agentes. El objetivo principal es comprobar si la arquitectura propuesta permite automatizar fases del flujo predictivo manteniendo control determinista, trazabilidad de las decisiones y capacidad de supervisión humana.

En consecuencia, los resultados se organizan en torno a tres niveles. En primer lugar, se analiza la composición y coherencia inicial del Experience Pool, ya que constituye la base empírica utilizada por el planificador. En segundo lugar, se estudian casos concretos del comportamiento agéntico, prestando atención tanto a la recuperación de experiencias

y la planificación de modelos como a la capacidad del agente de validación para integrar varios ficheros mediante el descubrimiento y evaluación de joins entre tablas. Por último, se cuantifica el coste asociado al uso de agentes, tanto en tiempo de ejecución como en intervención humana, y se analiza la robustez del controlador ante errores y reintentos.

Aunque el sistema admite tres tipos de problema —clasificación, regresión y predicción temporal—, el núcleo técnico de este trabajo se centra en forecasting. La clasificación y la regresión se incluyen como demostración secundaria de generalidad de la arquitectura, pero no constituyen el foco experimental principal del capítulo.

5.2. Diseño de la evaluación experimental

La evaluación experimental se diseña con el objetivo de analizar el comportamiento del sistema como pipeline MLOps asistido por agentes. Por ello, las pruebas no se centran únicamente en las métricas finales de predicción, sino también en la capacidad del sistema para validar datos, integrar varios ficheros, recuperar experiencias previas, generar planes de entrenamiento coherentes, gestionar errores y mantener trazabilidad durante la ejecución.

Las evidencias utilizadas proceden de ejecuciones completas del pipeline interactivo, registros del EventLog, trazas de herramientas, informes de validación, planes de join, informes de planificación, métricas de entrenamiento, tiempos por nodo y costes estimados de llamadas a modelos de lenguaje. Tal información permite analizar tanto el resultado final de cada ejecución como el proceso seguido por el sistema para llegar a dicho resultado.

Las pruebas se agrupan en cuatro familias. En primer lugar, se evalúa la composición inicial del Experience Pool, ya que constituye la base empírica utilizada por el planificador. En segundo lugar, se analizan casos de estudio del comportamiento agéntico, distinguiendo dos capacidades principales: por un lado, la recuperación de experiencias y la planificación adaptativa de modelos; por otro, el comportamiento del agente de validación en escenarios multi-tabla, donde debe descubrir claves candidatas, evaluar la calidad de posibles joins,

construir un join plan, materializar un dataset canónico compatible con el contrato de entrada y aplicar imputaciones cuando procede. En tercer lugar, se ejecutan escenarios de fallo controlado para comprobar la robustez del controlador determinista ante entradas inválidas o problemas no resolubles. Por último, se cuantifica el coste temporal y económico asociado a los nodos basados en LLM, así como la intervención humana requerida en las puertas HITL.

5.3. Composición y validación inicial del Experience Pool

Antes de ejecutar los casos de estudio interactivos, el sistema se inicializó con un conjunto de registros de experiencia obtenidos mediante el ejecutor de benchmarks, implementado en `run_benchmark.py`. Este script procesa cada dataset con la misma lógica determinista que utiliza el pipeline principal para entrenamiento, validación y registro de resultados. En el momento de escritura de esta memoria, el Experience Pool contiene 27 registros: 15 de predicción temporal, 7 de clasificación y 5 de regresión.

El objetivo de esta primera evaluación no es demostrar que los modelos seleccionados sean óptimos en sentido absoluto, sino comprobar que las experiencias almacenadas son coherentes y suficientemente informativas para ser utilizadas como evidencia por el planificador. En otras palabras, el Experience Pool se evalúa aquí como memoria empírica del sistema: debe contener casos variados, métricas trazables y decisiones de selección de modelos razonables para que su recuperación posterior aporte contexto útil en nuevas ejecuciones.

El número de ensayos de optimización de Optuna no es una decisión del planificador, sino un parámetro determinista del ejecutor. El sistema asigna un número fijo de ensayos por cada modelo de aprendizaje supervisado, idéntico para todos ellos e independiente de su número de hiperparámetros: 8 ensayos durante la siembra del pool mediante `run_benchmark.py` y 5 ensayos en las ejecuciones del pipeline en vivo. Los modelos estadísticos con un único hiperparámetro de tipo categórico —Seasonal Naive, ETS y AutoARIMA— no utilizan este presupuesto, sino que barren de forma exhaustiva y determinista una rejilla de valores de la longitud de estacionalidad (`season_length`), donde el propio barrido actúa como presu-

puesto. Por último, el modelo Naive, que carece de hiperparámetros, se entrena con un único ajuste. Mantener este presupuesto como una decisión determinista del ejecutor, fuera del componente agéntico, evita introducir variabilidad adicional entre ejecuciones y garantiza que dos ejecuciones equivalentes exploren el mismo número de configuraciones por modelo.

El objetivo de este presupuesto no es agotar la optimización de hiperparámetros, sino generar experiencias comparables y suficientemente informativas para poblar el Experience Pool. En predicción temporal se empleó validación mediante *expanding window*, mientras que en clasificación se utilizó validación cruzada estratificada de 5 particiones y en regresión validación cruzada de 5 particiones. Por tanto, los modelos campeones deben interpretarse como los mejores candidatos bajo el protocolo y presupuesto definidos, no como los mejores modelos posibles para cada dataset.

5.3.1. Experiencias de predicción temporal

La tabla 10 recoge los 15 datasets de predicción temporal almacenados en el Experience Pool. Para cada uno se indica la frecuencia temporal, el horizonte de predicción, el número de observaciones, el modelo seleccionado como campeón por el ejecutor de entrenamiento, la familia del modelo, el error cuadrático medio sobre el conjunto de validación (RMSE) y el error normalizado nRMSE, calculado en esta versión como $RMSE/\sigma_y \times 100$, es decir, el error relativo respecto a la desviación típica de la serie. Dicha métrica permite comparar de forma orientativa la magnitud del error entre series con escalas distintas y, a diferencia de la normalización por la media, se comporta de forma estable cuando la variable objetivo toma valores cercanos a cero o cambia de signo.

La tabla no pretende demostrar superioridad predictiva frente a referencias externas, sino comprobar que el Experience Pool almacena experiencias variadas y trazables. Esta distinción resulta importante: la finalidad de estos benchmarks es alimentar la memoria empírica del sistema para apoyar futuras decisiones del planificador, no establecer un ranking definitivo de algoritmos de forecasting.

El conjunto de experiencias utilizado debe interpretarse como una base inicial de demostración, no como un repositorio exhaustivo de conocimiento empírico. En el contexto de un Trabajo Fin de Grado, resulta preferible mantener un pool manejable y fácilmente auditable, donde pueda trazarse qué experiencias existen, qué propiedades tienen y cómo influyen en la recuperación posterior. Un Experience Pool mucho más amplio, con cientos de datasets y dominios adicionales, podría enriquecer las recomendaciones del planificador, pero también haría más difícil inspeccionar manualmente el origen de cada evidencia durante la validación del prototipo. Por ello, la selección inicial busca un equilibrio entre diversidad y trazabilidad: incluye series con distintas frecuencias, tamaños, horizontes y modelos campeones, sin perder la capacidad de revisar de forma explícita qué conocimiento empírico contiene el sistema.

Dataset	Frec.	H	Filas	Modelo campeón	Familia	RMSE	nRMSE (%)
Air Passengers	MS	12	144	ARIMA	Estadístico	23,56	19,6
CO ₂ Mauna Loa	MS	6	526	ETS	Estadístico	0,33	1,9
PPI Químicos	MS	12	474	ET	ML sup.	11,16	14,7
Gas Henry Hub	MS	12	330	ET	ML sup.	1,23	57,3
Manchas solares	YS	3	309	SVR	ML sup.	9,29	23,0
Río Nilo	YS	3	100	SVR	ML sup.	114,53	67,7
Bicicletas (diario)	D	14	731	ETS	Estadístico	1.385,24	71,5
Electrodomésticos	H	24	3.290	SeasonalNaive	Estadístico	68,84	84,8
Petróleo Brent	B	30	3.915	LGBM	ML sup.	4,29	17,3
Carbón (feedstock)	B	30	2.548	Naive	Estadístico	9,49	13,0
S&P 500	W	13	1.011	XGB	ML sup.	170,85	14,8
Petróleo WTI	W	13	1.013	ET	ML sup.	5,46	25,1
Oro + Macro	W	13	1.013	RF	ML sup.	65,79	15,0
Bitcoin (BTC)	W	13	335	RF	ML sup.	4.652,87	25,5
EUR/USD	W	13	335	GBM	ML sup.	0,02	30,9

Tabla 10: Resultados del benchmark de predicción temporal sobre los 15 datasets del pool. Frec.: MS = mensual, YS = anual, D = diario, H = horario, B = días hábiles, W = semanal. ETS = suavizado exponencial automático; ARIMA = AutoARIMA; SVR = máquinas de vectores de soporte para regresión; RF = *Random Forest*; XGB = XGBoost; ET = *Extra Trees*; GBM = *Gradient Boosting*; LGBM = LightGBM; SeasonalNaive = pronóstico estacional ingenuo; Naive = pronóstico ingenuo. $nRMSE = RMSE/\sigma_y \times 100$.

Los resultados muestran que el Experience Pool contiene experiencias de forecasting

con frecuencias temporales, tamaños y familias de modelo diferentes. Aparecen series mensuales, anuales, diarias, horarias, de días hábiles y semanales, con horizontes de predicción también distintos y volúmenes que van desde el centenar de observaciones hasta varios miles. Esta diversidad resulta relevante porque el planificador no recupera experiencias a partir del nombre del dataset ni de su dominio temático, sino a partir de características estructurales del perfil del problema.

También se observa que no existe una única familia de modelos dominante. En algunos casos resultan seleccionados modelos estadísticos, como ETS o AutoARIMA; en otros, modelos supervisados adaptados a forecasting, como XGBoost, Extra Trees, GBM o Bosque Aleatorio. Tal comportamiento es coherente con el objetivo del sistema: el Experience Pool no impone una regla fija de selección, sino que registra los resultados observados bajo un protocolo común. Posteriormente, el planificador puede utilizar estas experiencias como evidencia empírica recuperada mediante una métrica de similitud, pero la decisión final de entrenamiento sigue estando controlada por el ejecutor determinista y por las validaciones del sistema.

Las discrepancias entre intuición previa y modelo campeón no deben interpretarse como errores del pipeline. Por ejemplo, algunas series cortas terminan seleccionando SVR en lugar de un método estadístico clásico. Este resultado simplemente indica que, bajo el presupuesto de búsqueda y el protocolo de validación utilizados, ese candidato obtuvo mejor comportamiento empírico. El Experience Pool almacena el campeón real observado, no el modelo que se esperaría de forma teórica. Esta decisión aumenta la trazabilidad del sistema y evita transformar la memoria empírica en una lista de reglas manuales.

Los valores de nRMSE abarcan un rango amplio, desde menos del 2 % en CO₂ Mauna Loa —una serie con tendencia y estacionalidad muy regulares— hasta cerca del 85 % en Electrodomésticos, una serie horaria multivariada de consumo eléctrico con elevada componente de ruido. Este rango no debe leerse como una clasificación de calidad de los modelos, sino como un reflejo de la dificultad intrínseca de cada serie bajo el presupuesto de búsqueda utilizado. Las series con mayor error relativo —Gas Henry Hub, Bicicletas, Electrodomésticos o Río Nilo— corresponden a problemas con alta volatilidad, ruido elevado o histórico

corto frente al horizonte solicitado, mientras que las series con estructura de tendencia y estacionalidad más regulares obtienen errores relativos mucho menores. En conjunto, estas experiencias cubren comportamientos temporales diversos y proporcionan una base razonable para la recuperación posterior de casos similares.

Conviene anticipar una objeción natural: si el sistema se concibió para asistir a una empresa del sector químico, ¿por qué el pool no contiene conjuntos de datos de ese dominio concreto? La respuesta es que la composición del pool responde a un criterio estructural, no temático, como ya se argumentó en la sección 5.3. El mecanismo de recuperación compara perfiles —número de observaciones, frecuencia, presencia y disponibilidad de exógenas, estacionalidad, estacionariedad—, no etiquetas de dominio. Para que el planificador disponga de evidencia útil ante un problema de precios de materias primas químicas, lo relevante no es que el pool contenga series químicas, sino que contenga series con el mismo perfil estructural: commodities con frecuencia diaria o semanal y volatilidad elevada, índices de precios mensuales con tendencia, o problemas con variables exógenas de disponibilidad futura conocida. El pool incorpora precisamente esos perfiles —Petróleo Brent, Petróleo WTI, Gas Henry Hub, PPI Químicos, Oro con variables macro— sin necesidad de reproducir el dominio exacto del cliente. Por la misma razón, un dataset químico real de gran tamaño facilitado por la empresa no aportaría una validación más fuerte del sistema agéntico: ese caso mediría la capacidad predictiva sobre un dominio concreto, mientras que lo que aquí se evalúa es el comportamiento del sistema —recuperación, planificación, joins, latencia, robustez, coste— ante condiciones estructurales controladas. La elección de cada dataset se justifica por el perfil que aporta al pool, no por su temática.

5.3.2. Experiencias de clasificación y regresión

Aunque el foco principal del trabajo se sitúa en la predicción temporal, el Experience Pool también incluye experiencias de clasificación y regresión. Su finalidad es demostrar que la arquitectura no queda restringida a un único tipo de problema: el mismo esquema general de contrato de entrada, planificación, ejecución determinista y registro de experiencias puede

aplicarse a tareas tabulares no temporales.

La tabla 11 recoge los resultados del benchmark de clasificación. La métrica utilizada es macro-F1 sobre el conjunto de validación. Tal métrica resulta adecuada en escenarios donde puede existir desbalanceo entre clases, ya que calcula el rendimiento medio por clase sin ponderarlo únicamente por la frecuencia de cada una.

Dataset	Dominio	Filas	Modelo campeón	macro-F1
Iris	Botánica	150	RL	0,966
Vino	Enología	178	RF	0,980
Cáncer de mama	Medicina	569	LGBM	0,972
Enfermedad cardíaca	Medicina	303	RF	0,839
Titanic	Histórico	891	RF	0,977
Renta adultos	Socioeconómico	48.842	LGBM	0,750
Marketing bancario	Financiero	45.211	LGBM	0,749

Tabla 11: Resultados del benchmark de clasificación. Métrica: macro-F1 sobre el conjunto de validación. RL = regresión logística; RF = Random Forest; LGBM = LightGBM.

La tabla 12 resume los resultados del benchmark de regresión, utilizando RMSE como métrica principal. De nuevo, el objetivo no es comparar exhaustivamente algoritmos de regresión, sino generar experiencias representativas que puedan alimentar al planificador en ejecuciones posteriores.

Dataset	Dominio	Filas	Modelo campeón	RMSE	rRMSE/ σ (%)
Diabetes	Medicina	442	Ridge	55,38	71,8
Eficiencia energética	Ingeniería	768	CatBoost	0,369	3,7
Resistencia hormigón	Construcción	1.030	CatBoost	4,52	27,1
Alquiler bicicletas	Transporte	17.389	CatBoost	38,25	21,1
Vivienda California	Inmobiliario	20.640	LightGBM	0,45	39,2

Tabla 12: Resultados del benchmark de regresión. Se incluye el RMSE absoluto y el error relativo respecto a la desviación típica de la variable objetivo.

En problemas de regresión, el RMSE absoluto debe interpretarse con cautela, ya que depende directamente de la escala de la variable objetivo. Por este motivo, la tabla 12 incorpora también el error relativo respecto a la desviación típica del target, $rRMSE/\sigma = RMSE/\sigma_y$. Dicha métrica permite comparar el error del modelo con la variabilidad intrínseca del dataset: cuanto menor es este valor, mayor proporción de la variabilidad de la variable objetivo está siendo explicada por el modelo.

Esta normalización también ayuda a evitar una lectura simplista de los resultados. Un RMSE elevado no implica necesariamente que el modelo seleccionado sea inadecuado; puede indicar que el problema contiene poca señal predictiva, un tamaño muestral reducido o un nivel elevado de ruido irreducible. Bajo condiciones compatibles de cálculo, la relación entre el error relativo respecto a la desviación típica y el coeficiente de determinación puede aproximarse mediante:

$$R^2 \approx 1 - \left(\frac{RMSE}{\sigma_y} \right)^2.$$

El caso Diabetes ilustra esta idea. Aunque presenta el mayor error relativo de la tabla, con un $RMSE/\sigma_y$ del 71,8%, este resultado no debe interpretarse automáticamente como un fallo del pipeline. Se trata de un dataset pequeño, con 442 observaciones, y conocido por presentar una capacidad predictiva limitada a partir de las variables disponibles. En estas condiciones, incluso modelos más complejos pueden sobreajustar o no mejorar de forma clara a una regresión regularizada. Por ello, la selección de Ridge resulta razonable: el sistema escoge un modelo sencillo y regularizado, coherente con un problema de baja muestra y señal limitada.

La lectura relevante, por tanto, no es que Diabetes obtenga un rendimiento predictivo elevado, sino que el sistema evita seleccionar un modelo innecesariamente complejo en un escenario donde la señal disponible no lo justifica. Este punto es importante para la interpretación del Experience Pool: una experiencia útil no es únicamente aquella que produce una métrica muy alta, sino aquella que registra una decisión de modelado coherente con las características del dataset.

En el extremo contrario, Eficiencia energética obtiene un error relativo muy bajo respecto a la desviación típica del target. Dicho comportamiento indica que el dataset contiene una señal predictiva muy fuerte y que el modelo seleccionado captura casi toda la variabilidad relevante bajo el protocolo de validación utilizado. Este resultado debe entenderse como un caso favorable dentro del pool, no como una garantía de que el sistema vaya a alcanzar siempre rendimientos similares.

Los datasets Alquiler de bicicletas, Resistencia del hormigón y Vivienda California se sitúan en una zona intermedia. En ellos, modelos más flexibles como CatBoost o LightGBM resultan seleccionados en problemas con mayor volumen de datos o relaciones potencialmente no lineales, obteniendo errores relativos claramente inferiores al de Diabetes. Esta selección es coherente con la idea de que, cuando existe más muestra o una estructura más rica en los datos, modelos de mayor capacidad pueden aportar valor.

En conjunto, los resultados de clasificación y regresión reflejan patrones razonables para el objetivo secundario que cumplen dentro del trabajo. En clasificación, los conjuntos pequeños o medianos tienden a seleccionar modelos de baja o media complejidad, mientras que los conjuntos de mayor tamaño favorecen modelos basados en boosting como LightGBM. En regresión, Ridge aparece como campeón en el caso más pequeño y ruidoso, mientras que CatBoost y LightGBM resultan seleccionados en datasets con mayor señal o relaciones potencialmente no lineales. Estos resultados no pretenden establecer una comparación definitiva entre algoritmos, sino demostrar que el pipeline produce experiencias trazables y coherentes también fuera del dominio principal de forecasting.

5.4. Comportamiento agéntico en ejecuciones de forecasting

Una vez validada la composición inicial del Experience Pool, el análisis se centra en ejecuciones completas del pipeline interactivo sobre problemas de forecasting. Esta parte constituye el núcleo experimental del capítulo, ya que permite observar el comportamiento real de los componentes agénticos en las fases donde fueron introducidos: recuperación de

experiencias, planificación de modelos, validación de datos, descubrimiento de joins, gestión de errores y generación de evidencias para la supervisión humana.

Los datasets utilizados en estas pruebas —tanto los casos de joins como los de escalabilidad y robustez— son sintéticos y no pertenecen al dominio químico que motiva el trabajo. Tal elección es deliberada y responde directamente a lo que se evalúa: capacidades del sistema agéntico, no calidad predictiva sobre un dominio. Cada caso de estudio aísla una capacidad concreta —el descubrimiento de joins entre tablas, la imputación ante cobertura parcial, la evolución del coste con el tamaño del dataset o el comportamiento ante entradas inválidas—, y ninguna de ellas depende del contenido temático de los datos: un join entre dos tablas mediante una clave temporal se evalúa de forma idéntica con independencia de qué representen sus columnas, ya sean precios de una materia prima o ventas de una panadería. Cambiar la temática de estos datasets a un contexto químico no alteraría en absoluto las conclusiones de las pruebas. Más aún, el uso de datos sintéticos es la opción metodológicamente correcta para estos experimentos, porque permite controlar la variable que se desea aislar. Para medir cómo escala el coste con el tamaño, por ejemplo, se requieren conjuntos idénticos en todo salvo en el número de observaciones, condición que solo puede garantizarse mediante generación sintética; un conjunto de datasets reales del dominio introduciría variaciones simultáneas en número de exógenas, ruido y estructura temporal que impedirían atribuir las diferencias observadas a la dimensión bajo estudio. En este sentido, los datos sintéticos cumplen aquí la misma función que una muestra controlada en un experimento de laboratorio.

A diferencia de los benchmarks iniciales, estas ejecuciones atraviesan el flujo completo del sistema: carga de ficheros, validación mediante el agente `data_validator`, aprobación humana del dataset, planificación con recuperación de experiencias, entrenamiento determinista de candidatos, evaluación, generación de informe de auditoría y, cuando procede, aprobación final del modelo. Por tanto, no solo se analiza el modelo campeón obtenido, sino también el proceso seguido por el sistema para llegar a esa decisión.

La tabla 13 resume las principales ejecuciones de forecasting utilizadas como casos de estudio. Las métricas predictivas se incluyen únicamente como referencia cuando resultan relevantes; el objetivo principal de esta sección no es comparar modelos, sino analizar el



comportamiento del sistema durante la ejecución completa del pipeline. En particular, se evalúa si el sistema recupera experiencias coherentes, genera planes razonables, valida correctamente los datos, controla el coste agéntico y falla de forma segura cuando las entradas no permiten completar el flujo.

Caso	Filas	Frecuencia	Aspecto evaluado	Resultado principal
Air Passengers	144	Mensual	Recuperación exacta del pool	Recupera la misma experiencia almacenada y valida la métrica de similitud
Grid Demand	156	Semanal	Joins multi-tabla, cobertura total	Integra varios ficheros crudos mediante claves temporales compatibles
Store Sales	156	Semanal	Joins con cobertura parcial e imputación	Acepta joins no perfectos, mantiene el dataset base e imputa valores faltantes generados por la unión
Small dataset (Revenue)	120	Mensual	Coste agéntico en dataset pequeño	Ejecución completa con perfil compacto del dataset
Medium dataset (Bakery)	1.000	Diario	Variabilidad LLM y coste por tamaño	Tokens comparables al caso pequeño, con mayor coste determinista de entrenamiento
Large dataset (Factory)	10.000	Horario	Tratabilidad por tamaño	Ejecución completa; el coste agéntico no escala con el tamaño
Casos <i>break</i>	Variable	Mensual	Robustez ante errores	El sistema detiene el flujo sin entrenar sobre datos inválidos

Tabla 13: Resumen de ejecuciones de forecasting utilizadas como casos de estudio.

El caso Air Passengers se utiliza como prueba de control de la recuperación del Experience Pool. Dado que esa misma serie ya estaba presente en el almacén de experiencias, el resultado esperado era que la función de similitud identificase la experiencia equivalente. La ejecución confirma este comportamiento: el planificador recupera la experiencia almacenada correspondiente al mismo dataset con una similitud de 1.0 y la utiliza como evidencia directa en la generación del plan. Por tanto, este caso no se interpreta como una recuperación aproximada, sino como una validación de que el mecanismo de similitud reconoce correctamente

una coincidencia exacta.

El caso Grid Demand evalúa una situación más compleja desde el punto de vista de validación de datos. El sistema recibe varios ficheros crudos asociados a una misma serie temporal y el agente `data_validator` debe identificar claves temporales compatibles, proponer joins y construir un conjunto de datos canónico antes de continuar. En esta prueba, los joins presentan una cobertura elevada y no generan explosión de filas, por lo que el agente acepta el plan de integración y lo presenta al usuario en la puerta de aprobación del dataset.

El caso Store Sales introduce una situación más realista: los joins no son perfectos. A diferencia de los casos anteriores, no todos los valores de la clave de una tabla aparecen necesariamente en la otra, por lo que la unión puede generar valores faltantes en algunas variables auxiliares. La prueba permite comprobar que el sistema no exige una cobertura del 100 % para aceptar un join, pero tampoco acepta cualquier coincidencia parcial. El agente busca una relación estructural compatible entre una posible clave primaria del dataset base y una posible clave foránea de la tabla auxiliar, evalúa la cardinalidad de la relación, la cobertura obtenida, el riesgo de duplicación de filas y la coherencia semántica de las columnas implicadas.

Tras ejecutar el plan de integración, el sistema conserva el dataset base mediante una unión de tipo left join, identifica los valores ausentes introducidos por la cobertura parcial y aplica las herramientas de imputación correspondientes cuando procede. Tal comportamiento resulta relevante porque aproxima el pipeline a escenarios reales, donde las fuentes de datos auxiliares rara vez encajan de forma perfecta.

Los casos small, medium y large se utilizan para analizar la escalabilidad del coste del pipeline. En los datasets small y medium, el número de filas aumenta de forma significativa, pero el coste agéntico se mantiene en el mismo orden de magnitud, ya que los agentes no reciben el conjunto de datos completo, sino un perfil compacto generado por herramientas deterministas. Esta observación refuerza una decisión clave de la arquitectura: el coste de las llamadas LLM depende del contexto textual que se proporciona al agente, no del número bruto de observaciones del dataset.

En cambio, el caso large, con 10.000 observaciones de frecuencia horaria, muestra que el crecimiento del coste se concentra en la fase determinista de entrenamiento. En una primera versión del sistema, la búsqueda de estacionalidad sobre la serie horaria exploraba periodos candidatos de forma exhaustiva, lo que disparaba el tiempo de entrenamiento y validación hasta agotar el límite end-to-end de 600 segundos establecido para esta demostración. Tras incorporar una búsqueda de estacionalidad acotada por la frecuencia del dataset —que restringe los periodos candidatos a los compatibles con la granularidad temporal declarada en el contrato—, la ejecución completa el flujo dentro del presupuesto temporal, con un coste de entrenamiento determinista en torno a 84 segundos para este caso. El límite de 600 segundos se mantiene como salvaguarda práctica frente a ejecuciones excesivamente largas, pero ya no se alcanza en este escenario.

La lectura relevante de esta prueba no cambia: el aumento de tamaño del dataset no dispara el coste agéntico, sino el coste computacional propio del entrenamiento y la validación de modelos. La mejora introducida —acotar la búsqueda de estacionalidad— es precisamente una técnica de ingeniería MLOps sobre el ejecutor determinista, no una intervención agéntica adicional, lo que confirma dónde reside el cuello de botella de escalabilidad. Otras técnicas en la misma línea —límites de tiempo por modelo, reducción del espacio de búsqueda, paralelización, early stopping o mayor capacidad de cómputo— permitirían escalar a datasets aún mayores manteniendo el coste agéntico estable.

Finalmente, los casos break evalúan la robustez del sistema ante entradas inválidas. Estos experimentos fuerzan errores como ausencia de la columna objetivo, target completamente nulo o histórico insuficiente para el horizonte solicitado. En todos los casos, el pipeline alcanza un estado terminal controlado y evita entrenar modelos sobre datos inválidos. Esta prueba refuerza la función del controlador determinista como mecanismo de seguridad del flujo.

5.4.1. Recuperación de experiencias y razonamiento del planificador

Esta prueba analiza si el nodo planner utiliza de forma efectiva el Experience Pool durante la generación del plan de entrenamiento. A diferencia de la tabla resumen anterior, el objetivo aquí no es volver a describir todas las ejecuciones completas del pipeline, sino estudiar específicamente cómo se comporta el mecanismo de recuperación de experiencias ante dos situaciones: una coincidencia exacta con una experiencia ya almacenada y un caso nuevo con características estructurales similares a experiencias previas.

El caso Air Passengers se utiliza como prueba de control. La misma serie temporal ya estaba almacenada previamente en el Experience Pool, por lo que el comportamiento esperado era que el sistema identificase esa experiencia como coincidencia exacta. La ejecución confirma este resultado: la experiencia correspondiente a Air Passengers se recupera con similitud igual a 1 y aparece con la máxima prioridad dentro de las experiencias candidatas. Por tanto, este caso no se interpreta como una recuperación aproximada, sino como una validación de que la métrica de similitud reconoce correctamente un dataset ya presente en la memoria empírica del sistema.

El caso Grid Demand representa una situación diferente. En este caso, el dataset no coincide exactamente con una experiencia previa, pero comparte rasgos estructurales con otros problemas temporales almacenados en el pool: frecuencia definida, histórico limitado, estructura temporal clara y necesidad de aplicar una validación específica para forecasting. En esta ejecución, el planificador combina la información recuperada con el conocimiento estático del sistema y genera un plan prudente, priorizando modelos estadísticos y baselines temporales frente a modelos supervisados más complejos.

El aspecto más relevante de esta prueba es la trazabilidad de la decisión. El planificador no propone candidatos de forma aislada, sino que combina tres fuentes de información: el contrato de entrada, el perfil calculado del dataset y las experiencias recuperadas del Experience Pool. En Air Passengers se comprueba que el sistema reconoce una experiencia idéntica ya almacenada; en Grid Demand se observa que también puede utilizar experiencias



próximas como contexto cuando no existe una coincidencia exacta.

En consecuencia, el Experience Pool no actúa como una lista rígida de modelos recomendados, sino como una memoria empírica que contextualiza la planificación. La decisión final sigue quedando en manos del ejecutor determinista, que entrena y valida los candidatos propuestos bajo el protocolo definido.

5.4.2. Validación de datos y descubrimiento de joins

El descubrimiento de joins entre tablas constituye una de las capacidades más representativas del agente `data_validator`. Esta tarea combina señales estructurales —cobertura de claves, cardinalidad o riesgo de duplicación— con interpretación semántica sobre el papel de cada fichero dentro del problema. Por ello resulta especialmente adecuada para evaluar el valor añadido del enfoque agéntico frente a una validación puramente basada en reglas fijas.

En los casos ejecutados, el sistema recibe varios ficheros de entrada y debe decidir si existe una relación útil entre ellos. El agente analiza los perfiles de las tablas, propone claves candidatas, invoca herramientas deterministas para medir la calidad de los joins y construye un dataset canónico antes de continuar con la validación del contrato de entrada. Esta estrategia evita que el LLM opere directamente sobre todos los datos: el modelo razona sobre resúmenes, perfiles y métricas de calidad generadas por código.

Caso	Ficheros	Clave propuesta	Decisión	Filas base	Filas finales
Grid Demand	Serie + 3 tablas auxiliares	Fecha semanal (nombres distintos)	3 joins left aceptados (cobertura 100 %)	156	156
Store Sales	Serie semanal + 2 tablas de contexto	Fecha semanal (week / week_ref / week_key)	2 joins left aceptados (cobertura 72,4 %, huérfanas imputadas)	156	156

Tabla 14: Resultados del agente de validación en escenarios con varios ficheros.

Las dos filas de la tabla 14 representan situaciones distintas y, por tanto, permiten evaluar comportamientos complementarios del agente. Grid Demand constituye el caso más sencillo: la relación entre tablas se basa en claves temporales compatibles y la cobertura de la unión es completa. En este escenario, el agente debe identificar que las tablas auxiliares están alineadas con la serie principal, comprobar que el join no genera duplicados ni pérdida de filas y construir un dataset final coherente con el contrato de entrada.

El agente de validación, sin intervención humana, infirió los tres joins de tipo LEFT uno-a-uno, los evaluó (cobertura del 100 %, multiplicador de filas 1,00×, riesgo bajo) y los ejecutó, materializando un dataset de 156 filas × 8 columnas con cinco variables exógenas añadidas. El plan de join se presenta al revisor humano en la puerta de aprobación con su nivel de confianza (“alta”) y la justificación de cada candidato.

El caso Store Sales resulta más exigente por una razón distinta: aunque la clave de unión vuelve a ser temporal, una de las tablas de contexto —la meteorológica— no cubre todo el histórico. El agente debe alinear de forma semántica claves con nombres distintos (*week*, *week_ref*, *week_key*), que no coinciden literalmente, y —sobre todo— decidir qué hacer con las semanas que quedan sin contexto tras la unión.

La cobertura, en efecto, no es completa: la tabla meteorológica solo cubre el 72,4 % de las 156 semanas (113 de 156), dejando 43 semanas *huérfanas*; el calendario, en cambio, las cubre todas y aporta 12 semanas futuras adicionales. Esta situación permite comprobar que el sistema no exige una cobertura del 100 % para aceptar un join, pero tampoco acepta cualquier coincidencia parcial. El agente evalúa si existe una relación tipo clave primaria–clave foránea, si la cardinalidad es compatible, si el join mantiene controlado el número de filas y si los valores faltantes que introduce pueden imputarse de forma razonable; el umbral de aviso (cobertura < 80 %) dispara la alerta solo sobre el join meteorológico, no sobre el del calendario.

La decisión adoptada en Store Sales es especialmente relevante: el sistema conserva las 156 semanas del dataset base mediante un *left join*, en lugar de un *inner join* que habría eliminado las 43 semanas sin correspondencia meteorológica. Tras la unión, los valores ausentes

de las dos columnas meteorológicas se identifican y se tratan mediante las herramientas de imputación correspondientes. Como particularidad del dominio temporal, las exógenas se separan según su disponibilidad futura: el calendario (`is_holiday_week`, `week_of_year`) es de futuro conocido y puede alimentar el horizonte de pronóstico, mientras que la meteorología, de futuro desconocido, requiere una estrategia de extensión determinista. El campeón seleccionado fue AutoARIMA y, al superar la puerta de evaluación determinista, el recorrido se completó hasta el despliegue.

La aportación del agente no consiste en ejecutar físicamente los joins, ya que esa operación se realiza mediante código determinista. Su papel consiste en decidir qué tablas tiene sentido combinar, qué claves candidatas deben evaluarse y cómo interpretar las métricas devueltas por las herramientas. Esta separación mantiene el control programático sobre las operaciones críticas y permite que el usuario revise el plan de integración en la puerta de aprobación antes de continuar con el entrenamiento.

5.4.3. Escalabilidad del coste agéntico frente al tamaño del dataset

Además de los casos con varios ficheros, se evaluó el comportamiento del pipeline ante datasets de distinto tamaño. El objetivo de esta prueba no es comparar la precisión de los modelos entre conjuntos de datos diferentes, sino observar cómo evoluciona el coste del sistema cuando aumenta el número de observaciones y qué fases concentran el mayor tiempo de ejecución.

Para ello se ejecutaron tres escenarios de tamaño creciente: un dataset pequeño, con aproximadamente 120 observaciones; un dataset medio, con alrededor de 1.000 observaciones; y un dataset grande, con 10.000 observaciones de frecuencia horaria. Los tres casos completaron la ejecución dentro del límite temporal configurado. En una primera versión del sistema, el caso grande agotaba el timeout end-to-end de 600 segundos debido a una búsqueda de estacionalidad exhaustiva sobre la serie horaria; tras acotar dicha búsqueda a los periodos compatibles con la frecuencia del dataset, el caso grande completa el flujo concentrando su



coste, como era de esperar, en la fase determinista de entrenamiento y validación temporal de varios modelos candidatos. En la tabla 15 se recoge el comportamiento del pipeline sobre datasets de distinto tamaño.

Caso	Filas	Resultado	Fase dominante	Observación principal
Small	120	Completado	LLM / entrenamiento corto	Perfil compacto y ejecución completa
Medium	1.000	Completado	Planner / executor	Mayor coste por razonamiento y entrenamiento
Large	10.000	Completado	Executor determinista	~116 s de entrenamiento tras acotar la búsqueda de estacionalidad por frecuencia

Tabla 15: Comportamiento del pipeline en datasets de distinto tamaño.

El resultado más relevante de esta prueba no es el tiempo concreto del caso grande, sino la separación entre coste agéntico y coste computacional determinista. El incremento del número de filas no implica que los agentes reciban más datos crudos ni que el consumo de tokens crezca proporcionalmente al tamaño del dataset. La arquitectura evita pasar el conjunto de datos completo al LLM; en su lugar, las herramientas deterministas generan una representación compacta del problema: número de filas y columnas, nombres de variables, tipos de datos, estadísticas resumidas, presencia de valores ausentes, estructura temporal, disponibilidad de variables exógenas y resultados de validaciones o evaluaciones de joins.

Esta decisión limita el crecimiento del coste agéntico. En las ejecuciones small y medium, el número de observaciones aumenta aproximadamente de 120 a 1.000, pero el número de tokens de los agentes se mantiene en el mismo orden de magnitud (el agente de validación procesó 36.557 tokens de entrada con 120 filas y 32.360 con 1.000). Las variaciones observadas dependen más del número de herramientas invocadas, del histórico de mensajes, de los reintentos y del razonamiento requerido que del número bruto de filas del dataset.

En cambio, el caso de 10.000 observaciones sí incrementa de forma notable el coste de la fase determinista de entrenamiento. Dicho comportamiento es esperable en cualquier pipeline MLOps: entrenar varios modelos candidatos, aplicar búsqueda de hiperparámetros

y validar mediante ventanas temporales sobre un conjunto mayor exige más cómputo. La acotación de la búsqueda de estacionalidad por frecuencia permitió que este caso completase el flujo dentro del presupuesto temporal, pero el cuello de botella sigue residiendo en el ejecutor determinista, no en el uso de agentes.

Este experimento refuerza una decisión central de la arquitectura: los agentes no deben procesar directamente datos tabulares completos ni sustituir operaciones programáticas claras. Su papel consiste en interpretar resúmenes estructurados, coordinar herramientas y tomar decisiones de alto nivel. Cuando el problema es puramente computacional, como entrenar varios modelos sobre un dataset grande, la mejora debe abordarse mediante técnicas deterministas de ingeniería MLOps, como paralelización, reducción del espacio de búsqueda, límites de tiempo por modelo, *early stopping* o infraestructura de cómputo más potente.

5.4.4. Variabilidad del coste agéntico y control del contexto

Una característica relevante de los componentes basados en LLM es que su coste no resulta completamente determinista. Aunque el sistema fija contratos de salida, valida las respuestas mediante estructuras tipadas y limita la autonomía de los agentes mediante herramientas concretas, las llamadas al modelo pueden producir variaciones entre ejecuciones. Estas variaciones pueden aparecer en el número de herramientas invocadas, en la longitud de la respuesta generada, en la necesidad de realizar un reintento tras una salida inválida o en el número de tokens internos de razonamiento consumidos por el modelo.

Esta variabilidad no desaparece por completo aunque se intente reducir la aleatoriedad de la generación. En la práctica, los modelos de lenguaje pueden seguir mostrando diferencias entre ejecuciones equivalentes, especialmente cuando intervienen razonamiento, llamadas a herramientas y validaciones estructuradas posteriores. Por ello, en este trabajo no se asume que el coste de los nodos LLM sea estrictamente reproducible ejecución a ejecución. Lo que sí se busca es acotar su impacto mediante una arquitectura que limite el contexto enviado al modelo y mantenga las decisiones críticas bajo validación determinista. En la tabla 16 se

muestra un ejemplo de esta variabilidad del coste agéntico en repeticiones del caso Bakery.

Ejecución	Validador (s)	Planner (s)	Tokens salida planner	Reintento
1	25,8	64,4	10,5k	No
2	28,3	69,1	9,5k	No
3	21,4	51,7	7,5k	No
4	16,5	167,9	20,9k	Sí

Tabla 16: Ejemplo de variabilidad del coste agéntico en repeticiones del caso Bakery. Un reintento reejecuta el planificador (dos intentos en total): duplica aproximadamente sus tokens de salida y multiplica su tiempo por un factor de 2 a 3, según la varianza entre ambos intentos.

El punto clave observado en los experimentos es que esta variabilidad no implica que el coste agéntico escale proporcionalmente con el número de filas del dataset. La arquitectura evita pasar el dataset completo al LLM. En su lugar, las herramientas deterministas generan una representación compacta del problema: número de filas y columnas, nombres de variables, tipos de datos, estadísticas resumidas, presencia de valores ausentes, algunas filas de ejemplo, estructura temporal, disponibilidad de variables exógenas y resultados de validaciones o evaluaciones de joins. El agente razona sobre esta representación, no sobre la tabla completa.

Por tanto, al comparar datasets de distinto tamaño, el consumo de tokens puede variar entre ejecuciones, pero dicha variación depende principalmente del contexto textual construido, del número de herramientas utilizadas, de las experiencias recuperadas, de la complejidad del razonamiento y de posibles reintentos. No depende linealmente del número bruto de observaciones. Esta separación constituye una decisión arquitectónica importante: el LLM trabaja sobre resúmenes semánticamente ricos y acotados, mientras que las operaciones masivas sobre datos permanecen bajo control programático.

5.5. Robustez ante errores controlados

La robustez del sistema se evalúa mediante escenarios de fallo diseñados para comprobar que el pipeline no avanza cuando las condiciones de entrada no permiten una ejecución válida. Estos casos validan una de las decisiones centrales de la arquitectura: los agentes pueden interpretar la entrada, pero tanto la detección de una condición inválida como la decisión de parar son siempre deterministas. El agente de validación (un LLM) orquesta qué comprobaciones ejecutar y redacta un informe, pero su prosa nunca es vinculante: el veredicto pase/parada lo emiten herramientas y nodos deterministas a partir de su salida estructurada. Los tres escenarios ilustran además una defensa en profundidad, ya que cada fallo es atrapado por un componente determinista distinto y en una etapa distinta del flujo.

Se inyectaron tres fallos: una columna objetivo declarada pero ausente en el dataset, una columna objetivo completamente formada por valores nulos y una serie temporal demasiado corta para el horizonte de predicción solicitado. La tabla 17 resume dónde y cómo se detecta cada uno.

Fallo inyectado	Detectado en	Mecanismo y comportamiento observado
Columna objetivo declarada pero ausente	Herramienta <code>detect_temporal_gaps</code> (dentro del <code>data_validator</code>), antes de la validación de esquema	La herramienta exige que la columna objetivo exista y lanza una excepción al no encontrarla (<i>“target_column X not found in dataset”</i>). La ejecución se aborta antes de alcanzar <code>validate_against_schema</code> . Es el corte más temprano y abrupto.
Columna objetivo 100 % nula	Herramienta <code>validate_against_schema</code> (regla <code>nullable</code>) dentro del <code>data_validator</code>	La regla <code>nullable</code> detecta los 120 valores nulos. El intento de imputación de datos faltantes (<code>impute_missing_values</code>) no puede recuperar un objetivo enteramente vacío —el hueco abarca la serie completa—, la revalidación vuelve a fallar y el nodo fija <code>validation_passed=False</code> de forma determinista.
15 observaciones para horizonte 12	Función <code>resolve_validation_strategy</code> (chequeo de capacidad) en el nodo <code>planner</code>	La validación de datos pasa (esquema y calidad correctos) y el dataset supera incluso la puerta de aprobación humana. El agente señala la insuficiencia en su informe, pero ese texto no es vinculante. El bloqueo lo produce el chequeo de capacidad determinista del planificador (<i>“need ≥ 60 observations for horizon 12, have 15”</i>), que detiene el pipeline antes del entrenamiento.

Tabla 17: Escenarios de error forzado y comportamiento observado. Los tres alcanzan un estado terminal controlado mediante un componente determinista; ninguno entrena sobre datos inválidos ni promociona un modelo.

Los tres escenarios alcanzan un estado terminal controlado y son detectados en etapas distintas, lo que evidencia la defensa en profundidad del diseño. El caso de la serie demasiado corta es el más revelador: ni el informe del agente ni la aprobación humana llegan a frenar la ejecución, y aun así el pipeline se detiene, porque la garantía de seguridad no descansa sobre el juicio del LLM ni sobre el operador humano, sino sobre el chequeo determinista que conoce la relación entre horizonte e histórico disponible. En los tres casos el controlador enruta a END con `error_message` establecido y emite un evento `run_complete` con un mensaje informativo en lugar de un falso éxito; ninguno deja artefactos corruptos en el pool. La forma del corte difiere —excepción de una herramienta, informe de validación fallido o error de

capacidad—, pero la propiedad común es la misma: el sistema falla de manera visible y la decisión de parar queda siempre en manos deterministas.

5.6. Desglose de coste por nodo

La incorporación de agentes introduce un coste adicional respecto a una ejecución puramente determinista. Este coste se manifiesta principalmente en forma de latencia por llamadas al modelo de lenguaje, consumo de tokens y necesidad de revisar las decisiones en puertas de supervisión humana. Por ello, la evaluación del sistema debe analizar no solo la calidad de las decisiones, sino también el sobre coste que introduce la inteligencia agéntica.

Los cinco casos seleccionados permiten conectar este análisis de coste con las pruebas anteriores de tamaño y de descubrimiento de joins. Revenue, Bakery y Large cubren el eje de tamaño, con 120, 1.000 y 10.000 filas respectivamente; Grid y Store Sales cubren el eje de integración de datos, correspondiendo el primero a un join de cobertura total (cuatro ficheros) y el segundo a un join de cobertura parcial (tres ficheros, con semanas huérfanas imputadas).

La tabla 18 desglosa, para estos cinco casos de pronóstico, el tiempo de cómputo por nodo (excluyendo las pausas de revisión humana en las puertas HITL) y el coste monetario estimado. El tiempo se mide a partir de las marcas temporales del EventLog; el coste, a partir del recuento de tokens y la tabla de precios local (gpt-5.4-mini: \$0,75/\$4,50 por millón de tokens de entrada/salida; gpt-5.4-nano: \$0,20/\$1,25). Cada columna promedia entre 5 y 10 ejecuciones (fila N) y excluye las repeticiones con reintento del planificador —caracterizadas más abajo—, de modo que el tiempo del planner corresponde a una única llamada al modelo. Para los dos nodos de mayor varianza, planner y executor, se indica además la desviación típica entre ejecuciones. En la tabla, los nodos executor, evaluation y deployer son deterministas, mientras que el resto son agénticos (LLM). Los tiempos son intrínsecamente variables por dos causas distintas: la latencia de la API en los nodos LLM —sobre todo el planner— y, en el executor, el conjunto de candidatos que el planner propone en cada ejecución. Conviene precisar el origen de esta última variabilidad: la recu-

peración de experiencias es determinista —la métrica de similitud descrita en la Sección 4.7 devuelve siempre el mismo orden ante la misma entrada, con desempate por recencia—, de modo que la variación entre ejecuciones equivalentes proviene del LLM del planificador y se concentra en la cola de candidatos cuando la evidencia recuperada es débil o ambigua; la selección del campeón entre los candidatos ya entrenados permanece, en cambio, determinista. Como un plan con modelos pesados (p. ej. *xgboost* o árboles) multiplica el tiempo de entrenamiento, esa variación en la cola repercute directamente en el tiempo del executor.

Nodo	Tipo	Revenue	Bakery	Large	Grid	Store Sales
data_validator	LLM	21,5	21,1	22,7	50,6	44,9
planner	LLM	55,3±19	71,2±8	72,8±17	59,3±15	61,3±26
executor	Determ.	8,4±1	56,5±34	116,0±42	10,0±1	7,7±1
evaluation	Determ.	0,1	0,1	0,1	0,1	0,0
report_writer	LLM	5,9	5,5	6,7	6,2	6,4
deployer	Determ.	0,3	0,3	—	0,4	0,3
Total cómputo		91,5	154,7	218,3	126,6	120,6
Cuota LLM		90 %	63 %	47 %	92 %	93 %
Coste (USD)		0,087	0,091	0,094	0,173	0,157
<i>N (ejecuciones)</i>		9	9	9	10	5

Tabla 18: Desglose de tiempo de cómputo y coste por ejecución.

A partir de la tabla 18 se extraen varias observaciones. En primer lugar, el coste monetario por ejecución completa se mantiene en valores reducidos, entre 0,09 y 0,17 dólares. Este coste se concentra en los nodos `data_validator` y `planner`, que realizan las llamadas más extensas al modelo. El descubrimiento de joins reubica además una parte notable del coste en el `data_validator`: su coste pasa de unos 0,03 dólares en los casos de fichero único (Revenue, Bakery, Large) a entre 0,10 y 0,12 dólares cuando hay que emparejar varias tablas (Grid, Store Sales), porque cada ronda de análisis y evaluación de claves acumula contexto. El coste del planificador, en cambio, se mantiene estable, lo que confirma que el sobre coste de varias tablas es, esencialmente, un coste de validación de datos.

En segundo lugar, la fracción del tiempo de cómputo atribuible a los nodos LLM varía entre el 47 % y el 93 % según el peso del entrenamiento. Es baja cuando el executor domina —47 % en Large y 63 % en Bakery, donde entrenar varios candidatos sobre más datos concentra el tiempo— y alta cuando el entrenamiento es ligero —entre el 90 % y el 93 % en Revenue, Grid y Store Sales, de histórico corto—. Esta diferencia refleja la separación entre dos tipos de coste: por un lado, la latencia agéntica de razonamiento, llamadas a herramientas y generación estructurada; por otro, el coste computacional determinista del entrenamiento y la validación de modelos.

En tercer lugar, el tiempo del executor escala con el volumen de datos: alcanza unos 116 segundos en Large (10.000 filas horarias) frente a los 8–10 segundos de Revenue, Grid y Store Sales, y unos 56 segundos en Bakery. Este comportamiento es coherente con lo observado en la Sección 5.4.3: entrenar y validar varios modelos candidatos sobre un conjunto mayor exige más cómputo, y ese coste reside en el executor determinista, no en la capa de agentes.

Ahora bien, el executor revela un matiz contraintuitivo que su propia desviación típica ya anticipa (± 34 y ± 42 segundos en Bakery y Large). Aunque es un nodo determinista, su tiempo no depende solo del volumen de datos, sino del conjunto de candidatos que el planificador decide entrenar, y ese conjunto varía entre ejecuciones por la propia no-determinación del modelo de lenguaje. Sobre la panadería, los planes que incluyeron un modelo pesado (*xgboost* o un ensemble de árboles) tardaron alrededor de 150 segundos en entrenar, frente a unos 40 segundos de los planes que solo propusieron candidatos estadísticos, sobre los mismos datos y sin mediar reintento alguno. El matiz decisivo es que ese modelo pesado no resultó campeón en ninguna ejecución —el ganador fue siempre *ets* o *auto_arima*—, de modo que el sobrecoste corresponde a cómputo invertido en entrenar candidatos que la validación cruzada acaba descartando. Así, el nodo determinista por excelencia exhibe, de forma indirecta, un coste no determinista: la aleatoriedad residual del LLM no altera la decisión final, pero sí se filtra al coste a través de qué candidatos se proponen. Es una ineficiencia acotada y abordable —limitar el número de candidatos que el planificador puede seleccionar, o cachear el plan para un perfil de dataset ya visto, eliminaría esta varianza sin afectar a unas decisiones que ya son estables— y se recoge como línea de trabajo futuro.

Este sobrecoste agéntico no debe interpretarse únicamente como una penalización, sino como el precio operativo de las capacidades adicionales introducidas por la arquitectura. Un pipeline determinista tradicional solo asumiría la fracción correspondiente a executor, evaluation y deployer —este último apenas 0,3 segundos, pues su trabajo se reduce a registrar y promover el modelo ganador—, pero tampoco dispondría de selección adaptativa de modelos, recuperación de experiencias, descubrimiento asistido de joins entre tablas, validación semántica de contratos de entrada ni informes de auditoría en lenguaje natural.

También se observa que los reintentos del planificador constituyen otro factor de variabilidad temporal y económica. Cuando la salida estructurada de un nodo LLM no supera la validación determinista de su esquema, el sistema no la acepta, sino que reejecuta el nodo dentro del presupuesto de intentos configurado (dos como máximo). Este comportamiento aparece de forma ocasional: en las ejecuciones sin reintento el planificador se situó en torno a 70 segundos y 0,06 dólares, mientras que en las que reintentaron —dos intentos en total— alcanzó alrededor de 160 segundos y 0,11 dólares, aproximadamente el doble. Por eso las repeticiones con reintento se caracterizan por separado y se excluyen del promedio de la tabla 18. Aunque el sistema proporcione el mismo dataset, el mismo contrato de entrada y las mismas herramientas, una llamada LLM no garantiza una salida idéntica en tiempo, longitud ni estructura; en la mayoría de ejecuciones la respuesta cumple el esquema esperado, pero en otras puede aparecer una salida parcialmente incompatible con los contratos internos, y entonces el pipeline descarta ese intento y reejecuta el nodo. Parte de la variabilidad entre ejecuciones de un mismo dataset no procede, por tanto, del tamaño de la tabla, sino de la necesidad ocasional de repetir una llamada para obtener una salida válida.

Tal comportamiento no se interpreta como un fallo del sistema, sino como una garantía de integridad funcionando correctamente. La arquitectura prefiere asumir el coste de un segundo intento antes que propagar aguas abajo un plan malformado o incompatible con los contratos internos. Esta decisión refuerza el principio de diseño adoptado en el trabajo: los agentes pueden proponer y razonar, pero sus salidas deben superar validaciones deterministas antes de modificar el estado del pipeline.

Por último, los tiempos de los nodos LLM deben interpretarse teniendo en cuenta que

dependen de factores externos al propio código del sistema, como la latencia del proveedor, la carga del servicio y el número de tokens procesados. Por ello, las cifras de la tabla 18 no deben leerse como tiempos absolutos e invariantes, sino como una estimación experimental bajo la configuración utilizada; el coste en tokens, más estable, varía menos de un 20 % entre ejecuciones del mismo dataset. Lo relevante para este trabajo no es tanto el valor exacto de cada latencia, sino la identificación de dónde aparece el coste: los agentes concentran el sobre coste de razonamiento, validación semántica y generación estructurada, mientras que el entrenamiento y la validación de modelos concentran el coste computacional determinista.

5.7. Supervisión humana en las puertas HITL

Dado que el funcionamiento de las puertas de supervisión humana ya se describió en la arquitectura del sistema 4, en esta sección se analiza únicamente su utilidad observada durante las ejecuciones experimentales. En lugar de medir el tiempo exacto de revisión, que depende de factores subjetivos como el nivel de detalle con el que el usuario inspeccione los resultados, se evalúa su papel como mecanismo de control antes de continuar hacia fases costosas o críticas del pipeline.

En las ejecuciones analizadas, las puertas HITL resultan especialmente útiles cuando el sistema ha tomado decisiones que afectan a la calidad del dataset o al estado final del modelo. En casos sencillos, la revisión humana se limita a comprobar que el informe generado es coherente y que no existen advertencias relevantes. En cambio, en escenarios con varios ficheros, joins parciales o imputaciones, la supervisión adquiere mayor importancia: el usuario puede revisar si las claves propuestas, la cobertura de la unión y el conjunto de datos resultante tienen sentido desde el punto de vista del problema, antes de permitir que el entrenamiento continúe.

Dicho comportamiento tiene impacto directo sobre el coste operativo. Si un dataset canónico se ha construido de forma incorrecta, continuar hasta el entrenamiento supondría consumir recursos en modelos que no aportarían valor y que, probablemente, obligarían a

repetir la ejecución completa. La puerta de aprobación permite detener el flujo en ese punto, corregir el contrato de entrada, rechazar un plan de integración dudoso o volver a ejecutar la validación con información adicional. En este sentido, la pausa humana no representa solo una interrupción del flujo, sino una salvaguarda frente a experimentos mal planteados.

La puerta previa a la promoción cumple una función similar en la fase final. Aunque el sistema haya entrenado modelos, calculado métricas y generado un informe de auditoría, la decisión de actualizar el modelo campeón no se ejecuta de forma completamente automática. Esto permite revisar si la mejora observada es suficientemente sólida, si las métricas son coherentes con el contexto del problema y si la ejecución debe aceptarse como nueva referencia o conservarse únicamente como experiencia registrada.

Por tanto, el resultado relevante no es cuánto tiempo tarda el usuario en aprobar o rechazar una ejecución, sino qué tipo de errores o decisiones evita propagar. Las puertas HITL permiten concentrar el criterio experto en momentos concretos del flujo, reduciendo la necesidad de supervisar manualmente todas las operaciones intermedias. El sistema automatiza la validación, la planificación y el entrenamiento, pero mantiene puntos explícitos donde una persona puede bloquear una ejecución incoherente antes de que consuma más recursos o modifique el estado del modelo campeón.

En conjunto, las puertas de supervisión humana refuerzan la filosofía híbrida del sistema. No constituyen una capacidad exclusivamente agéntica, ya que podrían implementarse también en un pipeline determinista, pero adquieren valor al conectarse con informes, evidencias y planes generados por los agentes. De este modo, la arquitectura combina automatización con control experto, evitando que decisiones asistidas por LLM se propaguen sin revisión en puntos especialmente sensibles del pipeline.

6. Conclusiones y líneas futuras

Este trabajo ha abordado el diseño, implementación y validación de un prototipo de sistema MLOps asistido por agentes de inteligencia artificial para la automatización parcial de flujos predictivos. El objetivo principal no consistía en sustituir el trabajo experto ni construir una plataforma AutoML generalista, sino en estudiar cómo los modelos de lenguaje pueden apoyar fases concretas del ciclo de vida de un modelo predictivo manteniendo control, trazabilidad y reproducibilidad.

La principal conclusión es que la combinación de agentes LLM y componentes deterministas resulta adecuada cuando cada parte se utiliza en el lugar correcto, materializando el principio de diseño adoptado durante el trabajo: una arquitectura con columna vertebral determinista y bordes agénticos. Los agentes aportan valor en tareas con incertidumbre semántica —interpretar el contrato de entrada, razonar sobre el perfil de un dataset, proponer joins entre tablas, recuperar experiencias previas, generar planes de entrenamiento o redactar informes de auditoría—, mientras que las operaciones críticas —validación formal, entrenamiento, evaluación, selección del modelo campeón y registro de artefactos— permanecen bajo lógica determinista. El sistema permite que los agentes propongan, razonen y expliquen, pero sus decisiones no modifican el estado del pipeline sin superar contratos estructurados, validaciones programáticas y, en los puntos críticos, revisión humana.

Los resultados experimentales muestran que el Experience Pool puede actuar como una memoria empírica útil para el planificador. Las experiencias almacenadas no funcionan como reglas rígidas, sino como evidencias recuperables que contextualizan nuevas decisiones: el sistema recupera experiencias equivalentes cuando existen, recurre a experiencias similares cuando no hay coincidencia exacta y combina esta información con reglas estáticas y con el perfil del conjunto de datos para generar planes de entrenamiento trazables.

El agente de validación demuestra su utilidad en escenarios con varios ficheros: el descubrimiento asistido de joins combina razonamiento semántico y métricas deterministas de

calidad —cobertura, cardinalidad, duplicación de filas o valores faltantes introducidos por la unión—, algo especialmente relevante en contextos reales donde las fuentes de datos no encajan de forma perfecta. En forecasting, además, la distinción entre variables exógenas conocidas y desconocidas a futuro aproxima mejor el comportamiento a un escenario de despliegue real y reduce el riesgo de filtración de información futura durante la validación. Aunque el prototipo se acota a una única serie objetivo y no aborda forecasting multiserie o panel, la arquitectura establece una base razonable para tratar problemas temporales con variables auxiliares.

El análisis de robustez confirma que el sistema falla de forma controlada ante entradas inválidas: los casos de columna objetivo ausente, objetivo completamente nulo e histórico insuficiente para el horizonte solicitado se detienen antes de entrenar modelos sobre datos incorrectos, apoyando la seguridad del flujo en validaciones deterministas y no en la salida generada por el agente.

Desde el punto de vista operativo, el coste asociado a los agentes se mantiene reducido en los casos evaluados, aunque introduce latencia y variabilidad entre ejecuciones: una misma tarea no garantiza siempre el mismo tiempo de respuesta, consumo de tokens o número de herramientas invocadas, y en ocasiones produce salidas que no cumplen el esquema esperado y obligan a repetir la llamada. El diseño híbrido no elimina esa variabilidad, pero evita que se propague sin control al estado del pipeline. Las puertas de supervisión humana complementan este diseño: permiten revisar datasets canónicos, planes de integración, métricas, informes de auditoría y propuestas de promoción antes de modificar el estado del modelo campeón, concentrando la intervención humana donde aporta mayor valor.

Como limitación principal, el sistema se ha validado como prototipo experimental y no como plataforma industrial completa. El Experience Pool contiene una base inicial de experiencias, suficiente para demostrar el mecanismo de recuperación y trazabilidad, pero todavía limitada frente a la diversidad de problemas de un entorno productivo real. Asimismo, el presupuesto de optimización de hiperparámetros, los límites temporales de ejecución y la infraestructura utilizada condicionan los resultados obtenidos.

En términos cuantitativos, los experimentos realizados permiten resumir el comportamiento del prototipo con cifras concretas. Las ejecuciones completas de forecasting end-to-end, sobre los casos analizados, se completaron en tiempos de cómputo entre 107 y 240 segundos, con un coste monetario por ejecución comprendido entre 0,09 y 0,18 USD. La cuota agéntica del tiempo total osciló entre el 60 % y el 84 %, con la fracción restante atribuible al executor determinista, cuyo peso crece de forma natural con el tamaño del dataset y la complejidad de los modelos candidatos. El Experience Pool inicial recogió 27 experiencias trazables —15 de predicción temporal, 7 de clasificación y 5 de regresión—, suficientes para validar el mecanismo de recuperación por similitud ponderada. En los tres escenarios de fallo inyectado, el pipeline alcanzó un estado terminal controlado antes de entrenar modelo alguno. Por último, la variabilidad observada entre repeticiones de un mismo caso —con diferencias de hasta el doble en tiempo de planificador y consumo de tokens— refuerza que la incorporación de LLMs introduce una incertidumbre intrínseca que la arquitectura no elimina, pero sí acota mediante contratos estructurados, validaciones programáticas y reintentos limitados.

Entre las líneas futuras, la más inmediata consiste en ampliar y curar el Experience Pool con más datasets, dominios y ejecuciones, enriqueciendo la métrica de similitud con campos categóricos adicionales —la tipología de las variables exógenas o la estructura multicíclica de la estacionalidad en series de alta frecuencia— e incorporando mecanismos de recuperación más avanzados cuando el volumen crezca. También sería conveniente optimizar el executor mediante paralelización, límites de tiempo por modelo, reducción adaptativa del espacio de búsqueda y early stopping, así como reforzar la monitorización posterior al despliegue con detección de deriva, alertas de degradación y reentrenamiento automático; más allá, la evolución natural apunta a una integración empresarial completa con gestión de usuarios, permisos, despliegue continuo y gobierno de modelos. Aun así, el prototipo desarrollado demuestra que los agentes LLM pueden aportar valor real en un flujo MLOps si se integran de forma acotada, trazable y supervisada: la aportación principal no reside en automatizar todo el proceso, sino en mostrar una forma viable de combinar razonamiento agéntico, memoria empírica, ejecución determinista y control experto en un mismo pipeline predictivo.



Bibliografía

- [1] Amazon Web Services. What is mlops? <https://aws.amazon.com/what-is/mlops/>, 2026. Accessed: 2026-03-12.
- [2] Xin He, Kaiyong Zhao, and Xiaowen Chu. Automl: A survey of the state-of-the-art. *Knowledge-Based Systems*, 212:106622, 2021.
- [3] Jagadeesan Ganesh. Mlops, llmops and agentops: Operationalizing the future of ai systems. <https://medium.com/@jagadeesan.ganesh/mlops-llmops-agentops-operationalizing-the-future-of-ai-systems-93025dbfde52>, 2024. Medium article. Accessed: 2026-03-13.
- [4] Erik Schluntz and Barry Zhang. Building effective agents. <https://www.anthropic.com/research/building-effective-agents>, 2024. Anthropic Research. Accessed: 2026-04-09.
- [5] Google Cloud. Mlops: Continuous delivery and automation pipelines in machine learning. <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>, 2026. Accessed: 2026-04-09.
- [6] LangChain. Multi-agent workflows. https://langchain-ai.github.io/langgraph/tutorials/multi_agent/multi-agent-collaboration/, 2025. LangGraph tutorials. Accessed: 2026-05-12.
- [7] OpenAI. Openai api reference. <https://developers.openai.com/api/reference/overview/>, 2026. Official API documentation. Accessed: 2026-05-12.
- [8] LangChain. Langgraph overview. <https://docs.langchain.com/oss/python/langgraph/overview>, 2026. Official LangGraph documentation. Accessed: 2026-06-01.



- [9] MachineLearningPlus. Langgraph state management: Typeddict reducers. <https://machinelearningplus.com/gen-ai/langgraph-state-management-typeddict-reducers/>, 2026. Accessed: 2026-06-24.
- [10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [11] European Parliament and Council. Regulation (eu) 2024/1689: Artificial intelligence act, article 14: Human oversight. <https://artificialintelligenceact.eu/article/14/>, 2024. Accessed: 2026-05-12.
- [12] OpenAI. A practical guide to building agents. <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>, 2025. OpenAI business guide. Accessed: 2026-05-12.
- [13] Microsoft. Microsoft agent framework overview. <https://learn.microsoft.com/en-us/agent-framework/overview/>, 2026. Official Microsoft documentation. Accessed: 2026-06-01.
- [14] Microsoft. Microsoft agent framework version 1.0. <https://devblogs.microsoft.com/agent-framework/microsoft-agent-framework-version-1-0/>, 2026. Official Microsoft Agent Framework release announcement. Accessed: 2026-06-01.
- [15] Microsoft. Autogen: A programming framework for agentic ai. <https://github.com/microsoft/autogen>, 2026. Official AutoGen repository. Accessed: 2026-06-01.
- [16] Microsoft. Introduction to semantic kernel. <https://learn.microsoft.com/en-us/semantic-kernel/overview/>, 2026. Official Microsoft documentation. Accessed: 2026-06-01.
- [17] CrewAI. Crewai documentation. <https://docs.crewai.com/>, 2026. Official CrewAI documentation. Accessed: 2026-06-01.
- [18] OpenAI. Data controls in the openai platform. <https://developers.openai.com/api/docs/guides/your-data>, 2026. Official OpenAI API documentation. Accessed: 2026-06-01.



- [19] OpenAI. Openai api pricing. <https://openai.com/api/pricing/>, 2026. Official OpenAI pricing page. Accessed: 2026-06-01.
- [20] GitHub. Github models documentation. <https://docs.github.com/en/github-models>, 2026. Official GitHub documentation. Accessed: 2026-06-01.
- [21] GitHub. Github terms of service: Ai features, training, and your data. <https://docs.github.com/en/site-policy/github-terms/github-terms-of-service>, 2026. Official GitHub Terms of Service. Accessed: 2026-06-01.
- [22] GitHub. Updates to our privacy statement and terms of service: How we use your data. <https://github.blog/changelog/2026-03-25-updates-to-our-privacy-statement-and-terms-of-service-how-we-use-your-data/>, 2026. Official GitHub changelog. Accessed: 2026-06-01.
- [23] Microsoft. Data, privacy, and security for foundry models sold by azure in microsoft foundry. <https://learn.microsoft.com/en-us/azure/foundry/responsible-ai/openai/data-privacy>, 2026. Official Microsoft documentation. Accessed: 2026-06-01.
- [24] Anthropic. Is my data used for model training? <https://privacy.claude.com/en/articles/7996868-is-my-data-used-for-model-training>, 2026. Official Anthropic Privacy Center. Accessed: 2026-06-01.
- [25] Anthropic. Claude api pricing. <https://platform.claude.com/docs/en/about-claude/pricing>, 2026. Official Anthropic API documentation. Accessed: 2026-06-01.
- [26] Ollama. Ollama documentation. <https://docs.ollama.com/quickstart>, 2026. Official Ollama documentation. Accessed: 2026-06-01.
- [27] LM Studio. Openai compatibility endpoints. <https://lmstudio.ai/docs/developer/openai-compat>, 2026. Official LM Studio documentation. Accessed: 2026-06-01.
- [28] Hugging Face. Inference providers. <https://huggingface.co/docs/inference-providers/en/index>, 2026. Official Hugging Face documentation. Accessed: 2026-06-01.



- [29] OpenAI. Models openai api. <https://developers.openai.com/api/docs/models>, 2026. Official OpenAI API documentation. Accessed: 2026-06-01.
- [30] Lei Zhang, Yuge Zhang, Kan Ren, Dongsheng Li, and Yuqing Yang. MLCopilot: Unleashing the power of large language models in solving machine learning tasks. <https://aclanthology.org/2024.eacl-long.179/>, 2024. Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics. Accessed: 2026-06-15.
- [31] George E. P. Box and Gwilym M. Jenkins. Airpassengers time series. <https://vincentarelbundock.github.io/Rdatasets/doc/datasets/AirPassengers.html>, 1976. Serie descrita en Time Series Analysis: Forecasting and Control; obtenida vía statsmodels/Rdatasets. Accessed: 2026-06-14.
- [32] Scripps Institution of Oceanography and NOAA Global Monitoring Laboratory. Atmospheric co2 concentrations, mauna loa observatory. <https://gml.noaa.gov/ccgg/trends/>, 2004. Serie de Keeling; obtenida vía statsmodels.datasets.co2. Accessed: 2026-06-14.
- [33] WDC-SILSO, Royal Observatory of Belgium. Sunspot number: Yearly mean total. <https://www.sidc.be/SILSO/>, 2023. Obtenido vía statsmodels.datasets.sunspots. Accessed: 2026-06-14.
- [34] George W. Cobb. Nile river annual flow dataset. <https://www.statsmodels.org/stable/datasets/generated/nile.html>, 1978. Relacionado con The Problem of the Nile: Conditional Solution to a Changepoint Problem; obtenido vía statsmodels.datasets.nile. Accessed: 2026-06-14.
- [35] UCI Machine Learning Repository. Metro interstate traffic volume data set. <https://archive.ics.uci.edu/dataset/492>, 2019. UCI dataset ID 492. Accessed: 2026-06-14.
- [36] Mitchell O’Hara-Wild, Rob J. Hyndman, and Earo Wang. tsibbledata: Victorian electricity demand. <https://github.com/tidyverts/tsibbledata>, 2022. Dataset VIC2015; datos originales de AEMO. Accessed: 2026-06-14.



- [37] Yahoo Finance. Historical market data. <https://finance.yahoo.com/>, 2024. Series ^GSPC, CL=F, GC=F, BTC-USD, EURUSD=X y variables exógenas descargadas con yfinance; ventana hasta 2024-06-01. Accessed: 2026-06-14.
- [38] Ronald A. Fisher. Iris data set. <https://archive.ics.uci.edu/dataset/53>, 1936. Obtenido vía scikit-learn; publicación original: The Use of Multiple Measurements in Taxonomic Problems. Accessed: 2026-06-14.
- [39] UCI Machine Learning Repository. Wine data set. <https://archive.ics.uci.edu/dataset/109>, 1991. Obtenido vía scikit-learn. Accessed: 2026-06-14.
- [40] UCI Machine Learning Repository. Breast cancer wisconsin diagnostic data set. <https://archive.ics.uci.edu/dataset/17>, 1995. Obtenido vía scikit-learn. Accessed: 2026-06-14.
- [41] OpenML. Heart disease data set: Cleveland. <https://www.openml.org/d/53>, 1988. OpenML data ID 53; datos originales de UCI. Accessed: 2026-06-14.
- [42] OpenML. Titanic data set. <https://www.openml.org/d/40945>, 2017. OpenML data ID 40945. Accessed: 2026-06-14.
- [43] OpenML. Adult census income data set. <https://www.openml.org/d/1590>, 1996. OpenML data ID 1590; datos originales de UCI Adult. Accessed: 2026-06-14.
- [44] Sérgio Moro, Paulo Cortez, and Paulo Rita. Bank marketing data set. <https://www.openml.org/d/1461>, 2014. OpenML data ID 1461; asociado al artículo A Data-Driven Approach to Predict the Success of Bank Telemarketing. Accessed: 2026-06-14.
- [45] Bradley Efron, Trevor Hastie, Iain Johnstone, and Robert Tibshirani. Diabetes data set. https://scikit-learn.org/stable/datasets/toy_dataset.html#diabetes-dataset, 2004. Obtenido vía scikit-learn; asociado al artículo Least Angle Regression. Accessed: 2026-06-14.
- [46] Athanasios Tsanas and Angeliki Xifara. Energy efficiency data set. <https://archive.ics.uci.edu/dataset/242>, 2012. UCI Energy Efficiency; asociado al artículo Accurate Quantitative Estimation of Energy Performance of Residential Buildings Using Statistical Machine Learning Tools. Accessed: 2026-06-14.



- [47] I-Cheng Yeh. Concrete compressive strength data set. <https://archive.ics.uci.edu/dataset/165>, 1998. UCI Concrete Compressive Strength; asociado al artículo Modeling of Strength of High-Performance Concrete Using Artificial Neural Networks. Accessed: 2026-06-14.

- [48] Hadi Fanaee-T and João Gama. Bike sharing data set. <https://www.openml.org/d/42712>, 2014. OpenML data ID 42712; datos procedentes de UCI Bike Sharing. Accessed: 2026-06-14.

- [49] R. Kelley Pace and Ronald Barry. California housing data set. https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset, 1997. Obtenido vía scikit-learn; asociado al artículo Sparse Spatial Autoregressions. Accessed: 2026-06-14.

Anexo A. Reproducción y despliegue de la aplicación

Este anexo describe los pasos para desplegar el sistema completo —servidor de seguimiento MLflow, backend FastAPI con los agentes LangGraph y la interfaz web Next.js— en una máquina nueva. Se utiliza Docker por su facilidad y capacidad de reproducibilidad de entornos. Además, la organización del repositorio y la descripción de cada carpeta se documentan en el fichero `README.md` incluido en la raíz del proyecto.

A.1. Requisitos previos y variables de entorno

La única dependencia obligatoria para la vía Docker es Docker Desktop.

La configuración se lee de un fichero `.env` en la raíz del proyecto. Para crearlo, copie la plantilla incluida `.env.example` a un nuevo fichero `.env` y adáptelo rellenando al menos la variable `OPENAI_API_KEY`.

La tabla 19 resume las variables relevantes. La única imprescindible es la clave de la API del proveedor de modelos; el resto tienen valores por defecto sensatos.



Variable	Obligatoria	Descripción / valor por defecto
OPENAI_API_KEY	Sí	Clave de la API de OpenAI (o <i>token</i> personal de GitHub).
OPENAI_BASE_URL	No	<i>Endpoint</i> alternativo; p. ej. GitHub Models (Sección A.2).
OPENAI_MODEL_OVERRIDE	No	Fuerza un mismo modelo en todos los agentes, ignorando el declarado en sus <i>prompts</i> . Vacío por defecto.
MLFLOW_TRACKING_URI	No	<code>http://mlflow:5000</code> (Docker) / <code>sqlite:///./mlflow.db</code> (local).
MLFLOW_EXPERIMENT_NAME	No	<code>mlops-agents</code> .
LOG_LEVEL	No	INFO.

Tabla 19: Variables de entorno principales (`.env`).

Los modelos se asignan por nodo en los ficheros de *prompt* de cada agente: `gpt-5.4-mini` para el validador de datos y el planificador (que exigen un seguimiento preciso de instrucciones) y `gpt-5.4-nano` para el redactor de informes. Como alternativa gratuita a la API de pago, el sistema también puede ejecutarse contra GitHub Models (Sección A.2).

A.2. Ejecución gratuita con GitHub Models

Para no obligar a contratar la API de pago de OpenAI, el sistema conserva la posibilidad de ejecutarse contra GitHub Models, el servicio gratuito de inferencia de GitHub. Como el acceso al LLM se realiza mediante un cliente compatible con la API de OpenAI, no se requiere ningún cambio en el código del sistema; basta con:

1. Generar un *token* personal de GitHub (sin permisos especiales) y usarlo como valor de `OPENAI_API_KEY`.
2. Apuntar el cliente al *endpoint* de GitHub Models mediante la variable de entorno `OPENAI_BASE_URL`.
3. Sustituir el modelo de cada agente —definido en los ficheros de *prompt* `src/mlops_agents/prompts/` (`data_agent`, `planner`, `report_writer`)— por uno dis-

ponible en el catálogo de GitHub Models (p. ej. `gpt-4.1-mini`) o forzar el modelo directamente en todos los agentes mediante la variable `OPENAI_MODEL_OVERRIDE`.

La contrapartida es doble. Por un lado, la capa gratuita impone límites de uso —cuotas por minuto y por día que varían según el modelo—. Por otro, y de mayor relevancia, GitHub Models acepta la petición de salida estructurada pero no garantiza el cumplimiento estricto del esquema (*structured output*) del que depende el planificador: en la práctica, la etapa de validación de datos se completa con normalidad, pero el planificador puede devolver una salida que no se ajusta al contrato `PlannerOutput` y, en tal caso, agotar sus reintentos y abortar la ejecución. Por ello, para ejecutar el pipeline de extremo a extremo y reproducir las cifras de la Sección 5 se recomienda la API de OpenAI; GitHub Models se ofrece como vía de exploración sin coste.

A.3. Despliegue con Docker

Desde una instalación limpia:

```
git clone https://github.com/mariovuniovi/tfg_agentes_ia_langraph.git
cd tfg_agentes_ia_langraph
# copie .env.example a un fichero .env y rellene OPENAI_API_KEY
docker compose up --build # construye las imagenes y arranca los 3 servicios
```

El comando levanta tres contenedores sobre una red privada compartida (`mlops-net`), descritos en la tabla 20. Una vez en marcha, la aplicación es accesible en `http://localhost:3000`.



Contenedor	Puerto	Función
mlops-mlflow	5000	Servidor de seguimiento MLflow (<i>runs</i> , métricas, artefactos y registro de modelos). Persiste en el volumen <code>mlflow_data</code> .
mlops-api	8000	<i>Backend</i> FastAPI + agentes LangGraph. Invoca al LLM, ejecuta el <i>pipeline</i> y expone la API REST/SSE.
mlops-frontend	3000	Interfaz Next.js. Consume el <i>backend</i> en <code>http://localhost:8000</code> .

Tabla 20: Servicios desplegados por `docker compose`.

El backend y MLflow se comunican internamente por el nombre de red `mlflow:5000`; el navegador usa siempre `localhost:puerto`. Operaciones habituales:

`docker compose up # arrancar (sin reconstruir si no hay cambios)`

`docker compose up --build # reconstruir tras cambios de código`

`docker compose down # detener (conserva los volúmenes de datos)`

`docker compose down -v # detener y borrar el volumen de MLflow (reset total)`

`docker compose logs -f api # ver logs de un servicio`

A.4. Sembrado del pool de experiencias

El planificador recupera priors de un pool de experiencias previas. Dicho pool se distribuye ya sembrado con el repositorio: la base de datos `storage/mlops_metadata.db` contiene los `ExperienceRecord` de los datasets de benchmark (Anexo Anexo B), por lo que la aplicación dispone de priors desde el primer arranque sin ningún paso adicional.

Solo es necesario regenerarlo si se desea reconstruir el pool desde cero. En ese caso, el runner offline descarga los conjuntos públicos (scikit-learn / OpenML / Yahoo Finance / CSV locales), entrena el ejecutor sobre cada uno y reescribe la base de datos:

`uv run python scripts/run_benchmark.py --trials 4`



Este paso es determinista y no realiza llamadas al LLM, pero requiere conexión a internet para descargar los datasets reales.

A.5. Verificación

Una vez desplegado, la verificación funcional consiste en abrir `http://localhost:3000`, subir un dataset y comprobar que el pipeline recorre los nodos hasta la puerta HITL de despliegue.



Anexo B. Procedencia de los conjuntos de datos

Los conjuntos de datos empleados en la evaluación (sección 5) se dividen en dos categorías. Los conjuntos reales proceden de fuentes públicas y se citan mediante su referencia académica canónica cuando existe, o bien mediante su repositorio de distribución (UCI Machine Learning Repository, OpenML, statsmodels o Yahoo Finance). Los conjuntos sintéticos se generaron específicamente para este trabajo con el fin de aislar comportamientos concretos del sistema (escalado por tamaño, descubrimiento de joins, fallos controlados); no admiten cita externa y su reproducibilidad queda garantizada por los scripts generadores con semilla fija incluidos en el repositorio. En la tabla 21 se recogen los conjuntos de datos reales empleados junto con su fuente y referencia.



Dataset (memoria)	Fuente	Cita
<i>Pronóstico (tabla 10)</i>		
Air Passengers	statsmodels (Box & Jenkins)	[31]
CO ₂ Mauna Loa	statsmodels (Scripps/NOAA)	[32]
Manchas solares	statsmodels (WDC-SILSO)	[33]
Río Nilo	statsmodels (Cobb, 1978)	[34]
Metro Minneapolis	UCI 492	[35]
Electricidad Victoria	tsibbledata / AEMO	[36]
S&P 500	Yahoo Finance	[37]
Petróleo WTI	Yahoo Finance	[37]
Oro + Macro	Yahoo Finance	[37]
Bitcoin (BTC)	Yahoo Finance	[37]
EUR/USD	Yahoo Finance	[37]
<i>Clasificación (tabla 11)</i>		
Iris	scikit-learn (Fisher, 1936)	[38]
Vino	UCI / scikit-learn	[39]
Cáncer de mama	UCI / scikit-learn	[40]
Enfermedad cardíaca	OpenML 53 (Cleveland)	[41]
Titanic	OpenML 40945	[42]
Renta adultos	OpenML 1590 (Kohavi)	[43]
Marketing bancario	OpenML 1461 (Moro et al.)	[44]
<i>Regresión (tabla 12)</i>		
Diabetes	scikit-learn (Efron et al.)	[45]
Eficiencia energética	UCI (Tsanas & Xifara)	[46]
Resistencia hormigón	UCI (Yeh, 1998)	[47]
Alquiler bicicletas	OpenML 42712 (Fanaee-T & Gama)	[48]
Vivienda California	scikit-learn (Pace & Barry)	[49]

Tabla 21: Conjuntos de datos reales: fuente y referencia. Las series financieras comparten una única referencia de distribución (Yahoo Finance).

Los casos de estudio basados en errores del pipeline y los datasets utilizados para validar el descubrimiento de joins se generaron mediante scripts propios y, por tanto, no requieren



cita externa. Dentro de este grupo se incluyen Revenue mensual (`small_monthly_revenue`), Bakery diario (`medium_daily_bakery`) y Factory por horas (`large_hourly_factory`), que corresponden a series univariantes sintéticas de tamaño controlado.

También pertenece a este conjunto Grid Demand (`grid_demand`), junto con los ficheros auxiliares `calendar_registry`, `generation_mix` y `meteorological`. Este caso se diseñó como un escenario multi-fichero para demostrar el descubrimiento automático de joins, sin representar datos reales de una red eléctrica.

Por último, se incluyen los samples de join y cobertura parcial, como `join_partial_coverage`, así como los datasets de fallos controlados agrupados bajo `broken/*`. Todos ellos forman parte del conjunto de datos sintéticos empleados para evaluar comportamientos específicos del pipeline.